

Flight Dynamics Bible
Volume I:
6 Degree of Freedom Modeling

HUDSON REYNOLDS
Author

PRESTON WRIGHT
Co-Author

September 2025
Second Edition

Part of a



Flight Dynamics

Series

This document is dedicated to Purdue Space Program Liquids. PSP Liquids has given us a vast amount of experience and means to explore the realm of Flight Dynamics. We hope that we can contribute to the wealth of information provided by PSP with this document.

Contents

1	Overview	1
1.1	Document Outline	1
1.2	Necessary Background	2
1.3	Why 6-DoF?	3
1.4	The General Picture	5
2	Newtonian Dynamics	7
2.1	The 1-DoF Case	7
2.2	Frame of Reference	14
2.3	Derivative Transport	19
2.4	The 3-DoF Case	22
2.5	Euler Rotation Equations	27
2.6	Examples	32
2.7	Key Ideas	34
2.8	Further Reading	35
2.9	Practice Problems	35
	Notes	38
3	Energy Methods of Mechanics	41
3.1	Hamilton's Principle	42
3.2	Generalized Coordinates	47
3.3	Hamiltonian Mechanics	56

3.4	Lagrangian Derivation of Rigid Body Dynamics	57
3.5	Key Ideas	59
3.6	Further Reading	60
3.7	Practice Problems	60
	Notes	63
4	Computational Attitude Dynamics	65
4.1	Euler Angles	65
4.2	Quaternions	69
4.3	3D Rotation Example: Dzhanibekov effect	73
4.4	Key Ideas	76
4.5	Further Reading	77
4.6	Practice Problems	77
	Notes	80
5	Aerodynamic Modeling	83
5.1	Non-Dimensionalization	83
5.2	Aerodynamics Terminology	86
5.3	Atmospheric Modeling	89
5.4	Computational Aerodynamics	90
5.5	Key Ideas	91
5.6	Further Reading	91
5.7	Practice Problems	91
	Notes	93
6	Numerical Integration Schemes	95
6.1	Euler Integration	95

6.2	Improved Euler / RK2	98
6.3	RK4 Integration	101
6.4	Other Numerical Integration Considerations	103
6.5	Other Integration Schemes	107
6.6	Numerical Integration Code Optimization	109
6.7	Key Ideas	111
6.8	Notes and Further Reading	111
	Notes	112
7	The 6-DoF	115
7.1	State Vector	115
7.2	Forces and Moments in the 6-DoF	117
7.3	Validation of 6-DoF	122
	Notes	124
8	MATLAB and Coding Practices	127
8.1	Object Oriented Programming in MATLAB	127
8.2	Continuous Integration and Testing	133
8.3	Data Visualization	134
	Notes	137
	Appendices	137
A	Answers to Selected Problems	139
A.1	Chapter 2 Solutions	139
A.2	Chapter 4 Solutions	142
B	Proofs and Derivations	145

B.1	Buckingham Pi	145
B.2	Equivalence of Newtonian and Lagrangian Mechanics	146
C	Code	149
C.1	Lorenz Attractor RK4 Example	149
C.2	Dzhanibekov Effect Model	150
C.3	Numerical Integration Error Examples	150
	Special Terms	151
	References	159

Code Listings

1	Script Header	xvii
2.1	1 DoF	13
4.1	Euler Angle Dynamics Example	68
4.2	Quaternion Dynamics Example	71
6.1	RK2 Integrator	100
6.2	RK4 Integrator	102
7.1	Rotation Matrix	118
8.1	Class Example	129
8.2	Class Constructor Example	130
8.3	Class Header	133
8.4	Startup File for Plotting	135
A.1	Mass Spring Damper Solution	141
A.2	Huey Landing Solution Code	143

Preface

Preston and I began working on the Flight Dynamics sub-team in the fall of 2023. From May 2024 to January 2025, we focused our efforts on creating a 6-Degree of Freedom (DoF) model from scratch. During this time, we learned a lot about the mathematics and physics involved in creating such a model. We were often faced with frustration with different conventions and poor explanations of certain topics, all of which were exacerbated by the difficulty of what we were learning. This made much of the material intractable. In this document, we hope to provide detailed explanations of all the necessary concepts in one place to make this journey easier for those who may need it in the future.

Furthermore, we hope that this document can be used by those outside the realm of flight dynamics to better understand the methodologies and inner workings of a 6-DoF model. Often, we have seen that a 6-DoF model is treated as a ‘black box’, where inputs go in and outputs come out. We hope that this document can provide a level of understanding adequate enough to allow greater collaboration between those working within and outside of flight dynamics.

In this document, we will be using excerpts of MATLAB script in some examples (code less than a page is included inline and longer code is in the Appendix) but our hope is that our explanations are thorough enough to facilitate the creation of a 6-DoF in any coding language. MATLAB facilitates the use of quite short code because of the number of built-in functions, but we include derivations for most of these equations or sources describing these derivations.

To use this document to its fullest extent, we recommend that you write scripts as you learn and use this document for reference and as a guide as you build or improve your own models. Doing the practice problems in each chapter will also greatly aid in your understanding.

In this first volume, we cover primarily the translational dynamics and the basics of rotational dynamics needed to set up a 6-DoF model. We mainly focus on the implementation of these equations in MATLAB and best practices. The methods of deriving equations of motion are also heavily discussed for their importance. These are, in our opinion, the fundamental things that need to be understood before working

on other aspects of a flight dynamics model. As such, these subjects comprise the large majority of both the effort and the length of the first volume.

Because of the depth of the subjects described here, we have decided to cover most of attitude dynamics in a separate volume. In this volume, we mostly describe how attitude dynamics can be implemented in MATLAB. Volume II contains most of the theory about attitude dynamics. This second volume is especially useful for the mathematically inclined reader or those who want to implement attitude dynamics math in another programming language.

As a note, while our work on flight dynamics has mainly focused on modeling rockets, we hope that this series can be used more widely for a variety of 3-DoF and 6-DoF models. As such, we have attempted to include content that is useful for other systems, such as aircraft modeling and orbital dynamics, insofar as we have experience in these areas. That being said, to limit the scope of this document, we have decided to explore these concepts in the context of rocket modeling, and hope that our explanations provide enough clarity for the intrepid reader to explore modeling in other domains.

Some other areas are also covered in this document but are curtailed because Preston and I do not have the experience to give full authority on these subjects. However, as we learn more, these volumes continue to grow. In Volume II and III, we expand and add many of these other sections, especially those related to attitude dynamics, energy methods, and control theory, to better serve those in the future who may hope to recreate the work we have done here.

Preface to the Second Edition

In this second edition of Volume I, we have reworked many of the figures and the text to be more clear. There has also been an effort to reduce the amount of jargon and improve the pedagogical effectiveness of the book. Solutions to some of the problems have additionally been provided.

New content, specifically related to object-oriented programming, has been emphasized in the end of this volume. Some of the sections on energy methods of mechanics has been reduced and placed in Volume III instead. Material on derivative transport (otherwise known as the Basic Kinematic Equation) which was previously omitted has now been included in this volume.

User Guide

For those readers who want to learn both the theory and the practical implementation of a 6-DoF, we recommend reading the entirety of Volume I, and supplementing sections of Volume II and III as required.

For those who have requisite knowledge but want information on implementation of 6-DoF into MATLAB, [Chapter 4](#), [Section 5.4](#), [Chapter 6](#), and [Chapter 8](#) are recommended.

For those interested solely in Attitude Dynamics, [Chapter 4](#), Volume II, and Part I of Volume III (Particularly the section on Lie Theory) are suggested. If the reader's interests are in spacecraft dynamics in general, Part II of Volume III is additionally suggested.

For those interested in control system design and analysis, we recommend [Chapter 2](#), [Chapter 3](#), Part V and VI of Volume II, and Volume III (with an emphasis on Part III).

For those interested in holding an integral role on PSP Flight Dynamics team (such as Responsible Engineer / Subject Matter Expert or Team Lead), we suggest thorough reading and understanding of all three Volumes and practical skills in implementation into MATLAB.

With that said, we hope every reader can obtain useful knowledge from these three volumes!

Acknowledgments

- We would like give special thanks to Prof. Cunningham and Prof. Frueh for their help in reviewing our 6-DoF Model. Professor Frueh's notes have been especially helpful in compiling the information present in this document. We would also like to thank everyone who has taken their time to read through this document and help us make corrections and edits.

Notation and Standards

Mathematical notation for the subjects that we discuss is largely formalized and commonly used, but there are some areas where we may use notation slightly unfamiliar or different to what you have seen in the past. You may also see different notation when looking at papers and literature for future work on the 6-DoF. To remove any doubt, we describe the notation that we use here at the beginning for quick reference.

We also choose to use notation that is used in the Purdue AAE classes. Some of these may be unfamiliar if you have not encountered this coursework yet, but we do our best to describe notation throughout this document and in this section.

For derivatives with respect to time, we will use the compact Newtonian notation. For example, $\dot{x} = \frac{dx}{dt}$, $\ddot{x} = \frac{d^2x}{dt^2}$, and so on. Generally, we avoid the prime notation such as x' as this notation does not make clear what we are taking the derivative with respect to. The prime notation is only used in specific circumstances where the other notation is easily applicable.

For vectors, any vector with a hat like $\hat{x}$ refers to a vector with a magnitude of 1 (normalized vector). These are often seen as basis vectors or unit vectors describing the direction of a force. Vectors with a ‘arrow’ hat like $\vec{x}$ refers to a general vector with a non-normalized magnitude. Vectors with a non-normalized magnitude may also be represented by a bold-faced symbol. When taking the time derivative of a vector, we prefer to write using this boldface notation, such as $\dot{\mathbf{x}}$ instead of $\dot{\vec{x}}$.

Vectors and Matrices

The whole of the vector quantities used in this document are too great to enumerate in full, but we include the most important here. Notably, we stray from the typical notation of $\vec{r}_{P/O}$ in favor of $\vec{r}^{op}$ because of its usage in Purdue AAE.

Common usages are documented here:

$\vec{F}$	force
$\vec{a}$	acceleration
$\vec{V}$ or $\vec{v}$	velocity
$\vec{r}^{op}$	position vector from o to p
$\vec{X}$	state vector
$\vec{M}^o$	moment about point o
${}^i\vec{x}$	inertial vector quantity
$\vec{\omega}$	angular velocity
$\vec{\alpha}$	angular acceleration

Scalar Quantities and Symbols

J	functional
δ	variation
S	action
L	Lagrangian
q	generalized coordinate
$\dot{q}$	generalized position
k	spring constant
$\mathcal{M}$	number of degrees of freedom
T	kinetic energy
U	potential energy
$\mathcal{H}$	Hamiltonian
Re	Reynolds number
M	Mach number
α	angle of attack
C_x	aerodynamic force coefficient
Δt	timestep

Coding Standards

An ideal 6-DoF not only models the motion of a vehicle through the atmosphere as accurately as possible but is also well-structured and easily readable. Strict adherence to an agreed upon and appropriate set of coding standards is essential for a

successful program. These standards cover everything from naming conventions to heading structure, and make the integration of codes from multiple contributors more seamless.

For short pieces of code written inline, we use a mono-spaced font with the appearance of `code snippet` as shown. Longer code is included in listings with line numbers using the same mono-spaced font.

Generally, we use the following conventions in our coding practice:

- Variables utilize camelCase where possible
- Symbolic variables may use underscores
- File names, functions, and classes utilize PascalCase

MATLAB script file headers are of the following format:

Code Listing 1: Script Header

```
% PSP FLIGHT DYNAMICS:
% Title:
% Author:
% Last Modified:
%
% Description:
%
% Inputs:
%
% Outputs:
```


CHAPTER 1

Overview

“If you wish to make an apple pie from scratch, you must first invent the universe.”

– Carl Sagan

In much the same way, we must construct all of the mathematical background required to make a 6-DoF from the ground up. This chapter details the general requirements and an outline of the process we will be following.

1.1 Document Outline

A general overview of the scope of this document is given in the preface. Here, we will go into more detail about the layout of each section. Our goal is to present this document in a similar manner to a textbook while still remaining approachable and interesting to read.

Our pedagogical approach is strongly reliant on examples and trying problems. We find that this is the best method for readers to learn the content. In each chapter (aside from this overview chapter), there will be some additional content at the end that may be especially useful for quick reference. For quick reference, we include a ‘Key Ideas’ section that may be used to quickly reference the big takeaways from each chapter.

Additionally, a ‘Further Reading and Notes’ is included which the curious reader may benefit from. Many topics are covered in this document; each of which deserve their own book-length discussion. Because we have neither the desire nor capability to write such a long document, we have linked these beneficial resources whenever

possible. It is certainly not necessary to read any/all of these sources to comprehend the content here, but they may prove useful for deeper research into topic areas that interest the reader.

The end of some sections also provides several practice problems to ensure the key ideas of each topic are understood by the reader. These problems are mainly to validate the readers understanding and provide an example of using the presented content.

For readers new to this content, it is expected that some problem may take some consultation with other sources to complete. However, we have attempted to make these problems challenging while not being out of reach. These problems are designed to make the reader apply the knowledge and truly learn. Generally, each subsequent problem in a section is more challenging than the last.

1.2 Necessary Background

The goal of this document is to provide a comprehensive overview of the process and means of creating, maintaining, and building on a 6 Degree of Freedom model. Although we strive to make the document as easy as possible to understand, there are some instances where technical jargon is necessary and a strong understanding of mathematical concepts is required.

The first time an important technical jargon is used, we will *italicize* the word for recognition. It is important to learn these terms to effectively and concisely describe your work. That being said, we attempt to keep the usage of jargon to a minimum to make this text approachable. A glossary with these terms has also been added so that all of these words are included in a single location.

To aid in comprehensibility, we have added sources that we find especially useful and avoided the use of superfluous sources. These sources are attached in the bibliography for ease of access. In addition, useful links and footnotes have been highlighted to be easily located.

To fully comprehend and successfully use this document, it is recommended that you have a good understanding of calculus, linear algebra, and calculus-based physics. A rudimentary understanding of differential equations and dynamics will also prove useful but is not strictly required. Topics covered in AAE 203 / Introduction Dy-

namics will be used.

Some concepts will arise in this document that are derived from graduate-level coursework and complex mathematics. In these instances, we may not attempt to derive the background and full understanding, but rather only what is necessary for building a 6-DoF model. In these cases, we also try to link to source material if you are inclined to learn more about those topics.

1.3 Why 6-DoF?

Let's start off with some terminology in this section. A Degree of Freedom (DoF) refers to the number of ways an object can move in space. A more technically precise and rigorous definition defines this as the number of independent parameters that define the motion of a system. This concept is explored more deeply in [subsection 3.2.1](#).

When modeling rigid body dynamics, it is common to see 1-DoF, 3-DoF, and 6-DoF models all used in aerospace applications. Each of these are useful constructs, and sometimes a simulation with more degrees of freedom is not necessarily better, depending on the application.

1.3.1 1-DoF

The first modeling that is often done for the sizing of rockets may use 1-DoF modeling. In this type of model, there is only one axis along which an object may translate or rotate about. In this case of rocket modeling, the only degree of freedom is the altitude of the rocket. The goal of this type of model is mainly to provide a heuristic for determining rocket performance in the very preliminary stages of sizing by estimating the maximum altitude they can achieve.

Another example of this type of model is a simple mass-spring system along one axis. Such models may be useful for estimating the vibrations of bodies or as a first-order approximation of fuel slosh.

We will use the example of a 1-DoF in [Section 2.1](#) to explain the creation of equations of motion in the simplest way. However, these models are often too constrained to give the most valuable information about a system, which is why 3-DoF and 6-DoF

models are more often used, especially in the context of flight dynamics.

1.3.2 3-DoF

After 1-DoF, the next step up is often a 3-DoF model. This type of model should be familiar to all flight dynamics members, as it is the subject of our onboarding project. In a 3-DoF model, there may be any combination of translational degrees of freedom and rotational degrees of freedom.

Often, 3-DoF modeling is utilized when either the rotational dynamics or translational dynamics of an object are considered unimportant. One example of this may be the calculation of an interplanetary transfer. The primary concern of the simulation is not the orientation of the satellite, but rather the location and velocity of the satellite and planets at various points in 3D space.

Another example of this type of model is planar motion. Often, problems in engineering are constrained in some way. Very often, we can consider one of the translational degrees of freedom and two of the rotational degrees of freedom to be non-important to the problem. This simplification allows us to greatly reduce the scope of the problem. One consequence of this action is that rotations are only permitted in a single axis. With two translational and one rotational degree of freedom, we can create useful models of many systems without great complexity.

Lastly, we see 3-DoF models used when we are considering only the rotational dynamics of a system. For objects such as gyroscopes and other spinning bodies, we may choose to model rotational motion in this way.

Although the rest of this document will mainly discuss 6-DoF models, really consider if you need that full complexity for your model! Often, a 3-DoF or 4-DoF model is perfectly adequate for a wide variety of problems without the hassle incurred by a 6-DoF model.

1.3.3 6-DoF

6-DoF models are the most complex representation of a single rigid body mechanical system. In 3D-space, there are only 6 possible degrees of freedom, and we simulate the dynamics in all of them.

Doing so is quite a difficult challenge, but it is not insurmountable. With careful planning, it is possible to gain an appreciation and understanding of the dynamics of a full 3D system. Doing so allows unrivaled modeling potential. In the case of a rocket, the primary reason that modeling in 6-DoF is so important is because it allows us to analyze the aerodynamic stability of the system. In both 1-DoF and 3-DoF modeling, this important aspect of the system is largely neglected.

1.3.4 Structure Hierarchy of a 6-DoF

The structure of a 6-DoF consists of three main, large “blocks” of code that are executed in sequence. The first is the initialization section. This is where all known, fixed parameters of the vehicle, atmosphere, and physics are initialized to be called later in the program. The second section is the integrator. This block utilizes a certain integration scheme to step through time, calculating the derivatives of the vehicle’s *state vector*. Within the integrator, there are three major sub-blocks corresponding to major aspects of the dynamics: the forces, the moments, and the attitude dynamics. The forces section takes on the more familiar particle dynamics – regarding position, velocity, and acceleration - that are seen in a high school physics class. The moments section takes on the torques and rotational forces acting on the system. Lastly, there is attitude dynamics – the orientation of the vehicle in space. This can take the form of either Euler Angles or Parameters, which will be discussed further later. The final block is the visualizer block. This block writes pertinent information to matrices, animates the movement of the vehicle from launch to land, and outputs graphs of important metrics for the user to visualize.

Each of the previously mentioned sections will be discussed in more detail throughout this document.

1.4 The General Picture

It is often useful to see where we are going – and our general method of getting there – before we start. With this ‘map’ at our disposal, it is much easier to get through the complexities of the mathematics involved in creating a 6-DoF model.

The essence of the 6-DoF is to simulate the movements of a system through time. To do so, we need some description of how the system may respond to certain forces.

We will spend quite a while discussing how these forces arise and how we may use them to understand the subsequent motion of a vehicle. However, as many of you likely remember from your physics classes, forces acting on a rigid body or more generally about a point give rise to moments. These moments impart some angular motion on our system, which motivates the study of mathematically describing orientation due to these rotations. After understanding how to describe both the translational and rotational motion of a rigid body, the dependence of aspects of these equations on variables we must integrate to solve form. This conundrum is mostly avoided through numerical integration, which allows us to approximate the motion of our vehicle governed by these derived equations. Before this; however, we must also consider how to accurately model the aerodynamic forces such as lift and drag on our vehicle. While equations are known, certain parameters within these equations are themselves functions of multiple other parameters, meaning computer simulations are required to accurately determine them. Finally, at the very end, we will be discussing the 6-DoF itself and the coding practices used in its construction. To help you better understand the framework, we have included a diagram below for a visualization.

Talk about forces into moments, rotations, and why numerical integration is required. Mention aerodynamic forces. Include diagram similar to the one shown in the code review.

CHAPTER 2

Newtonian Dynamics

“I can calculate the motion of heavenly bodies, but not the madness of people.”

– Isaac Newton

The first step when approaching any physics problem of this type is to consider the Newtonian dynamics of the system. In our case, we define Newtonian dynamics as the translational dynamics of the system as well as the moments acting on the system.

Our definition strays from particle dynamics because we are considering a rigid body which can exhibit rotation. The exploration of this rotation will be explored in much more detail in [Chapter 3](#), [Chapter 4](#), and Volume II and III. Here, we will introduce the methods used in our future analysis.

2.1 The 1-DoF Case

To begin our exploration of Newtonian dynamics, we will return to the example of the 1-DoF which we mentioned earlier. Our goal with this is to use a simple example that you are familiar with from high school physics and bring it up to speed with the terminology and techniques we will use for more complex analysis.

In [Figure 2.1](#), we show an example of a very simple 1-DoF system free body diagram. We will use this example to explain the process of calculating the Newtonian dynamics in 1-dimension, which we then generalize to the full 3D case.

A total of three forces are shown, all of which act along the $\hat{y}$ -direction. Because

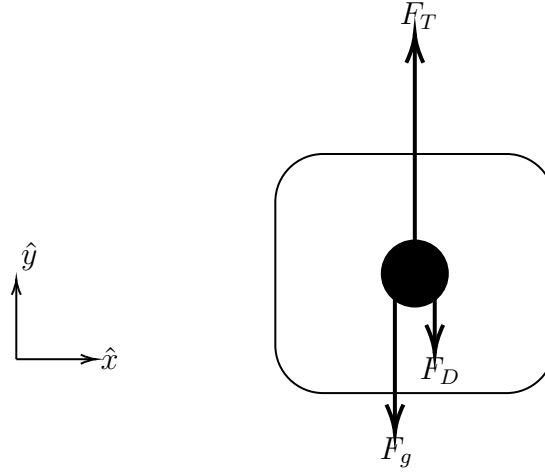


Figure 2.1: 1-DoF Forces on a body

all of the forces lie along the same direction, they are collinear, and the dynamics can be described with only one axis. This feature is what makes this a 1-DoF model. The three forces shown are the thrust force, the drag force, and the gravity force.

Following what you may do in an introductory physics class, we will describe each of these forces via their functional form. These are described in [Equation \(2.1\)](#):

Force	Functional Form	
$\vec{F}_g$	$-mg$	(2.1)
$\vec{F}_D$	$-\frac{1}{2}\rho_\infty V_\infty^2 S C_D$	
$\vec{F}_T$	$F_T \cdot [1 - u(t - t_b)], t \geq 0$	

The notation here is important to understand. In future AAE classes at Purdue, this is likely the notation that you will see. Terms with the subscript ∞ denote *freestream* quantities, those that are freely flowing far away from the body of interest. Also of note is the unit step function, $u(t - t_b)$, which is a function which has value zero until reaching time t_b , where it thereafter equals one. The quantity ρ refers to the density of the fluid, and S to the *reference area*. The *drag coefficient* is C_D , a term which encapsulates the very complex nature of the fluid flow around an object. Calculating the value of C_D will be explored more deeply in later sections of this document. For now, we may assume a constant value.

It is helpful to think about these quantities as vectors, because we can add them together to achieve a resultant.¹ In the 1-dimensional case this is quite trivial since

all vectors lie along the $\hat{y}$ unit vector, but this will become increasingly important as we move to higher dimensions.

2.1.1 Numerical Integration in the 1-DoF

Now we will move into something that you not have seen before. Given all the forces, we apply Newton's 2nd Law, $\sum \vec{F} = m\vec{a}$, to arrive at an expression for the acceleration, known as an Equation of Motion (EOM), as shown in (2.2). Doing so might look something like this:

$$\begin{aligned} F_T \cdot [1 - u(t - t_b)] - \frac{1}{2}\rho_\infty V_\infty^2 SC_D - mg &= m\vec{a} \\ \vec{a} &= \frac{F_T \cdot [1 - u(t - t_b)] - \frac{1}{2}\rho_\infty V_\infty^2 SC_D}{m} - g \end{aligned} \quad (2.2)$$

From here, we use integration to find the velocity and then position of the object with time. Formally, we might define this formally as:

$$\vec{v} = \int \vec{a} dt$$

However, you may notice something strange with our equation. In our expression for acceleration, we need to know the velocity to compute the drag. However, we do not know the velocity without integrating the equation first. Since this dependence is non-linear, this becomes even more complicated to solve. In general, there is no known analytical solution to this paradigm! We must take a different approach to the problem. ²

The way we resolve this is to make an approximation of the solution in a process called *numerical integration*. Here, we will outline the simplest type of numerical integration, known as Euler's Method. This section aims to show how to implement Euler's method in code, and the mathematical reasoning will be given a more formal treatment in [Section 6.1](#). We will describe more complex numerical integration schemes and why you might want to use them, but it's good to see the code for a simple case first.

Euler's Method involves discretizing our time interval into many small pieces. You may remember the concept of a Riemann sum from calculus, where you approximate an integral in discrete time steps. We will take a similar approach with Euler's Method. Euler's method is applicable to solving first order ordinary differential equations (ODE's). However, in our problem, we have a second order differential equation, because $\vec{a} = \ddot{y}$.

To resolve this issue, we will make this one differential equation into a system of first-order differential equations. In general, we can convert an n th-order ODE into a system of n first-order ODEs. In this case, we define a variable, $v = \dot{y}$. Now, we can express equation (2.2) as a system of two differential equations:

$$\begin{aligned} \dot{v} &= \frac{F_T \cdot [1 - u(t - t_b)] - \frac{1}{2}\rho_\infty V_\infty^2 SC_D}{m} - g \\ \dot{y} &= v \end{aligned} \tag{2.3}$$

Now, we will apply Euler's method to solve the problem. We will use the following steps to do so, using equation (2.3) as our example:

1. Rewrite the system in Leibniz notation, where $\dot{x} = \frac{dx}{dt}$.
2. Perform separation of variables, to arrive at $dx = v \cdot dt$.
3. Discretize the equation by 'converting' dx and dt to Δx and Δt .³
4. Rewrite Δx as $x_{new} - x_{old}$. We can rearrange the equation as $x_{new} = v \cdot \Delta t + x_{old}$
5. To start, we define an initial state. This initial state will be the first value of x_{old} .
6. Iterate through values of Δt until the desired time of simulation is achieved.

What we have done is formally called writing a *difference equation*. Now only is this important for numerical integration, but in digital processing, because computers must discretize a signal! We will see these equation again when we study practical control systems in Volume III.

We follow the same process to write the difference equation for the integration of acceleration to find the velocity. Of note, we will use the previous value of $\vec{v}$ in the computation of the new acceleration.

Example 2.1: Discretizing the velocity equation

Since the position-velocity relation is quite simple in this situation (it almost always will be) it is helpful to see how we discretize the equation for the velocity-acceleration relationship. We will follow the same steps as before. We will, however, reduce the equation

$$\frac{F_T \cdot [1 - u(t - t_b)] - \frac{1}{2}\rho_\infty V_\infty^2 SC_D}{m}$$

to the more compact form

$$\frac{F_T - F_D}{m}$$

We can then start with the same steps as before.

1. Rewrite in the Liebniz notations:

$$\frac{dv}{dt} = \frac{F_T - F_D}{m} - g$$

2. Perform separation of variables:

$$dv = \left(\frac{F_T - F_D}{m} - g \right) dt$$

We notice that F_D has a dependence on v , but there is no way to isolate it to the other side of the equation. We will just leave it as is.

3. Next, we discretize the equation:

$$\Delta v = \left(\frac{F_T - F_D}{m} - g \right) \Delta t$$

4. We expand the Δv . We leave Δt the same since that is our timestep. Generally, we want to write an equation that has v_{new} isolated on the LHS of the equation. Starting with the expansion, we have:

$$v_{new} - v_{old} = \left(\frac{F_T - F_D}{m} - g \right) \Delta t$$

We remember that F_D is a function of v . We will show this by writing $F_D = \kappa v^2$. The variable κ is used since all other parts of the drag are independent of velocity. Since we don't know v_{new} , we have to use v_{old} as the value of v in the drag equation.

So, we can do a rearrangement of the equation to arrive at the final form:

$$v_{new} = \left(\frac{F_T - \kappa v_{old}^2}{m} - g \right) \Delta t + v_{old}$$

Lastly, we define v_0 to be 0 for most cases. The iteration through values of Δt is the exact same as before. As we can see, the only real difference that is incurred is that we use the value of v_{old} in our drag expression! Otherwise, this integration looks nearly identical to the example with velocity.

We should note that this method of numerical integration is not used too often in practical applications. However, writing difference equations *is* quite important, so it's still a good skill to have nonetheless.

In future applications of numerical integration, the most important skill to have is to be able to construct the EOM and to separate the equation into first order equations.

If you still feel slightly unsure about some details, note that a more formal definition of Euler's method is given in [Section 6.1](#). This may help clear up any confusion here.

We also show this Euler Integrator example in MATLAB [Script 3](#) below. Note that $\frac{1}{2}\rho_\infty S C_D$ is called k in the script for simplicity. This simplification assumes that ρ_∞ and the C_D are constant, which we will later see is a quite poor assumption for rocket modeling but is okay for a first approximation.

Code Listing 2.1: 1 DoF

```

% Euler Method 1DoF

%constants:
m = 1;      % mass [kg]
g = 9.8;    % gravity [m/s^2]
k = 1e-3;   % drag const [kg/m]
tb = 5;     % thrust time [s]
Ft = 100;   % force of thrust [N]

% define constant forces
Fg = -m*g;

% pos and vel init:
x = 0;
v = 0;
t = 0;

% array init:
i = 1;

% timestep definition:
dt = 0.01;

while v >= 0
    % variables updated every loop
    Fd = -k*v(i)^2;
    Ft = Ft * (1-heaviside(t(i)-tb));

    % Euler Integration:
    a(i+1) = Ft+Fg+Fd;
    v(i+1) = a(i)*dt + v(i);
    x(i+1) = v(i+1)*dt + x(i);

    % Iteration update
    i = i+1;
    t(i) = i*dt;
end

```

```
% plot result:
figure(1)
plot(t,x)

figure(2)
plot(t,v)
```

There are a few important things to note about the MATLAB implementation of the script. Firstly, the only force that is defined outside of the loop is gravity, because the magnitude and direction of this force is independent of the current state of the system. We contrast this with the drag force and the thrust, which must be calculated on every iteration through the algorithm to find an updated value for the force.

We also note the use of the `heaviside` function. This is functionally identical to a unit step function, just a different notation.

2.2 Frame of Reference

Before attempting to describe movement in multiple dimensions, we should first discuss how we can describe important quantities through different lenses. Later, it will become apparent that a vector or problem is easier described or completed through one lens than another. We call these different lenses reference frames.

A frame of reference is an *orthonormal* set within three-dimensional space. The orthonormal nature of the basis vectors that create every frame of reference allows for any other vector in 3-dimensional space to be described as a linear combination of those three basis vectors (hence why we call it a basis for 3-space). This tool becomes incredibly useful within the study of Newtonian and Attitude Dynamics, as every arbitrary vector can be broken down into three set “scalar” directions. Naturally, the question arises of how we determine which set directions to utilize, and so we will be discussing a few frames of reference critical to the creation of a 6-DoF and aerospace engineering as a whole.

2.2.1 The Inertial Frame

The first and most important frame of reference is the *inertial frame*. Typically denoted by e or i , or in our case $(\hat{x}, \hat{y}, \hat{z})$, this motionless frame is the one that allows for the use of Newton’s second law in the first place, as the frame itself has “zero” translational and rotational movement.⁴ We say “zero” as this motion must be negligible in relation to the vehicle of interest. For example, our specific 6-DoF sets the inertial frame to be anchored within the Earth on the launch pad. And while we are well aware the Earth is moving and rotating through space at a non-zero rate, this motion is negligible relative to the motion of our vehicle which remains close to Earth within the atmosphere for a short period of time. Therefore, we can assume that the Earth acts appropriately as an inertial frame of reference, and Newton’s second law is valid for this frame.

Returning to the specific example of a 6-DoF, the directions you set your basis vectors to point in are mostly left to the coder’s discretion. If your chosen directions retain the orthonormal nature of a frame of reference, as well as obeying the right hand rule (RHR), the choice is all yours. Within the example we are following, our inertial frame’s origin is anchored within the launch pad as previously stated. The first inertial basis vector points up perpendicular to the Earth’s surface, the second inertial basis vector points due East, while the third inertial basis vector follows the right-hand rule and points due North. An image for quick reference is provided in [Figure 2.2](#):

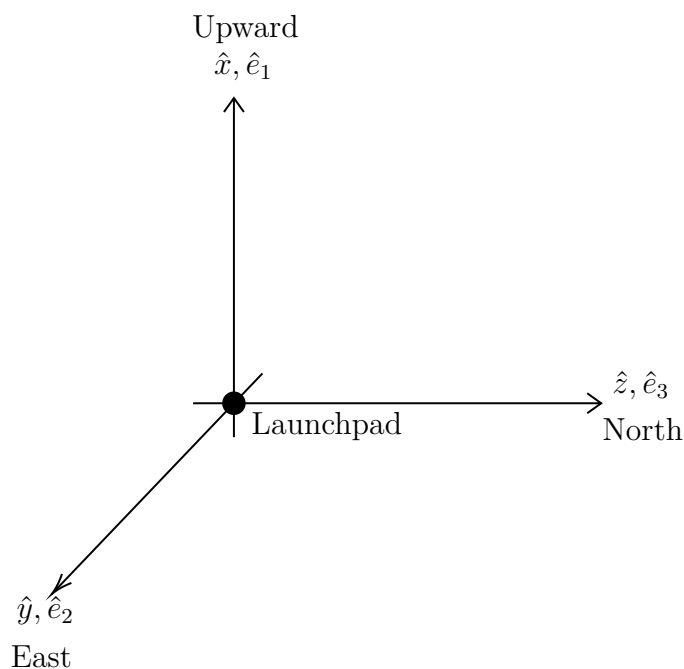


Figure 2.2: Inertial Frame Convention

2.2.2 The Body Frame

The second most important frame of reference is the *body frame*. Typically denoted by b , or in our case $(\hat{X}, \hat{Y}, \hat{Z})$, this frame of reference is always anchored at some point in the vehicle and allows us to describe the state vector in relation to the vehicle itself. This frame is moving and rotating with respect to the inertial frame. For every body frame you create – from a frame for the entire vehicle to a frame for a small electronic part – you want simplicity. This means that in aerospace, it is often the case for a general body frame to have a basis vector pointing along the longitudinal axis of the vehicle, as well as one out a wing. A generic image of such a convention is provided in [Figure 2.3](#) In addition, it means at a zero angle of rotation, every basis vector of the body frame is parallel to the corresponding basis vector in the inertial frame. This is imperative to rotational dynamics, as the offset of each basis vector in the body frame will correspond to the pitch, yaw, and roll of the vehicle. But more on that in Volume II.

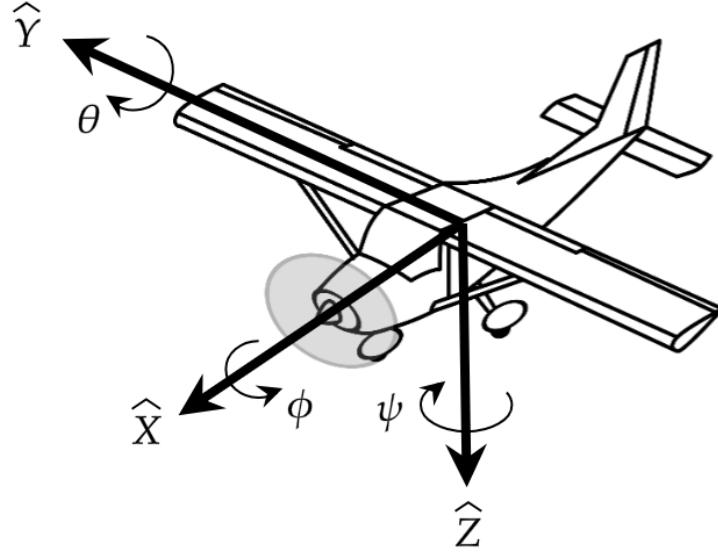


Figure 2.3: Typical Body Frame Orientation

For our current model, the body frame of our 6-DoF has its origin in the nose of the vehicle. It has the first body basis vector, $\hat{X}$, pointing up along the longitudinal axis of the rocket following standard convention. The second basis vector, $\hat{Y}$, points right due East when on the launch pad, and the third, $\hat{Z}$, points due North on the launch pad. We use the cardinal directions of the launchpad as not only are there no conventional wings on the rocket, but it allows the body frame to be defined parallel to the corresponding inertial frame basis vectors. Note that they won't remain pointing in these directions as the rocket moves through space, and they may not even begin parallel to the inertial axis to begin with if the rocket launches from a tilt.

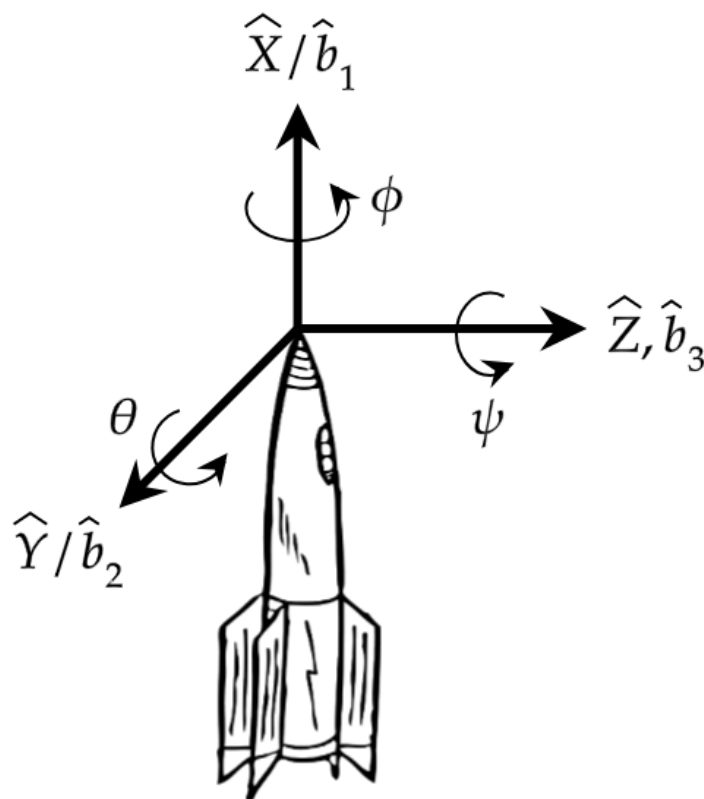


Figure 2.4: Body Frame Convention of our Rocket

2.2.3 The Wind Frame

The last and by far least important frame of reference is the *wind frame*. This frame of reference, like the body frame, is always anchored at some point in the vehicle. Also like the body frame, it moves and rotates with the vehicle and allows for a new way of describing the state vector. Unlike the body frame, the most important definition is not along the longitudinal axis of the vehicle. As indicated by its name, the necessary orientation of a basis vector points in the direction of the free stream velocity.

Relating this to the body frame, this translates to an offset from the body frame by the *angle of attack* and sideslip. This frame is often useful for aircraft, but we do not use it explicitly in our 6-DoF. The wind frame itself is in fact skipped over during wind calculations, as the angle of attack and imparted forces/moments are calculated

directly onto the body frame. So as this isn't as useful as the other two reference frames, we won't elaborate on this frame any further, and it won't be shown as much in our example 6-DoF.

2.3 Derivative Transport

The last issue that we tackle when different reference frames get involved is the fact that objects are moving around, specifically accelerating, so it can be hard to express quantities with just an inertial frame. We can create various reference frames, but to correctly apply Newton's second law, we must use our inertial frame as the original reference.

A reference frame may very well translate a vector from one frame to another, but transforming a derivative requires a bit more subtlety. The tool we use to do this 'derivative frame translation' is called derivative transport, or the Basic Kinematic Equation (BKE). The BKE, then, is just a tool that we use to convert derivatives between various reference frames.

Example 2.2: The Frisbee

To understand the BKE, let us consider a scenario. You are out on a lovely day with your dog, Hans. Hans loves to play Frisbee with you. Of course, when you throw a Frisbee, you apply some spin to make sure the Frisbee remains stable as it is flying. The motion of the Frisbee is now quite complex. There is translational motion, but the Frisbee is also rotating about its vertical axis. What should you do if you want to find the velocity of a point at the edge of the Frisbee?

We will use some of our tools with reference frames to help us. We will establish an inertially fixed frame, $\hat{e}$. This frame will stand still and gives us a 'ruler' to measure everything against.

We also want a reference frame that is embedded in the frisbee, which we may call $\hat{a}$. We can embed the $\hat{a}$ -frame in the Frisbee such that $\hat{e}_3$ and $\hat{a}_3$ are both pointing out of the page. We can construct a visual of this, as shown in [Figure 2.5](#). In this figure, we can see that the Frisbee has some rotation with respect to the $\hat{e}$ -frame.

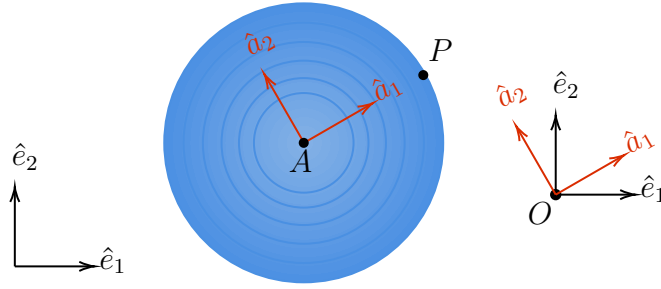


Figure 2.5: Derivative Transport on a Frisbee

Our end objective is to describe the position, velocity, and acceleration of this point P on the Frisbee. We may start by describing the position, which is the easiest to handle. Say we want to describe the location of point P on the Frisbee. We can do this by expressing the location of point A from the origin in the $\hat{e}$ -frame, and then go the rest of the way in the $\hat{a}$ -frame. That is to say, we can describe the position of the center of mass (A) and then it is simply a translation along the $\hat{a}_1$ -direction to arrive at point P . Doing so would look like:

$$\vec{r}^{OP} = \vec{r}^{OA} + \vec{r}^{AP}$$

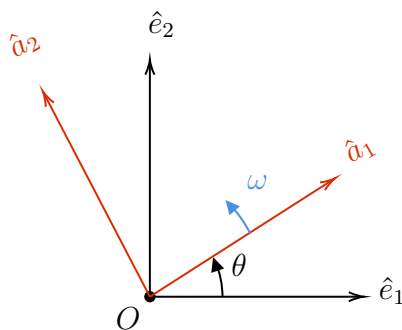
Let us attach some variables to this to make it more tangible. Say we have thrown the pendulum directly north, along the $\hat{e}_2$ direction, by 30 m. This gives us $\vec{r}^{OA}$, which is $30\hat{e}_2$. $\vec{r}^{AP}$ is computed similarly. Taking R as the radius, this vector is $R\hat{a}_1$. Altogether:

$$\vec{r}^{OP} = 30\hat{e}_2 + R\hat{a}_1$$

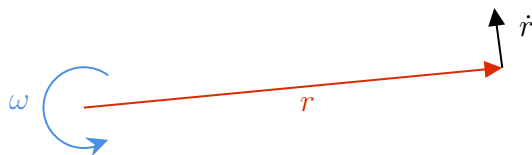
Some complication comes when we try to express the velocity of point A . We know that the velocity is the time derivative of the position, but our position is being described using non-inertial coordinates in the $\hat{a}$ frame!

Using our standard derivative, there is no machinery that allows us to take this derivative in the $\hat{a}$ -frame. We can solve this by computing the rotation of one reference frame with respect to the other. This will give us the ability to compute a valid derivative.

Going back to our frisbee, we can notice that $\hat{a}$ and $\hat{e}$ are offset by some angle. If we know this angle, and the rate at which it is changing, we can understand the relationship between the $\hat{a}$ and the $\hat{e}$ frame.

**Figure 2.6:** Angle Between Frames

So we have an angle between the two frames, θ , and an angular rate, ω . We want to find an expression for the infinitesimal change in $\hat{a}$.

**Figure 2.7:** Infinitesimal rotation

Considering an infinitesimal rotation, r will be displaced in a direction that is orthogonal to its current direction due to the rotation by ω . We can see that this $\dot{r}$ is orthogonal to both r and ω (which is pointing out of the page). Following the RHR, this direction must be given by $\omega \times \vec{r}$.

This cross product also gives us an appropriate magnitude for the vector, as the magnitude will increase with a larger ω or if the length of the lever arm, r , increases.

There are more fundamental reasons for this cross-product relationship, but these will have to wait until Volume II and III to be explained by Lie theory.

With the rotation considered, the rest is quite simple. We must consider the change of r itself, so we consider the derivative of r with respect to the $\hat{a}$ -frame

to do so. Altogether, we have the full BKE:

$${}^e \frac{d}{dt} \vec{r} = {}^a \frac{d}{dt} \vec{r} + {}^e \vec{\omega}^a \times \vec{r}$$

Basic Kinematic Equation

In general, we can use the BKE to express how frame translations affect the derivatives of states. We can perform analysis using multiple intermediate frames to build a sequence of rotations, or apply the BKE twice to get an acceleration.

2.4 The 3-DoF Case

2.4.1 Vectors in the 3-DoF Case

For particle dynamics, the most complex case is full 3D translational motion. Luckily, nothing very fundamental changes as compared to the 1-DoF case. The most important difference is the use of vector notation for compactness and clarity of mathematics and code. For position ($\vec{r}$), velocity ($\vec{v}$), and acceleration ($\vec{a}$), we represent them as tri-dimensional column vectors:

$$\vec{r} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \vec{v} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix}, \vec{a} = \begin{bmatrix} \ddot{x}_1 \\ \ddot{x}_2 \\ \ddot{x}_3 \end{bmatrix}$$

This is done not only for the sake of compactness, but also to facilitate the use of matrix operations for translation between reference frames. This concept is further explored in [Section 2.2](#). For this reason, expressing these elements in vector form will become crucial as we move forward. In MATLAB, we express a tri-dimensional column vector as:

```
pos = [0;0;0];
```

Where each semicolon represents a new row of the vector. We define a row vector by using a comma instead of a semicolon:

```
pos = [0,0,0];
```

Often, it is useful to convert between a row and column vector in MATLAB because some functions will expect the input to be in a different format. We can do so with a transposition. In MATLAB, this looks like:

```
pos = [0, 0, 0]';
```

Expressed in vector notation, we write Newton's 2nd Law in the 3D case as $\vec{F} = m^i \ddot{\vec{r}}^{op}$. The presence of i indicates that this acceleration vector must be in the inertial frame. The quantity $^i \ddot{\vec{r}}^{op}$ can be deduced from the use of the BKE in simple cases.

2.4.2 Vector directions in the 3-DoF

We present below a modified version of the 1-DoF below in full 3D in [Figure 2.8](#).

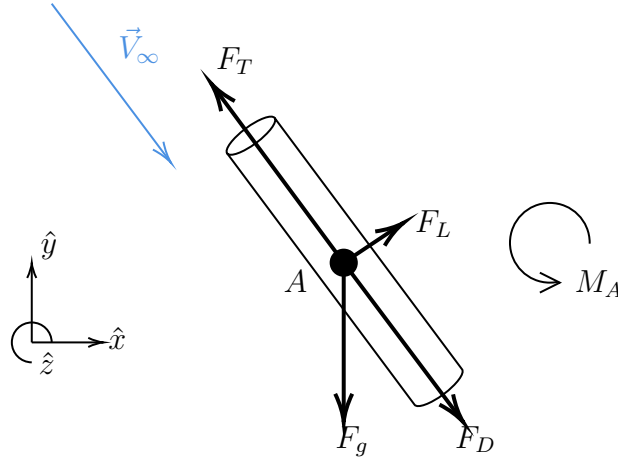


Figure 2.8: 3-DoF Forces on a Body

As compared to 1-DoF case, we have three new quantities that are present. The first of these is V_∞ , the freestream airflow vector.⁵ This is represented diagrammatically because it is no longer constrained along the $\hat{y}$ direction.

When we have an angle between the nose (the direction through which the thrust force points in [Figure 2.8](#)) and the freestream velocity, we refer to this as an angle of attack. This angle of attack is often denoted with the letter α . We will discuss angle of attack and its effects more in [Chapter 5](#). For now, we simply need to understand how to calculate α given the vector through the nose and the freestream velocity vector. We show the angle of attack diagrammatically in [Figure 2.9](#).

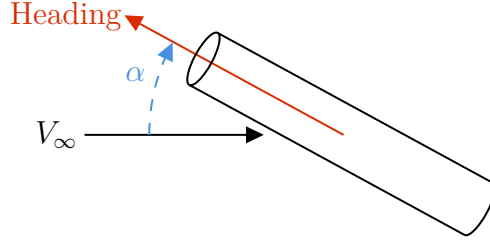


Figure 2.9: Angle of Attack

To do this, we use the fact that the dot product is related to the cosine of the angle between the vectors. Denoting the vector through the nose of the rocket $\hat{X}$, we can find the angle of attack as:

$$\alpha = \cos^{-1} \left(\frac{\hat{X} \cdot \vec{V}_{\infty}}{\|\hat{X}\| \|\vec{V}_{\infty}\|} \right) \quad (2.4)$$

The next new quantity is $\vec{F}_L$, the force of lift. The force of lift is always perpendicular in direction to V_{∞} . Knowing this, we can find the direction of the force of lift as:

$$\hat{L} = \frac{(V_{\infty} \times \hat{X}) \times V_{\infty}}{\|(V_{\infty} \times \hat{X}) \times V_{\infty}\|} \quad (2.5)$$

Here, the cross product is used because it generates a vector orthogonal to the two input vectors. This is exactly the property that we need when defining lift.

For the sake of completeness, we should also note that the force of drag lies opposite the direction of $\vec{V}_{\infty}$. This is much simpler to calculate, and looks like:

$$\hat{D} = \frac{-\vec{V}_{\infty}}{\|\vec{V}_{\infty}\|} \quad (2.6)$$

The last force directions to define are the direction of thrust and the direction of gravity. Luckily, these are easily defined because they point directly along basis vectors.⁶ The force of gravity is defined with respect to the inertial reference frame and is defined in (2.7). The force of thrust is defined with respect to the vector pointing through the nose, $\hat{X}$, and is shown in (2.8):

$$\vec{F}_g = -mg \cdot \hat{y} \quad (2.7)$$

$$\vec{F}_T = F_T \cdot \hat{X} \quad (2.8)$$

2.4.3 Moments

The last new quantity is the moment about point A, denoted $\vec{M}^A$. For now, we will simply note that for an arbitrary selection of point A, we will have a rotational moment that is generated because the forces on the body do not necessarily act through point A, even if it is the center of mass (see section 4 for more details). In [Figure 2.8](#), we have drawn all of the forces acting through a single point for simplicity, but this is not generally the case.

For our rocket modeling, it is generally assumed that gravity and thrust act about the center of gravity, and the aerodynamic forces about the center of pressure.

We will show how to compute this moment if the location of the center of gravity and the center of pressure is known. To show this, we will briefly return to the 2D case for illustration, but keep in mind that this concept is extensible in 3D. We show this in [Figure 2.10](#).

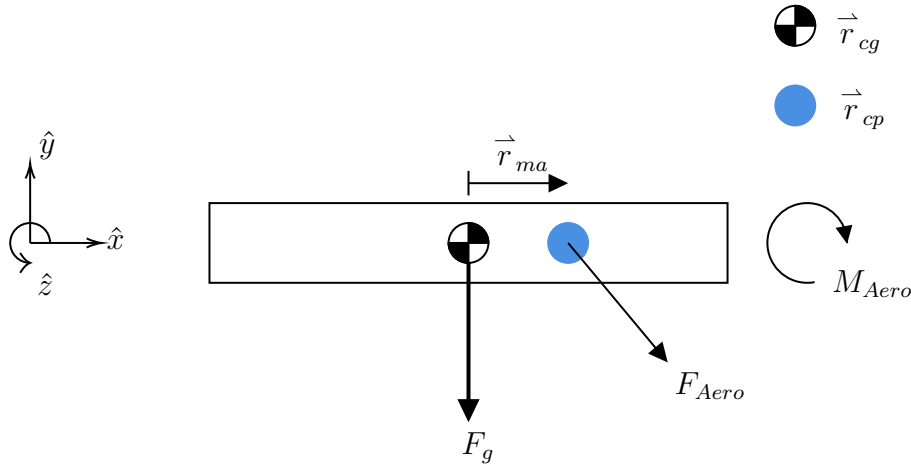


Figure 2.10: Moment Demonstration

In [Figure 2.10](#) we include a few new symbols. All force magnitudes, however, are equivalent to what we show in [Figure 2](#) (with the assumption that they lie in the x-y plane for this example). We denote the location of the center of gravity as $\vec{r}_{cg}$ and the location of the center of pressure as $\vec{r}_{cp}$.⁷ The difference between $\vec{r}_{cg}$ and $\vec{r}_{cp}$ is denoted as $\vec{r}_{ma}$. We refer to this as the moment arm of the aerodynamic forces. It should be noted that all of locations are tridimensional vectors.

We also define a new vector, $\vec{F}_R$, the resultant aerodynamic vector, which is the

vector sum of $\vec{F}_D$ and $\vec{F}_L$. Because free rotations occur about the center of mass, it is most helpful to calculate our final moment from this location. To do so, we use the moment equation:

$$\vec{M}^{cg} = \vec{r}_{ma} \times \vec{F}_R$$

We note that the direction of this moment is in the $-\hat{z}$ direction from the RHR. If we assume that our forces are planar, this moment will generate a rotation about a single axis. This makes our model into a 3-DoF.

One thing to note is that some quantities are more easily defined with respect to vectors that are defined with respect to the vehicle (which we will call the body frame), such as the vector $\hat{X}$. As seen in Equation (2.4) and (2.5), we often want to use these vectors in the body frame for computation. As such, we will need a method in which we can convert between the body and inertial frames. This conversion between frames is explored in Section 2.2. We should also note that our moments will be in this body frame as well, which turns out to be especially useful in Section 2.5.

The last thing that we note in this section is that our forces and moments are coupled. This means that the aerodynamic forces are a function of the orientation of the vehicle, so the moments and forces must be solved as a system.

2.4.4 State Vector

As a last note for this section, we will briefly discuss the concept of a state vector. The state vector is a column vector that contains information on the current state of our system. In the example of our 3-DoF, we will include the position, velocity, rotation, and rotation rate of the system in the state vector. This would look like⁸:

$$\vec{X} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\theta} \end{bmatrix}$$

It is also useful to define the derivative of our state vector, which would just be the derivative of each of its elements. This is expressed as

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\theta} \\ \ddot{x} \\ \ddot{y} \\ \ddot{z} \\ \ddot{\theta} \end{bmatrix}$$

For the sake of compactness, these vectors are generally not written in their full form. In this text, we will often refer to a state vector as just $\vec{X}$. Note that we also use the vector $\hat{X}$ to denote a unit vector in the body frame, so the difference in the hat becomes an important distinction here.

2.5 Euler Rotation Equations

In [subsection 2.4.3](#) we have explored the moments that are created when a force acts at a point that is not through the center of mass. Here, we will make the jump from particle dynamics into *rigid body* dynamics.

In analogy to Newton's 2nd Law of motion, there is an equally important relationship for rotational dynamics. This equation is known as Euler's 2nd Law:

$$\boxed{\vec{M}^o = \frac{{}^i d\vec{H}^o}{dt}} \quad (2.9)$$

Euler's 2nd Law

This equation reads as “The moment about point O is the inertial time derivative of the angular momentum, H , about point O .” The superscript i refers to the fact that this derivative must be taken in the inertial frame.

We also have the relationship between angular momentum, $\vec{H}$, and the *moment of inertia*, I , for an rigid body that you have likely seen in physics classes:

$${}^i \vec{H} = {}^i (I\vec{\omega}) \quad (2.10)$$

You will notice that our moment of inertia here is defined with respect to the inertial frame. As a result, when our body changes orientation, the moment of inertia may also change!

Often however, it is more helpful to define this moment of inertia in terms of our body frame. We will go through this derivation of the moment of inertia below.

2.5.1 Moment of Inertia and Inertia Tensor

The moment of inertia of a body is a measure of its resistance to rotational acceleration. It is in essence, the rotational analogue of mass (mass is a measure of resistance to linear acceleration, by the same line of reasoning).

Since we need the moment of inertia to describe our relationship for angular momentum in (2.10), it is useful for us to see its derivation here.

We can start by showing that the mass of a region can be found by integrating the density over its volume. Mathematically, we show this as:

$$\iiint_V \rho dV$$

Similarly, we know that the moment of inertia of a particle is mr^2 , which is it's mass multiplied with the length of the moment arm, r , squared. This is often referred to as the 'second moment' of mass. By integrating this second moment, we have:

$$\iiint_V \rho r^2 dV$$

It is useful to define the moment of inertia along each direction, though. We can rewrite r in terms of x , y , and z for each of the component directions:

$$\begin{aligned} I_{xx} &= \iiint_V \rho (y^2 + z^2) dV \\ I_{yy} &= \iiint_V \rho (x^2 + z^2) dV \\ I_{zz} &= \iiint_V \rho (x^2 + y^2) dV \end{aligned} \tag{2.11}$$

For our rocket, we will find that these are the only quantities that we need to define. We also take define these quantities with respect to the center of mass since this is the point about which free rotations occur.

However, in general, we also have three more quantities, which are called the product of inertia. This is defined by the product of two of our elements, such as xy . This will have the same dimensions as the quantity r^2 , so it can be applied in a similar way to the moment of inertia. The product of inertia is defined as follows:

$$\begin{aligned} I_{xy} = I_{yx} &= \iiint_V -xy\rho dV \\ I_{xz} = I_{zx} &= \iiint_V -xz\rho dV \\ I_{yz} = I_{zy} &= \iiint_V -yz\rho dV \end{aligned}$$

For our rocket, because of the symmetry about the $\hat{X}$ axis, these quantities go to zero. However, for future implementations that may include fuel slosh and asymmetries, the products of inertia may be non-zero!

We express our *inertia tensor*, denoted I , as the 3×3 matrix which contains the moments and products of inertia.

$$I = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}$$

For our rocket, we get the simplification that:

$$I = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix}$$

In fact, we can always find axes such that the products of inertia are zero. These are called the *principal axes* and are important in other applications, such as structures [2]. For our rocket, it just happens that these principal axes are the ones we have chosen to define as our body frame. The moment of inertia along these principal axes are solutions to the eigenvalue problem $I\vec{\omega} = I^*\vec{\omega}$, where I^* is a scalar. Essentially, we are finding a matrix I such that the output is just a scalar multiple of the input, ω . We can see that having this property will greatly simplify our results. This is another reason why we like to perform many calculations in the body frame.

Using the inertia tensor, a matrix form of (2.10) is written as:

$$\vec{H} = I\vec{\omega} \quad (2.12)$$

In the simplification using the principal axes, we often refer to the terms I_{xx} , I_{yy} , and I_{zz} simply as I_x , I_y , and I_z . There are called the principle moment of inertias (PMOIs).

2.5.2 Parallel Axis Theorem

Similar to the way in which we described the moment of inertia of a particle to be mr^2 , we can describe the change in moment of inertia of a body due to the offset of its axis.

We accomplish this with the *parallel axis theorem*, which relates the moment and products of inertia through different rotation points. To use parallel axis theorem, we simply take the known moment of inertia and add mr^2 along that component direction. This looks like:

$$I_{x'x'} = I_{xx} + m(y^2 + z^2)$$

This is similarly computed for the other axes, with the r always being the axis that is not the one we are computing the moment of inertia of. We can think of this intuitively because we are translating the center along the two axes perpendicular to the one describing the moment of inertia.

We also must do the same for the products of inertia. We note that even if we select body axes with no products of inertia, we may still have them in the translated center!

The parallel axis theorem for the products of inertia is given as:

$$I_{x'y'} = I_{xy} - mxy$$

This is similarly computed for the other axes.

2.5.3 Rotations in the Body Frame

Now that we have defined our inertia tensor in the body frame, it is useful to reexamine (2.9) and (2.10) to express them in terms of the body frame.

We can express equation (2.9) in terms of the body frame using the BKE. We write this as:

$${}^i \frac{dH^o}{dt} = {}^b \frac{dH^o}{dt} + {}^e \vec{\omega}^b \times H^o \quad (2.13)$$

In the case of principal axes, (2.12) can be simplified to yield:

$${}^i H^o = I_x \omega_x \hat{b}_1 + I_y \omega_y \hat{b}_2 + I_z \omega_z \hat{b}_3$$

Plugging this result into (2.13), we get the following:

$${}^i \frac{dH^o}{dt} = {}^b \frac{d}{dt} (I_x \omega_x \hat{b}_1 + I_y \omega_y \hat{b}_2 + I_z \omega_z \hat{b}_3) + {}^e \vec{\omega}^b \times (I_x \omega_x \hat{b}_1 + I_y \omega_y \hat{b}_2 + I_z \omega_z \hat{b}_3)$$

Where ${}^e \vec{\omega}^b$ is $\omega_x \hat{b}_1 + \omega_y \hat{b}_2 + \omega_z \hat{b}_3$. Assuming a rigid body along with a principal axis system, we say the time derivative of our inertia tensor is zero. The algebraic manipulation and vector math here is left as an exercise for the reader. Collecting the final terms, our final expression is:

$${}^i H^o = [I_x \dot{\omega}_x + (I_z - I_y) \omega_y \omega_z] \hat{b}_1 + [I_y \dot{\omega}_y + (I_x - I_z) \omega_x \omega_z] \hat{b}_2 + [I_z \dot{\omega}_z + (I_y - I_x) \omega_x \omega_y] \hat{b}_3$$

Equating this with the moments in the body frame that were found in subsection 2.4.3 as ${}^b M^o = M_x \hat{b}_1 + M_y \hat{b}_2 + M_z \hat{b}_3$, we arrive at the final form of our equations for Euler rotations⁹:

$$\begin{aligned} M_x &= I_x \dot{\omega}_x + (I_z - I_y) \omega_y \omega_z \\ M_y &= I_y \dot{\omega}_y + (I_x - I_z) \omega_x \omega_z \\ M_z &= I_z \dot{\omega}_z + (I_y - I_x) \omega_x \omega_y \end{aligned}$$

Euler Rigid Body Equations

We can rearrange this to a form that is more useful for us (although somewhat less sightly), putting the unknown $\dot{\omega}$ (we also refer to this quantity as the angular acceleration, $\vec{\alpha}$) quantities alone:

$$\begin{aligned} \dot{\omega}_x &= \frac{M_x - (I_z - I_y) \omega_y \omega_z}{I_x} \\ \dot{\omega}_y &= \frac{M_y - (I_x - I_z) \omega_x \omega_z}{I_y} \\ \dot{\omega}_z &= \frac{M_z - (I_y - I_x) \omega_x \omega_y}{I_z} \end{aligned} \quad (2.14)$$

These sets of equations are especially powerful because they allows us to numerically integrate the angular velocity. Our moments can be found using Newtonian dynamics and the PMOIs are quantities that are easily found using the integrals described above in subsection 2.5.1. We will use these angular velocity derivatives extensively in Volume II to describe the time rate of change of the attitude of the rocket.

2.6 Examples

We've just laid out a large amount of material, so it is best to show a few examples to get the gist of things.

Example 2.3: Explorer 1 Satellite

NASA asks you to help analyze the rotational dynamics of the Explorer 1 satellite for your job interview. The satellite can be modeled approximately as a cylinder with a length of 2 m and a diameter of 16 cm. Assume the mass is 16 kg with a uniform mass distribution.

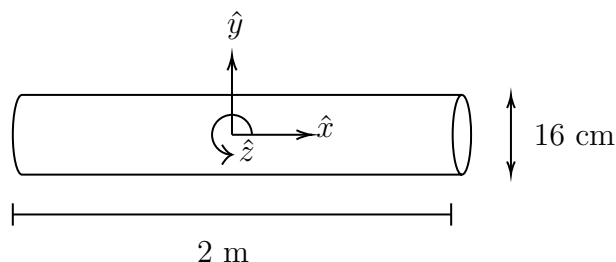


Figure 2.11: Explorer 1 Satellite

Moments of Inertia

To start analyzing the rotations, we'll first need to compute the moments of inertia, as these will define the 'rotational mass' of the satellite. Since the satellite is cylindrical, we have symmetry about the $\hat{x}$ -direction. So, we will convert into polar coordinates, which gives the following relations:

$$\begin{aligned}x &= x \\y &= r \cos \theta \\z &= r \sin \theta\end{aligned}$$

We can use these relations in our moment of inertia integrals (2.11). We'll start by computing the moment in the x -direction, substituting in our polar coordinates:

$$I_{xx} = \iiint_V \rho(y^2 + z^2) dV \rightarrow I_{xx} = \iiint_V \rho[(r \cos \theta)^2 + (r \sin \theta)^2] dV$$

Next, we change the integral bounds. These integration bounds will be the same for all I_{xx} , I_{yy} , and I_{zz} since the bounded areas are identical. We use the volume of the cylinder as the integration bounds, and use the jacobian associated with polar coordinates for dV (which is $rdrd\theta dx$).

$$I_{xx} = \int_0^r \int_0^{2\pi} \int_{-\frac{h}{2}}^{\frac{h}{2}} \rho[(r \cos \theta)^2 + (r \sin \theta)^2] r dr d\theta dx$$

With the integral set up, it is simply a matter of computation to complete it. Some trig relations may be useful in completing the integral. The only other thing to note is that we can rewrite ρ as $\frac{m}{V}$, where V for the cylinder is $\pi r^2 h$.

Doing these integrals for all of the axes, we get:

$$\begin{bmatrix} \frac{1}{2}mr^2 & 0 & 0 \\ 0 & \frac{1}{12}m(3r^2 + h^2) & 0 \\ 0 & 0 & \frac{1}{12}m(3r^2 + h^2) \end{bmatrix}$$

Which is valid for all uniform density cylinders.

Plugging in the given values for Explorer 1, we have:

$$\begin{bmatrix} 0.0512 & 0 & 0 \\ 0 & 5.3590 & 0 \\ 0 & 0 & 5.3590 \end{bmatrix} \text{ kg}\cdot\text{m}^2$$

Spin Stabilization

NASA wants to spin-stabilize the satellite along the $\hat{x}$ -direction with a final rotation rate of 2π rad/s. We will assume that the impulse is provided by two motors acting as a couple. The spin motors will also separate the satellite from the launch vehicle, so they are inclined 10° from horizontal to give the satellite a small forward momentum. So, we will calculate:

- The total impulse needed to bring the satellite to a final rotation rate of 2π rad/s
- The ΔV between the launch vehicle and the satellite after the burn, assuming the total impulse requirement is met.
- The average thrust if a 0.5 s burn is desired.

The first thing to do is calculate the total impulse needed, which can be found easily. The impulse is defined as begin the same as H using the impulse-momentum equality:

$$H = I_{xx}\Delta\omega_x = M_x\Delta t$$

Since Δt is unknown, we consider the left-hand side of the equality. Taking $\Delta\omega_x = \omega_f - \omega_i = 2\pi - 0$ and I_{xx} from before, we have a total impulse requirement of 0.3217 N·m·s.

Despin

After the spin stabilization maneuver is done to inject the satellite into its final orbit, NASA wants to slow the satellite down by using a yo-yo de-spin. Suppose the satellite has two weights, each of mass 0.5 kg, that are tied to massless strings on the satellite. Assuming that these weights can be approximated as point masses, find the necessary length of the strings so that the final rotation rate of the satellite is 5 rpm.

2.7 Key Ideas

1. Newton's 2nd Law can be used to create 2nd order differential equations that describe particle motion.
2. An n^{th} order differential equation can be reduced into n first order differential equations.
3. First order differential equations can be easily numerically integrated by a computer, allowing us to get approximate solutions to a huge number of problems.

4. We can describe the properties of a bodies rotation in an analogous way to Newton's 2nd Law.
5. We can create equations for the angular acceleration and angular velocity using Euler rotation equations.
6. We can use different frames of reference to more easily express quantities. The body frame and inertial frame are the most important in our context.

2.8 Further Reading

We assume that most readers are broadly familiar with the topics discussed in AAE 203. Understanding the application of the BKE (otherwise known as derivative transport theorem) is fundamental to the understanding of dynamics. More information about basic dynamics in general can be found in [8]. For Newtonian dynamics, refer to Chapter 1 and 2. Euler rotation equations are discussed in Chapter 4. This book uses different notation (notably, $\vec{r}^{op}$ is normally denoted as $\vec{r}_{p/o}$) but can help solidify this knowledge.

2.9 Practice Problems

1. On a nice Wednesday evening during one of your strolls through campus, you are kidnapped by a business major, who will only let you free if you can fix the vibration of the squeaky door at their frat house.

You can consider the door hinge as a mass-spring system constrained to motion along the $\hat{x}$ -direction. The spring has an equilibrium length of L_0 and a spring constant of $k = 4 \text{ N/m}$ with a mass of 1 kg. At time t_0 , the length is L_0 . An instantaneous impulse of $10 \text{ N} \cdot \text{s}$ is applied to the hinge in the $-\hat{x}$ -direction by a misguided ping-pong ball hitting the door. This is shown diagrammatically in [Figure 2.12](#). Assume that the system is at rest in the equilibrium position at t_0 .

- (a) Consider the system to be frictionless. Derive the EOMs for the system in both of the component directions.

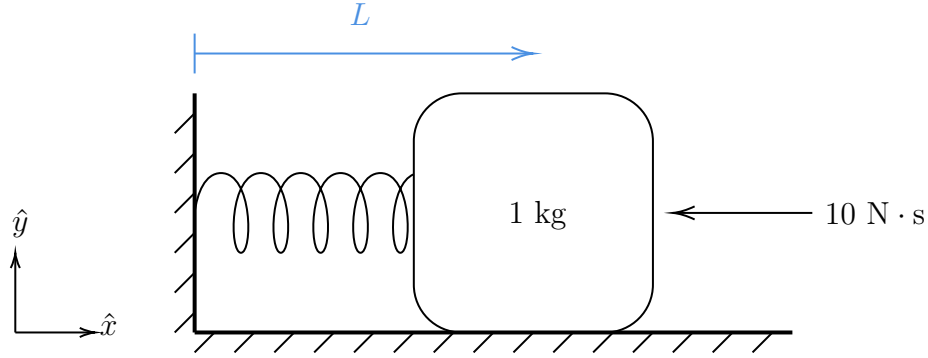


Figure 2.12: Mass-Spring System Problem Set-Up

- (b) Find the analytical solution of the system from the EOM you derived. Graph the solution.
 - (c) Numerically integrate the EOMs ascertain the position and velocity as a function of time. Graph the solution. How does it compare to the analytical solution? You may want to refer to [Chapter 6](#) for other numerical integration methods.
 - (d) Consider the system with some frictional force that is proportional to the velocity, with a proportionality constant of $c = 1 \frac{\text{kg}}{\text{s}}$. This frictional force opposes the motion of the system. Repeat step c with this new system. How does the friction change the system?
2. After your escape from the frat house, you run into your friend, Buckminster, who is a big fan of circular-shaped objects. He asks you if you can model the orbit of a satellite to see if it looks anything like his *geodesic* domes. Not wanting to let him down, you write a MATLAB script to do so. You can assume the earth has a radius $R_{\oplus} = 6380 \text{ km}$ and $\mu_{\oplus} = 3.986 \cdot 10^5 \frac{\text{km}^3}{\text{s}^2}$.

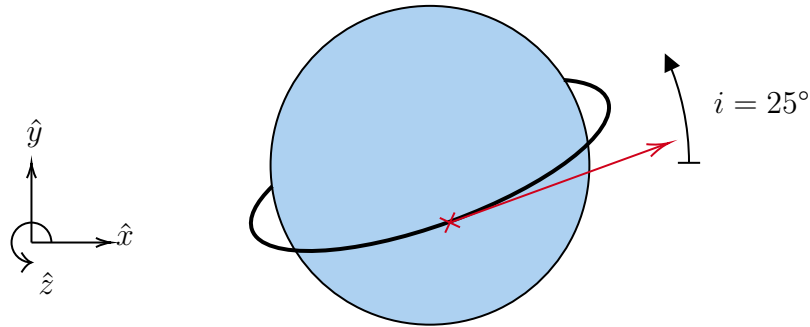


Figure 2.13: Satellite in Orbit Problem Set-Up

- (a) Assuming 3-DoF, derive the EOM for the satellite motion assuming that gravity is the only force acting on the satellite and the earth is a point mass. *Hint: You may want to use a different coordinate system.*
- (b) You find a satellite in orbit with a height of 400 km. You also measure the inclination of the orbit as it passes overhead as 25° . Assuming a circular orbit, what is the magnitude of the velocity and the component velocities in $\hat{x}$ and $\hat{y}$ as it passes overhead?

Notes

1. More formally, we define these vectors as existing in a linear space, where multiplicative and additive properties are preserved. This formal definition allows us to apply this to any arbitrarily number of dimensions. For our uses, this will usually be up to 3 dimensions.
2. Sometimes systems like this can be solved with traditional ODE methods. The most easy identifier is if the system is linear or not. However, most complex systems like our 6-DoF have no known analytical solutions. We will focus on numerical integration because it will be more useful in general.
3. Note that we are not ‘converting’ in the sense of an equality, since the equations are no longer equivalent when we convert a differential element into a discrete one. We are just making an approximation of the true solution. Luckily, numerical integration schemes can get us quite close to true solutions.
4. An inertial frame can have some constant velocity and still be inertial. Of course, the earth has some velocity around the sun and with respect to the distant stars which we can approximate as constant for our time scales and thus ignore.
5. Freestream velocity is drawn in the direction of incoming air by convention. However, for calculations, we will use the convention that $\vec{V}$ is the direction of the velocity of the vehicle (opposite sign to drawing). These direction of the vector is only dependent on the reference frame of the observer.
6. We assume here that there is no misalignment in the thrust vector and it is coincident with the vector. A more complex formula can be used when thrust misalignment is present, but this geometrical argument is not shown here.
7. The different notation of a subscript is used because this is not a standard position vector from one point to another, they rather denote the center of mass and center of pressure.
8. The order of the state vector does not particularly matter, so long as we are consistent with the location of the elements. Typically, it is common to write all vectors first and then their derivatives, but there is no hard rule. We follow this convention in our discussion.
9. Very observant readers may notice that this form of the equation is technically incorrect, since our center of rotation, O , will change throughout the flight as the center of mass changes position.

Moreover, the moments will change as fuel is drained, so another derivative should exist. Thus, we will have some small effects that are not accounted for in this equation. For our purposes here, these are ignored for simplicity of presentation. However, a more general approach that implements this may be used in the future if slosh modeling is implemented.

CHAPTER 3

Energy Methods of Mechanics

“Mathematics compares the most diverse phenomena and discovers the secret analogies that unite them.”

– Joseph Fourier

Energy methods of mechanics employ the concepts of the energy of a system to solve a problem in mechanics. This is in contrast to the Newtonian method, which considers the forces of a system. The good news is that both will result in the same answer (see proof in [Section B.2](#)). However, having both in your set of tools as an engineer can prove especially useful. To grasp energy methods, it is best to have an intuition for Newtonian dynamics first. A thorough reading of [Chapter 2](#) and other sources will help prepare the reader for this section.

There are two main types of energy mechanics, referred to as Hamiltonian and *Lagrangian* mechanics. In this chapter, we will focus on the Lagrangian version of these mechanics. Hamiltonian mechanics are arguably more fundamental, but we will leave that discussion for Volume III.

The energy methods of mechanics are more advanced than the other topics discussed here. These topics will require a bit of background in multivariate calculus and are not strictly required for understanding the rest of the 6-DoF implementation. However, we believe it is best to be exposed to these ideas early such that the reader can gain an appreciation for where more advanced mechanics (and the subsequent volumes of this text) may take them.

Although the Hamiltonian and Lagrangian are not directly used in the case of our 6-DoF, they are fundamental to other areas of mechanics and make derivations simpler in many cases. The Hamiltonian and Lagrangian descriptions of mechanics are more often taught to physics students rather than engineers, but we feel that they deserve inclusion among the topics presented here. You may find that certain EOM

derivations, such as the spinning top and problems in orbital mechanics are more easily accomplished with energy methods.

Another reason to motivate these systems is that these energy methods are rotationally invariant because energy methods can employ a generalized coordinate system. These generalized coordinates are very powerful and allow us to describe a huge variety of systems. We will see this later in some examples. A brief overview of the differences between the two schemes of EOM derivation is shown in [Figure 3.1](#).

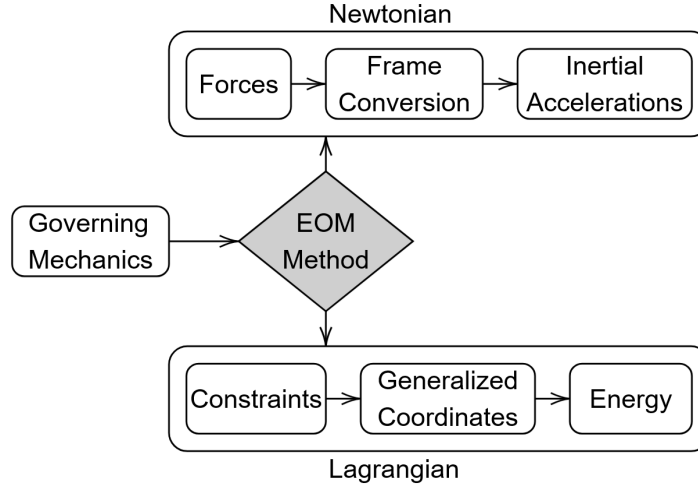


Figure 3.1: Newtonian vs. Lagrangian Mechanics Approach

The discussion of these systems in the volume is curtailed because it is not directly applicable to the work on our specific rocket 6-DoF model. However, the fundamental implications of energy methods are important, but that full discussion is saved for Volume III. A mathematical background on the calculus of variations, which provides a derivation for Lagrangian mechanics, is also presented in Volume III. We present here only the formulaic approach and its used.

3.1 Hamilton's Principle

Hamilton's Principle is the fact that underlies all of energy mechanics. To understand energy mechanics at all, we must study this idea closely. Before we can describe Hamilton's principle, we need to understand a new type of calculus, one

which is the calculus of whole functions. We call this type of calculus the ‘calculus of variations’. The calculus of variations underpins the ideas of Lagrangian and Hamiltonian mechanics, so it is important to understand the principles behind it before moving on to the actual mechanics. In analogy to how single variable calculus is fundamental to Newtonian mechanics, the calculus of variations is fundamental to energy mechanics.

The fundamental idea of the calculus of variations is simple. We want to find the maximums and minimums of some quantity in our problem. Why? In physics, we often see that quantities are minimized or maximized for some reason. For example, a soap surface will minimize its surface area, or a beam of photons will follow a geodesic of spacetime. Light will also refract through surfaces to minimize time as seen in Snell’s Law.¹⁰

Because so many physical phenomena are governed by this behavior to minimize certain quantities, Hamilton (the same mathematical hero we will see with attitude dynamics), decided to formulate a mathematical description of this.

In essence, our goal is to find a function that is the minimum of some set of conditions. These conditions also take the form of a function, so we attempt to find a function that is a minimum of another function. We call this quantity which we are minimizing a *functional*, which we define as:

$$J = \int_{x_1}^{x_2} f[y(x), y'(x); x] dx$$

The functional, J , is a quantity which maps a function onto the real number line. This means that it takes the function as an input and outputs a single real number (that is, a function of a function). We are attempting to find an input function, $y(x)$, that yields an extrema of J . Our goal is to find criteria for $y(x)$ that makes J a maximum or a minimum.

For now, we present the way to achieve this without derivation. The fundamental equation of the calculus of variations is called the *Euler’s equation*:

$$\frac{\partial f}{\partial y} - \frac{d}{dx} \frac{\partial f}{\partial y'} = 0 \quad (3.1)$$

Really, what we have done is find a condition such that the integrand is zero. We define a quantity δ , the variation, which can help us show this. We can reformulate the problem of calculus of variations to the following:

$$\delta J = \int_{x_1}^{x_2} \left(\frac{\partial f}{\partial y} - \frac{d}{dx} \frac{\partial f}{\partial y'} \right) \delta y dx$$

We see that the Euler Equation is the integrand of this equation. Since we know that this is zero, δJ must be zero! And thus, (3.1) can be written concisely as:

$$\boxed{\delta J = 0}$$

Following directly from the ideas of the calculus of variations, Hamilton (alongside many other mathematicians) wanted to apply this to physical systems. Hamilton's Principle is as follows:

Theorem 1 (Hamilton's Principle). *A system which may travel from one point to another within a specific time interval, the path that is followed is the path which minimizes the time integral of the difference between the kinetic and potential energies.*

$$\delta \int_{t_1}^{t_2} T - U dt = 0$$

The quantity $T - U$, the kinetic energy minus the potential energy, is a quantity called the Lagrangian, denoted L .

$$L \equiv T - U$$

The time integral of the Lagrangian is the *action*, denoted S . This is often why Hamilton's Principle is also called the *principle of least action*.

$$S = \int_{t_1}^{t_2} L dt$$

So, our action takes the form of the functional from the calculus of variations. From Hamilton's principle, this functional, the action, must be minimized. So, we often see the compact form of Hamilton's Principle as:

$$\delta S = 0$$

Expressed using Euler's differential equation, (3.1), we get the *Euler-Lagrange Equations*:

$$\boxed{\frac{\partial L}{\partial x_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{x}_i} = 0} \quad (3.2)$$

Euler – Lagrange Equations

Before we look into some examples of how to utilize these equations in practice, we should consider in what situations these will be better than the Newtonian methods.

We note that in this chapter, we will only consider potential functions which are *conservative*, or path independent. This does mean that frictional forces and aerodynamics are not modeled with the Lagrangian we describe here (although they can be with the utilization of Rayleigh's dissipation function and other techniques, such as those discussed in [15]). This is the reason we do not directly use these equations in the 6-DoF.¹¹

So, for systems that are non-conservative, it is best to stick with the Newtonian method of mechanics, as we do for our 6-DoF. However, for those that are frictionless, the methods described here will work well. While it may seem that frictionless systems are very idealistic, we do see systems that can be approximated as frictionless quite often! Orbits, of course, are one case where this is especially true. For many other systems, such as a spinning top or other rotating bodies, we can often ignore the small amount of friction and arrive at analytical solutions without much hassle using the Lagrangian. These methods also become extraordinarily important for our derivations of rotational equations.

With that noted, let us use the Lagrangian in a few examples to see how it can turn some complex Newtonian systems into fairly trivial problems.

3.1.1 Mass-Spring System

As a very basic first system, we will consider a mass spring system as shown in [Figure 3.2](#). We consider the length x to be referenced from the rest length of the spring.

We know the analytical form of the EOM from Newtonian Dynamics:

$$m\ddot{x} + kx = 0$$

So, in essence, we are using the Lagrangian *a posteriori* to match this solution.

First, we will find the Lagrangian of the system. The kinetic energy of the particle is given by the linear translation only (there is no rotational motion for a point particle), so we have $T = \frac{1}{2}m\dot{x}^2$. The potential energy is the energy of the spring,

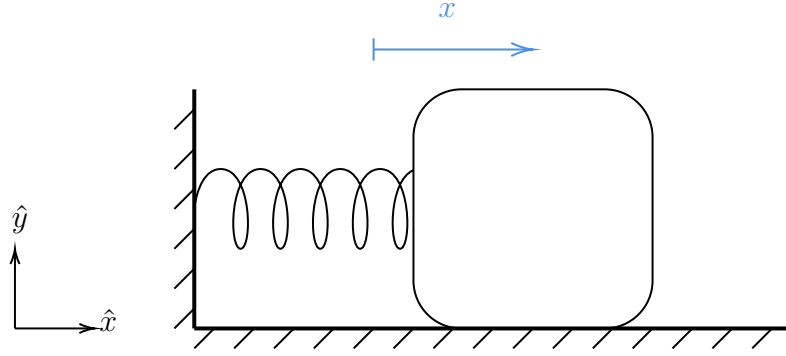


Figure 3.2: Mass-Spring System for Lagrangian

given as $U = \frac{1}{2}kx^2$. So, the Lagrangian is:

$$L = \frac{1}{2}m\dot{x}^2 - \frac{1}{2}kx^2$$

Now, applying (3.2) in the $\hat{x}$ -direction, as this is the only degree of freedom:

$$\frac{\partial L}{\partial x} - \frac{d}{dt} \frac{\partial L}{\partial \dot{x}} = 0$$

Taking each term individually,

$$\frac{\partial L}{\partial x} = -kx$$

$$\frac{\partial L}{\partial \dot{x}} = m\dot{x}$$

$$\frac{d}{dt}(m\dot{x}) = m\ddot{x}$$

So, the (3.2) equation gives us:

$$-kx - m\ddot{x} = 0$$

A simple change of signs gives the exact same result as the Newtonian approach:

$$m\ddot{x} + kx = 0$$

In this problem, we used a simple case where the dynamics were constrained along a single axis of motion. Before we move to a more complex problem, we will introduce the method by which Lagrangian mechanics deals with these reference frames.

3.2 Generalized Coordinates

We recall that in Newtonian mechanics, we describe reference frames for an object. We represent the position, velocity, and acceleration of a particle in terms of these coordinate systems as a tridimensional vector. Generally, we may write the $\vec{r}^{op}$ vector in a frame a as:

$$x\hat{a}_1 + y\hat{a}_2 + z\hat{a}_3$$

Because any tridimensional set of coordinates that we choose that are linearly independent of each other will span $\mathbb{R}^3$, we can generalize this to any reference frame, q , where the terms of a can be described by the terms of q :

$$x(q_1, q_2, q_3) + y(q_1, q_2, q_3) + z(q_1, q_2, q_3)$$

As a shorthand, we will write this full set of q , $(q_1, q_2, \dots, q_n)$, as q_j .

We can also generalize this to the velocity and the acceleration. The components of velocity can be written in terms of $q_j, \dot{q}_j$ and those for acceleration in terms of $q_j, \dot{q}_j, \ddot{q}_j$. When we do Newtonian dynamics, we need to use the BKE to transform these quantities if they are not in the inertial frame.

Note that this is what this looks like for a single particle. When multiple particles are involved, we may need to increase this number of q coordinates. As a very general statement, we can say for N particles, we will need n generalized coordinates in q . Expressed vectorially, given a particle of the i^{th} index:

$$\vec{r}^{oi}(q_j)$$

In other words, the position of an arbitrary particle is given in terms of n generalized coordinates.

The acceleration of the particles, when Newton's Laws are applied, are a function of the q_j as well as their derivatives, $\dot{q}_j$. So, the evolution of the system exists in a $2n$ -dimensional state space. However, we can use our previously mentioned Lagrangian to circumvent the need for Newton's Law and the BKE in our derivation of the EOM.

3.2.1 Constraints

Constraint equations are fundamental to Lagrangian mechanics. In the previous section, we very vaguely stated the need for n generalized coordinates. To find a precise number, we turn our attention to constraints.

For each particle, we need 3 values to define the position, so we need $3N$ generalized coordinates.

However, we often have constraints that reduce this number. Often, we see a problem that is planar or where the particles are connected in a rigid body. These constraints will reduce the number of generalized coordinates needed. Here, we are only considering constraints on the positions of particles. Constraints on the positions of particles are known as *holonomic constraints*.

A holonomic constraint relates generalized coordinates to each other. In the commonly used notation, the holonomic constraint is expressed in terms of the q_j 's as:

$$\Phi_k(q_j, t) = 0, \quad k = 1 \cdots K$$

or in terms of the position vectors:

$$f_k(\vec{r}^{op1}, \vec{r}^{op2} \cdots \vec{r}^{opN}, t) = 0$$

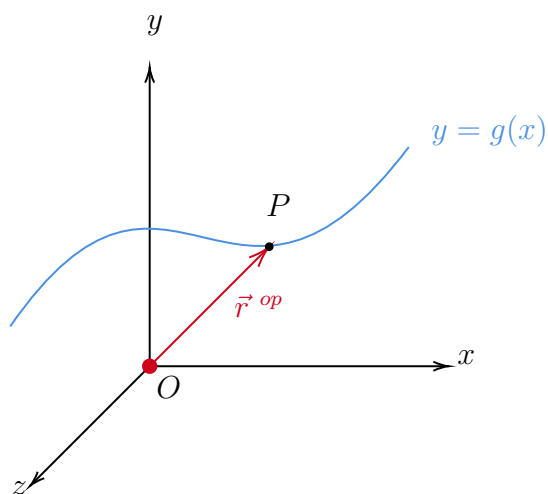
The number of holonomic constraints, K , will change the number of generalized coordinates as:

$$\boxed{\mathcal{M} = 3N - K} \tag{3.3}$$

The quantity $\mathcal{M}$ is the number of DoF. However, this is the number of DoF for the whole system, which is why it can exceed 6 in some cases!

Example 3.1: Particle on a path

Suppose we have a single particle that is constrained to motion along a path in the xy -plane. This path is given by $y = g(x)$.

**Figure 3.3:** Particle Moving on Constrained Path

We can select our generalized coordinates to be the same as the cartesian coordinates, so that:

$$q_1 = x$$

$$q_2 = y$$

$$q_3 = z$$

So, to write these as holonomic constraints, we can write:

$$q_3 = 0$$

$$q_2 - g(x) = 0$$

With these two constraints, we have reduced the number of degrees of freedom, $\mathcal{M}$, to 1.

This makes sense intuitively, as the particle can only move along the path specified by the equation $g(x)$, therefore only having one degree of freedom.

The power of using generalized coordinates is that we could define a coordinate that more easily defines this system. For example, our one coordinate could be the arc length along the path, s defined from some starting point.

In general, these generalized coordinates do not even need to have units of length!

We can select any parameters that are most convenient for defining the state, even those that are energies, so long as we can define a continuous derivative of that state and use the Euler-Lagrange Equation.

Rigid Body Constraints

For most of the systems that we will deal with, including our rocket, we make the assumption that particles are rigidly attached to each other, meaning that there is no change in distance between particles. Using the notation of constraints described above, we will show how this is done for a two particle system, which can then naturally be extended with more particles.

Given two particles that are connected together by a massless rigid rod of length L , we have a constraint on the particles distance:

$$\|\vec{r}^{op1} - \vec{r}^{op2}\| = L$$

Written in the more standard notation:

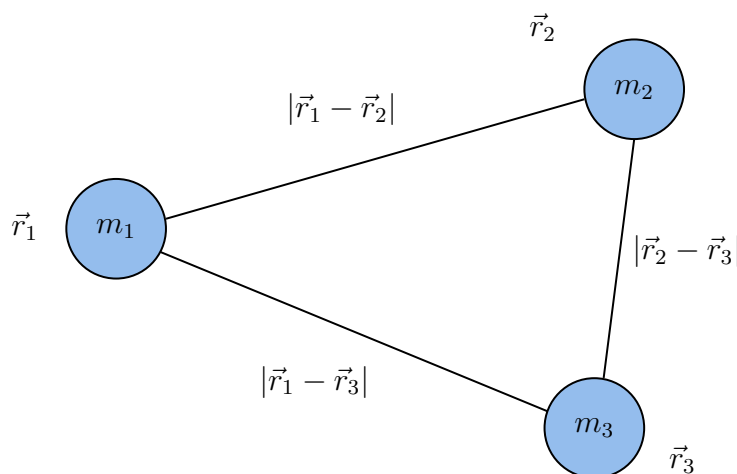
$$f(\vec{r}^{op1}, \vec{r}^{op2}) = \|\vec{r}^{op1} - \vec{r}^{op2}\| - L = 0$$

So this constraint takes the 6-DoF for the two particles and reduces it to 5-DoF.

An example of these constraints would be the position of the first particle in Cartesian coordinates (giving 3 of the generalized coordinates, x, y, z , and the azimuthal and elevation angle, ϕ and θ). The roll angle does not change the system in any measurable way, hence we only need 5 parameters to define a two particle system.

In the general case of N particles composing a rigid body, we have $3N$ -DoF in a completely unconstrained system. However, we must constrain each pair of particles. An example of this for the 3 particle system is shown in [Figure 3.4](#).

So, we have 3 constraint equations in a system with 9 particles. This gives a total of $3N - 3 = 6$ -DoF. When any additional particles are added, they must be fully constrained in all 3 translational degrees of freedom by constraints between r_1, r_2 , and r_3 . So, we always have constraints of the form $3N - 3(N - 2)$ for a rigid body. This is why we always see a rigid body displaying 6-DoF. We can then, of course, parametrize this in general coordinates with the cartesian coordinates and 3 angles.

**Figure 3.4:** 3 Particle Rigid Body

Minimal Set of Constraints

So far, we have only discussed holonomic constraints, those that are related to position or on the generalized parameters, and possibly have some relation in time. When a system is a *holonomic system*, that is, purely defined by holonomic constraints, it is always possible to find a set of $\mathcal{M}$ generalized parameters to describe the particle motion. In other words, the minimum number of parameters is equivalent to the number of degrees of freedom in the system.

That said, we often choose to use more parameters, particularly in the case of rotations. As we will see in Volume II, we often find it more useful to use a non-minimal set of constraints to encode rotations for reasons to avoid singularities in the solution.

It is also common to see a non-minimal set of constraints used when the minimal set is cumbersome and does not give an intuitive feel to the problem.

Non-Holonomic Constraints

The other class of constraints are those constraints that are not merely a function of the generalized coordinates. These constraints, called *non-holonomic constraints*, define the set of all constraints that depend on more than just the position of particles. This is a huge class of constraints so we will not cover every type. These are also less

important to our derivations so we will only briefly discuss them.

The most common among these, and where we will focus our attention here, is constraints on the time derivative of the generalized coordinates. These are called *generalized velocities*, denoted $\dot{q}_j$. Such an example of a constraint on generalized velocities is a roller skate. A roller skate only allows a velocity in the direction of the skate and restricts velocity along the direction normal to the roller skate.

One important thing about non-holonomic constraints is that they do not reduce the number of parameters needed to define the system.

3.2.2 Lagrange's Equation in Generalized Coordinates

The good thing about the Lagrangian method is that it is invariant to changes in our coordinate system. Because the Lagrangian is an equation of energy, and energy is independent of coordinate transformations, we can easily express the Euler-Lagrange Equation in generalized coordinates with little hassle.

Using the generalized coordinates, the kinetic energy is a function of the generalized positions, velocities, and time. The potential is a function of the generalized position and time, so the Lagrangian becomes:

$$L \equiv T - U = T(q_j, \dot{q}_j, t) - U(q_j, t)$$

Stated another way, the Lagrangian is a function of the generalized positions, velocities, and time, as $L = L(q_j, \dot{q}_j, t)$. The Euler-Lagrange Equation then becomes:

$$\boxed{\frac{\partial L}{\partial q_j} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}_j} = 0, \quad j = 1, 2, \dots, M} \quad (3.4)$$

Generalized Euler – Lagrange Equations

Next we will show a problem where the rotational invariance and generalized coordinates proves especially useful in simplifying our derivation:

3.2.3 Pendulum EOM Example

Problem Set-Up

Suppose we have a pendulum of length L . The mass is rigidly attached to the pendulum. A reference frame a that moves with the pendulum is shown in [Figure 3.5](#). The inertial frame is also shown. The pendulum moves in only the xy plane.

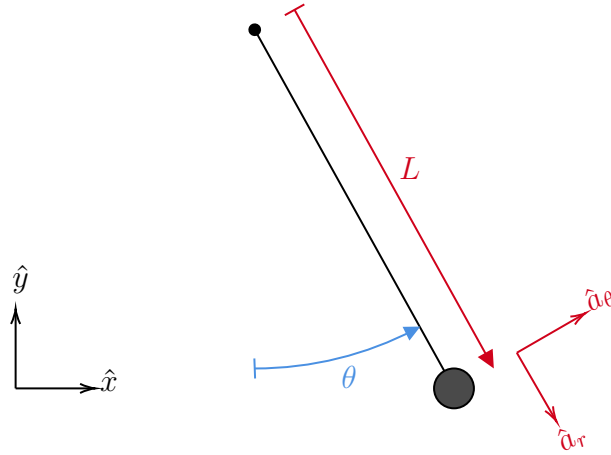


Figure 3.5: Pendulum Set-Up

Newtonian Approach

You have likely seen the Newtonian approach many times, but we show it here for completeness and to contrast the approach with the Lagrangian more directly.

In the Newtonian Approach to deriving EOM's, we always start with the forces governing the system. In this case, we have two forces. The gravitational force, mg , acts in the $-\hat{y}$ -direction and the tension force acts in the $-\hat{a}_r$ -direction. From the constraint on length, the acceleration in the $\hat{a}_r$ must be zero. Thus, the component of gravity in the $\hat{a}_r$ direction must equal the tension.

We express the gravity in the a -frame to find this:

$$\hat{y} = -\cos\theta\hat{a}_r + \sin\theta\hat{a}_\theta$$

So, by balancing the forces in the $\hat{a}_r$ -direction:

$$m\ddot{a}_r = mg \cos \theta \hat{a}_r - \vec{F}_T$$

In the $\hat{a}_\theta$ -direction, we have unconstrained motion, so $\sum \vec{F} = m\vec{a}$. Our force equation is thus:

$$F = -mg \sin \theta \hat{a}_\theta$$

Next, we need to find our acceleration in the inertial frame using the BKE to solve the RHS of the force balance. Starting with the position vector:

$$\vec{r}^{op} = L\hat{a}_r$$

We can apply the BKE to find our inertial velocity:

$${}^i\vec{v}^p = {}^a\frac{d}{dt}\vec{r}^{op} + {}^i\omega^a \times \vec{r}^{op}$$

Plugging in appropriate quantities:

$$\begin{aligned} {}^i\vec{v}^p &= {}^a\frac{d}{dt}(L\hat{a}_r) + \dot{\theta}\hat{a}_3 \times (L\hat{a}_r) \\ {}^i\vec{v}^p &= \dot{\theta}L\hat{a}_\theta \end{aligned}$$

We do this again to find the inertial acceleration:

$$\begin{aligned} {}^i\vec{a}^p &= {}^a\frac{d}{dt}{}^i\vec{v}^p + {}^i\omega^a \times {}^i\vec{v}^p \\ {}^i\vec{a}^p &= {}^a\frac{d}{dt}\dot{\theta}L\hat{a}_\theta + \dot{\theta}\hat{a}_3 \times \dot{\theta}L\hat{a}_\theta \\ {}^i\vec{a}^p &= \ddot{\theta}L\hat{a}_\theta - \dot{\theta}^2L\hat{a}_r \end{aligned}$$

Now, we equate the forces in each direction:

$$-mg \sin \theta \hat{a}_\theta = \ddot{\theta}L\hat{a}_\theta$$

Rearranging, we arrive at our EOM:

$$\ddot{\theta} + \frac{g}{L} \sin \theta = 0$$

Lagrangian Approach

In the Lagrangian approach, we will start by defining our constraints and coordinates. Some preliminary analysis shows that using polar coordinates for this problem will be useful. If we do not recognize this, we can always convert later (a distinct advantage of the Lagrangian approach, since it is coordinate frame invariant). Here we will choose a cylindrical coordinate system:

$$\begin{aligned}q_1 &= r \\q_2 &= \theta \\q_3 &= z\end{aligned}$$

Next, we define constraints. Since there is no motion in the z -axis:

$$z = 0$$

And the constraint on the length gives us:

$$q_1 - L = 0$$

So, we have 1 DoF from (3.3). Since both z and r are constrained, we can express our EOM in terms of one generalized coordinate, θ .

Next, we define the Lagrangian. Knowing that the linear velocity of the particle is $\dot{\theta}L$, we get:

$$T = \frac{1}{2}m\dot{\theta}^2L^2$$

For the potential, we choose the origin as our reference, giving us:

$$U = -mgy$$

And a coordinate transformation into polar using $y = r \cos \theta$ yields:

$$U = -mgL \cos \theta$$

This gives the Lagrangian:

$$L = \frac{1}{2}m\dot{\theta}^2L^2 + mgL \cos \theta$$

Applying the Euler-Lagrange formula for generalized coordinates, (3.4) on the single generalized coordinate:

$$\begin{aligned}\frac{\partial L}{\partial \theta} &= -mgL \sin \theta \\ \frac{\partial L}{\partial \dot{\theta}} &= mL^2 \dot{\theta} \\ \frac{d}{dt} (mL^2 \dot{\theta}) &= mL^2 \ddot{\theta}\end{aligned}$$

So, the final system is:

$$-mgL \cos \theta - mL^2 \ddot{\theta} = 0$$

With some rearrangement:

$$\ddot{\theta} + \frac{g}{L} \sin \theta = 0$$

We notice that with the Lagrangian approach, we did not need to have an understanding of the vector directions, frame translations, or BKE to find our EOM. This, in addition to the flexibility of the generalized coordinates, is the real power of this approach.

3.3 Hamiltonian Mechanics

After the formulation of Lagrangian Mechanics, another formulation was created by Hamilton later on. The Hamiltonian formulation, like the Lagrangian, does not introduce any new concepts, because it is simply an alternate approach to dynamics. However, like the Lagrangian, it makes some types of problems more simple. We present the Hamiltonian here briefly for its use case in *symplectic integrators*.

The exact derivation of the Hamiltonian using Legendre transforms is beyond the scope of our present discussion, so we will mostly discuss the consequences of this derivation.

The Hamiltonian method replaces the generalized velocities with *generalized momenta*. The generalized momenta are a function of $q, \dot{q}$, and t . We express the generalized momenta as p . p is defined to be:

$$p(q, \dot{q}, t) \equiv \frac{\partial L}{\partial \dot{q}_i}$$

Thus, we express our Hamiltonian, $\mathcal{H}$, in terms of the generalized positions, generalized momenta, and time. This looks like:

$$\mathcal{H} = \mathcal{H}(q_j, p_j, t)$$

In a conservative system, we can also write:

$$\mathcal{H} = T + U$$

Using relations with the Lagrangian, we can write two first order differential equations for the EOM with the Hamiltonian.

$$\begin{aligned} \dot{q}_k &= \frac{\partial \mathcal{H}}{\partial p_k} \\ -\dot{p}_k &= \frac{\partial \mathcal{H}}{\partial q_k} \end{aligned}$$

Hamiltonian EOM's

This is all we will note about Hamiltonian Mechanics for now. We will note here that these are more useful in the case of numerical integration because these are first order differential equations, as opposed to the second order differential equations of Lagrangian Mechanics.

3.4 Lagrangian Derivation of Rigid Body Dynamics

In [Section 2.5](#), we explored the Newtonian method of deriving the Euler rotation equations using the BKE. Here, we present an alternative derivation that utilizes the Lagrangian approach and is ultimately a more fundamental derivation.

We showed earlier in [Figure 3.2.1](#) that we can describe a rigid body system with 6 generalized parameters, three of which are angles.

3.4.1 Torque Free Equations

In the case of no applied torque, the derivations of Euler's equations become surprisingly simple. We can assume that the change in height of the rigid body is irrelevant for pure rotation, so the Lagrangian is simply the kinetic energy of the object. We can suppose that we are also in a coordinate system where the translational kinetic energy is 0. This gives:

$$L = \frac{1}{2}I\omega^2 \quad (3.5)$$

We can select our generalized coordinates as the three Euler angles, ψ , θ , and ϕ (see [Section 4.1](#)).

Selecting the generalized coordinate, ψ , as an example, the Euler-Lagrange equation gives:

$$\frac{\partial L}{\partial \psi} - \frac{d}{dt} \frac{\partial L}{\partial \dot{\psi}} = 0$$

Using the derivation of the components of angular velocity using Euler Angles, we have:

$$\begin{aligned} \omega_1 &= \dot{\phi} \sin \theta \sin \psi + \dot{\theta} \cos \psi \\ \omega_2 &= \dot{\phi} \sin \theta \cos \psi - \dot{\theta} \sin \psi \\ \omega_3 &= \dot{\phi} \cos \theta + \dot{\psi} \end{aligned}$$

So, we can also express our Lagrangian in terms of the angular acceleration:

$$\frac{\partial L}{\partial \omega_i} \frac{\partial \omega_i}{\partial \psi} - \frac{d}{dt} \frac{\partial L}{\partial \omega_i} \frac{\partial \omega_i}{\partial \dot{\psi}} = 0$$

At this point, the derivation is simply a matter of 'plug and chug'. Calculating the derivatives with respect to ψ :

$$\begin{aligned} \frac{\partial \omega_1}{\partial \psi} &= -\dot{\phi} \sin \theta \cos \psi - \dot{\theta} \sin \psi \\ \frac{\partial \omega_2}{\partial \psi} &= -\dot{\phi} \sin \theta \sin \psi - \dot{\theta} \cos \psi \\ \frac{\partial \omega_3}{\partial \psi} &= 0 \end{aligned}$$

We notice here that $\frac{\partial \omega_1}{\partial \psi} = \omega_2$ and $\frac{\partial \omega_2}{\partial \psi} = -\omega_1$. Next, taking our derivatives with

respect to $\dot{\psi}$:

$$\begin{aligned}\frac{\partial \omega_1}{\partial \dot{\psi}} &= 0 \\ \frac{\partial \omega_2}{\partial \dot{\psi}} &= 0 \\ \frac{\partial \omega_3}{\partial \dot{\psi}} &= 1\end{aligned}$$

Last, we can make a substitution for the $\frac{\partial L}{\partial \omega_i}$ term by taking a derivative of the kinetic energy(3.5):

$$\frac{\partial L}{\partial \omega_i} = I_i \omega_i$$

Plugging everything into the Euler-Lagrange formula, we have:

$$I_1 \omega_1 \omega_2 - I_2 \omega_1 \omega_2 - \frac{d}{dt} I_3 \omega_3 = 0$$

Some simplification yields:

$$(I_1 - I_2) \omega_1 \omega_2 = I_3 \dot{\omega}_3$$

We can perform the analogous analysis for the other axes, yielding the full set of torque free rigid body equations:

$$\begin{aligned}(I_y - I_z) \omega_y \omega_z &= I_x \dot{\omega}_x \\ (I_z - I_x) \omega_x \omega_z &= I_y \dot{\omega}_y \\ (I_x - I_y) \omega_x \omega_y &= I_z \dot{\omega}_z\end{aligned}$$

Torque – Free Rotations

3.5 Key Ideas

1. We can find the EOM of a system using either the Newtonian approach or the Lagrangian approach, and it is often easier to find an EOM using the Lagrangian for a conservative system.
2. The Lagrangian is scalar, so it is invariant to coordinate transforms. This means we can choose any variables we want for our generalized coordinates when performing derivations.

3. The methodology of Newtonian mechanics and Lagrangian mechanics are fundamentally different. Newtonian mechanics deals with the influence of external actions (forces), while Lagrangian Mechanics emphasizes the internal state of the system.

3.6 Further Reading

The following section borrows heavily from the derivations of Chapter 6 and 7 of [22] and Chapter 5 of [24]. A good video overview of this topic can be found at [25].

There are an abundance of sources for classical mechanics. We can personally recommend the video series at [18], which covers this material in a manner more suitable to engineering contexts.

3.7 Practice Problems

1. Arthur keeps finding bugs on the apples that he is feeding his horse. Arthur heard that you have been learning about generalized coordinates. He wants a way to describe the position of the bugs on his apples using a minimal set of coordinates.

Find a suitable set of generalized coordinates to describe this system. Write out any equations of constraints of the system. *Hint: A apple is topologically similar to a sphere!*

2. You are a prominent mathematician in the year 1696. Bernoulli proposes a problem to you to find a curve from a point (x_0, y_0) to another point (x_f, y_f) . When a particle is placed on this path, it should be a faster path than any other path the particle could take down the slope under the influence of only gravity.
 - (a) Find the velocity as a function of the distance from the initial height, h , for the particle using energy conservation.
 - (b) Write a general expression for the amount of time it takes a particle to descent a path using an integral. *Hint: you can write $t = \frac{ds}{v}$.*
 - (c) Use the Euler-Lagrange equation to solve for the form of this curve (You may find the integral form useful!). Plot the result.

3. Jonah wakes up one day and he is running late to his dynamics final! In a rush, he hops on his Onewheel and starts riding to class. You can model Jonah as a mass on the end of a massless rod and his Onewheel as a cart constrained to motion along the $\hat{x}$ -axis. The angle of Jonah from the vertical is denoted as θ . Derive the EOM of the system using energy methods. The set-up is shown in Figure 3.6.

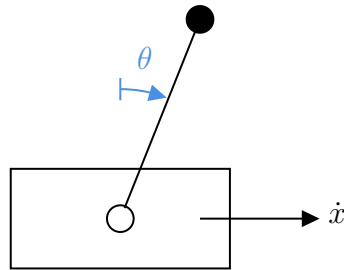


Figure 3.6: Inverted Pendulum Set-Up

4. Luckily, Jonah was an avid reader of the Flight Dynamics Bible and aced his dynamics exam! On his way back home, he needs to grab a small model rocket for some aerodynamics testing. On the way, the parachute pops out of the nose and Jonah grabs it right before it hits the ground. Jonah is holding the rocket by the parachute and the rocket and parachute are swinging around.

The dynamics of the system can be modeled as a double pendulum with two rigid arms, both of length L , which are free to rotate in the xy -plane. Find the EOM for the system as shown in Figure 3.7.

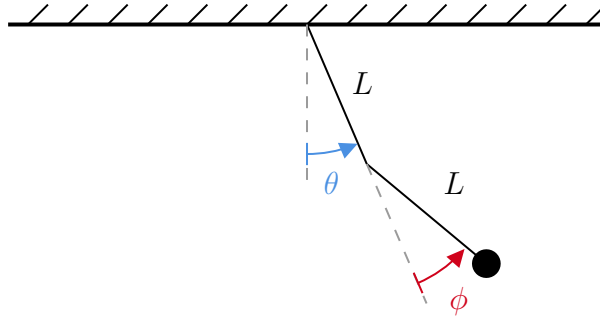


Figure 3.7: Double Pendulum Set-Up

Notes

10. More information on all of these topics is found in the introductory part of Volume III.
11. Much of the rotational dynamics we describe can be represented equivalently using the Lagrangian approach, so we do use these equations in a roundabout manner. We will explore this more extensively in Volume III

CHAPTER 4

Computational Attitude Dynamics

“Quaternions came from Hamilton after his really good work had been done; and, though beautifully ingenious, have been an unmixed evil to those who have touched them in any way, including Clerk Maxwell.”

– Lord Kelvin, 1892

Attitude Dynamics is the math describing the orientation of a vehicle in space as the result of applied moments. Relying heavily on advanced mathematics, this concept is naturally more difficult to grasp. In fact, we have dedicated the entirety of Volume II to the theory behind attitude dynamics, determination, and control. Derivations for everything from simple, two-dimensional problems and single-axis rotations to *quaternions* are included in that book, as well as the more foundational principles derived using topics from abstract algebra. Following the core purpose of this volume, we will only be describing implementation of attitude dynamics using MATLAB functions. However, the entirety of Volume II contains the theory necessary to gain a more definitive understanding of what, and more importantly why, everything is happening with the code in this section.¹²

4.1 Euler Angles

The first method to express the orientation of a vehicle in space is the use of *Euler angles*. These form the basis for our understanding of attitude dynamics, being the more intuitive method. To keep this section as independent as possible, basic definitions and derivations will be included here to give you the bare bones of what is needed in the function calls to help you and your code be successful.

First and foremost, we use the intrinsic (body-fixed) 3-2-1 rotation sequence to

determine Euler angles. This means we have an angle ϕ to describe the rotation about the x -axis, an angle θ to describe the rotation about the y -axis, and an angle ψ to describe the rotation about the z -axis. The order in which we complete these rotations is Z-Y-X, or ψ - θ - ϕ .¹³ Remembering this rotation sequence is very important, as every function call within your MATLAB script will depend in part on this!

4.1.1 The Direction Cosine Matrix

To start, we want a way to easily convert any arbitrary vector from being described using one reference frame to another. We do this through the use of the Direction Cosine Matrix, or DCM. This allows us to take this matrix and multiply it with our vector in the inertial frame and obtain an identical vector expressed using the body frame as shown:

$${}^b\vec{r} = A_{3,2,1} {}^i\vec{r}$$

In MATLAB, there exists a function - `angle2dcm` - that creates this for us!¹⁴ By inputting the scalar values of our Euler angles (in radians!) as well as the chosen rotation sequence, MATLAB creates the DCM as an output, allowing us to pre-multiply this matrix with our inertial vector we want to express in the body frame. Note the function allows an input of any number of rotations to find DCMs for. This is a fun little aspect of the function; however, it is irrelevant for our use as we input our new Euler angles every time-step during the `ode45` integration scheme. One additional thing to consider: the `angle2dcm` function returns the DCM as a transform from the inertial frame to the body frame. If we were to transform a vector from the body to inertial frame, we would need to take the inverse - in this case transpose - of the given matrix.¹⁵ Further reading on this function can be seen in Mathwork's own documentation at Source [11].

4.1.2 Relating Euler Rates with Angular Velocity

As with our translational values, we want to take the integral of our rate of change of these angular parameters in order to find our new, updated Euler angles. While it might be easy to assume the values we would be integrating are the angular velocities, doing so DOES NOT return the Euler angles. Giving a quick reason as to why: angular velocity is expressed exclusively with respect to the body frame, while our Euler angles are defined about their corresponding intermediate frame in the

rotation sequence. Meaning this job for integration is left to *Euler rates*, something else we need to include in the code. There exists what we affectionately call the “B” matrix that allows us to use Euler angles to express a relation between Euler rates and angular velocities. Unfortunately, there doesn’t exist a MATLAB function that can calculate the “B” matrix, leaving us to our own devices to construct one from scratch. The heavy lifting for this is completed in Volume II, meaning we can copy the matrix derived there straight into our MATLAB script. That matrix and equation is as follows, for quick reference:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = B \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

Euler Rates

$$B = \begin{bmatrix} 1 & \tan(\theta)\sin(\phi) & \tan(\theta)\cos(\phi) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \end{bmatrix}$$

The “B” Matrix for 3 – 2 – 1 Rotations

It is important to note, that the “B” matrix will be different for every rotation sequence. So, the matrix given above is only valid for our specific chosen sequence!

4.1.3 Euler Angle MATLAB Function Example

Putting it all together, we are going to write a simple code using Euler angles to practice using MATLAB’s built in functions. Say we have a very important vector we want to express in the body frame and another important vector we want to express in the inertial frame: gravity and thrust respectively. If our rocket weighs 100 Newtons and outputs a thrust of 1000 Newtons, we know this appears in vector notation as: $-100\hat{x}$ and $-1000\hat{X}$ respectively. Consider a moment in time, when - using the 3-2-1 rotation sequence - our vehicle is at a roll angle of 2.71 radians and a yaw and pitch angles of 0.2 radians, where we want to convert our gravity vector from the inertial frame to the body frame. In addition, with knowledge that the angular velocity at that instant is $[2.5 \ 0.1 \ 0.1]^T$, we can also find the Euler rates of the system. The following code listing quickly runs through the process, with sufficient commentary within the script explaining every action taken.

Code Listing 4.1: Euler Angle Dynamics Example

```

% PSP FLIGHT DYNAMICS:
%
% Title: GravityToBodyEuler
% Author: Preston Wright
% Last Modified: 8/30/25
%
% Description: Example converting a gravity force from the
%             inertial to body frame and thrust force from body to
%             inertial frame using Euler angles

% Start with your initializations. Every angle should be
% in radians, and our forces are in Newtons
phi = 2.71; % x-axis angle [rad]
theta = 0.2; % y-axis angle [rad]
psi = 0.2; % z-axis angle [rad]
gravityInertial = [-100;0;0]; % inertially expressed
% gravity force [N]
thrustBody = [-1000;0;0]; % body expressed thrust
% force [N]
omega = [2.5;0.1;0.1]; % angular velocity [rad/s]
bMatrix = [1, tan(theta)*sin(phi), tan(theta)*cos(phi);
...
0, cos(phi), -sin(phi); ...
0, sin(phi)/cos(theta), cos(phi)/cos(theta)];

% Calculate the DCM. Remember we're using a 3-2-1 rotation
% sequence for this problem, so input angles in this
% order!
DCM = angle2dcm(psi,theta,phi,"ZYX");

% Matrix multiply the DCM with the inertial gravity vector
% to find the body frame representation of the gravity
% vector. Multiply the thrust vector by the transpose (
% inverse) of the DCM to obtain the thrust vector in the
% inertial frame
gravityBody = DCM*gravityInertial;
thrustInertial = (DCM')*thrustBody;

```

```
% We now multiply our b matrix by the angular velocity
    vector to get the Euler rates
eulerRate = bMatrix*omega;
```

Our results come out to be the following, with the gravity force expressed in the body frame and thrust expressed in the inertial frame as:

$${}^b\vec{F}_g = \begin{bmatrix} -96.0530 \\ -26.1902 \\ 9.3748 \end{bmatrix}; \quad {}^i\vec{F}_T = \begin{bmatrix} -960.5305 \\ -194.7092 \\ 198.6693 \end{bmatrix}$$

At this point, we would use our numerical integration scheme to approximate the next timestep's Euler angles using the Euler rates we just found. We won't do that here, but you can see a similar implementation with quaternions in the 6DoF itself or the Dzhanibekov example!

4.2 Quaternions

To match industry/academia standards and improve the robustness of our simulation, our 6DoF model uses *quaternions*. The use of normalized hyper-complex numbers allows us to avoid losing a degree of freedom at a specific orientation of our rocket (called gimbal lock), which is what makes the simulation more robust. Just as with Euler angles, MATLAB has an extensive suite of functions that makes the implementation of quaternions just as easy.

4.2.1 Quaternion Rotation Matrix

We are going to combine a few initial steps of the code surrounding quaternion based attitude dynamics. To start, we want to convert from Euler angles to quaternions, as Euler angles are more intuitive to define and initialize in the simulation. MATLAB's function `eul2quat` does just that. By inputting the Euler angles in the appropriate sequence, along with the chosen rotation sequence (make sure to match the Euler angles with the order), the function returns the corresponding quaternion. Mathworks documentation for this function can be found at Source [11].

Having initialized our quaternion, we can now use another MATLAB function to construct our DCM. The function `quat2dcm` takes the quaternion as an input, and returns the DCM at that instant.¹ Incredibly simple and incredibly useful; however, we must once again consider that this DCM converts only from the inertial frame to the body frame. Meaning just as we had done with Euler angles, it is necessary to take the transpose (inverse) of the matrix if we want to transform from the body frame to the inertial frame. The Mathworks documentation for the `quat2dcm` function can be found at Source [12].

4.2.2 Quaternion Rates

Just as we had done for Euler angles with Euler rates, we want to find the quaternion rates that allows us to integrate and obtain a new quaternion. Also just as with Euler angles, there is no MATLAB function allowing us to relate quaternion rates with angular velocity. But once again, just as with Euler angles, we can derive the necessary method to do so! The outcome of this derivation is shown in (4.1). So plugging it into our script is an easy fix to make this operation possible. The calculation is included here for quick reference:

$$\dot{\mathbf{q}} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \vec{q} \quad (4.1)$$

Quaternion Rates

4.2.3 Quaternion MATLAB Function Example

Let's take the exact same example as before. We have the force of gravity equal to $-100\hat{x}$ and the thrust force equal to $-1000\hat{X}$. We have a roll angle of 2.71 radians, yaw and pitch angles of 0.2 radians, and we have chosen a 3-2-1 rotation sequence. We finally have an instantaneous angular velocity of $[2.5 \ 0.1 \ 0.1]^T$. Wanting to

¹This step begins the divergence from the true method of quaternion encoded rotations, as we choose to use elements of the quaternion to build the elements of the DCM for clarity's sake. True quaternion encoded rotations are discussed in-depth in Volume II.

obtain the gravity vector expressed in the body frame instead of the inertial frame, and vice versa for the thrust vector, this time we're going to use quaternions to do so. In addition, we're now going to find the quaternion rate at the given moment. The listing below does this, and is sufficiently explained in the comments between operations to hopefully give you as the reader a sufficient understanding of what is happening. Note: you should see numerous similarities, with some sections of the code identical to the previous.

Code Listing 4.2: Quaternion Dynamics Example

```
% PSP FLIGHT DYNAMICS:
%
% Title: GravityToBodyQuat
% Author: Preston Wright
% Last Modified: 8/30/25
%
% Description: Example converting a gravity force from the
%              inertial to body frame and thrust force from body to
%              inertial frame using quaternions

% Start with your initializations. Every angle should be
%   in radians, and our forces are in Newtons
phi = 2.71;                % x-axis Euler angle [rad]
theta = 0.2;              % y-axis Euler angle [rad]
psi = 0.2;                % z-axis Euler angle [rad]
gravityInertial = [-100;0;0]; % inertially expressed
%   gravity force [N]
thrustBody = [-1000;0;0]; % body expressed thrust
%   force [N]
omega = [2.5;0.1;0.1];    % angular velocity [rad/s]
bMatrixQuat = [0, -omega(1), -omega(2), -omega(3); ...
               omega(1), 0, omega(3), -omega(2); ...
               omega(2), -omega(3), 0, omega(1); ...
               omega(3), omega(2), -omega(1), 0];

% Before moving on to calculating our rotation matrix, we
%   want to convert the Euler angles to quaternions like so
%   (remember our rotation order!):
quat = eul2quat([psi,theta,phi],"ZYX");
```

```

% Normalized quaternions (versors) are the only way to
    express quaternion encoded rotations, so do that next
quat = quat/norm(quat);

% We can now call the function to find our rotation matrix
quatDCM = quat2dcm(quat);

% Multiplying our DCM by the gravity vector will give us
    the gravity vector expressed in the body frame.
    Multiplying the transpose (inverse) of the DCM by the
    thrust vector will give us the thrust vector in the
    inertial frame
gravityBodyQuat = quatDCM*gravityInertial;
thrustInertialQuat = (quatDCM')*thrustBody;

% Transpose the quaternion for matrix multiplication
quat = quat';

% Calculating our quaternion rate using the known formula
    appears as follows:
quatRate = (1/2)*bMatrixQuat*quat;

```

Taking a look at the outputs this generates, we see our results come out to be the following, with the gravity force expressed in the body frame and thrust expressed in the inertial frame as:

$${}^b\vec{F}_g = \begin{bmatrix} -96.0530 \\ -26.1902 \\ 9.3748 \end{bmatrix}; \quad {}^i\vec{F}_T = \begin{bmatrix} -960.5305 \\ -194.7092 \\ 198.6693 \end{bmatrix}$$

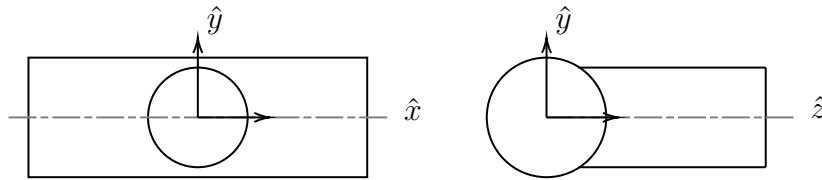
A quick note about this script in relation to the other: the results shown above and the results shown in [subsection 4.1.3](#) are identical. This gives confidence that we've implemented MATLAB's functions correctly. And once again, at this point, we would use our numerical integration scheme to approximate the next timestep's quaternion using the quaternion rates we just found. We won't do that here, but as a reminder, you can see the implementation in the 6DoF itself or the Dzhanibekov example!

4.3 3D Rotation Example: Dzhanibekov effect

To help digest the complex nature of attitude dynamics, a full example is especially useful. One of the consequences of Euler's Equations described above is that rotation about an intermediate axis is unstable. This effect is known as the Intermediate Axis Theorem or the *Dzhanibekov Effect*.

This is a good phenomenon to describe 3D rotations because it requires us to put together lots of knowledge throughout this section. Because our object rotates about all three axes, we must parametrize our rotation with quaternions to avoid possible singularities in the solution.

Before we can describe our system dynamics, we need to establish our coordinate system for this problem.



$$I_x = 1.202 \cdot 10^5 \text{ g/mm}^2$$

$$I_y = 4.615 \cdot 10^5 \text{ g/mm}^2$$

$$I_z = 4.010 \cdot 10^5 \text{ g/mm}^2$$

Figure 4.1: Non-Symmetric Body for Demonstrating Dzhanibekov Effect

4.3.1 State Vector

We'll start by describing our state vector and initial conditions for this problem:

We define our original position is at the origin, giving us a position vector:

$$\vec{r}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

We also define the initial velocity to be zero, giving a velocity vector:

$$\vec{v}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

We also need an initial orientation. We can either directly initialize a quaternion or write our initial orientation with Euler angles and then convert to quaternions. Often, Euler angles are more intuitive, so we will use these for our initialization. We can define an initial angle of all zeros, meaning that our body frame axes are initially coincident with the inertial frame. This gives Euler angles of:

$$\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Using the command `eul2quat` with a specification of `ZYX` for the frame, the initial quaternion vector is given as:

$$\vec{q}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Lastly, we need to describe the initial angular velocity. To properly demonstrate the effect, we need a small perturbation on our non-intermediate axes. We can write our omega vector as:

$$\vec{\omega}_0 = \begin{bmatrix} 0.05 \\ 0.05 \\ \pi \end{bmatrix} rad/s$$

Putting all of the together, we arrive at our state vector:

$$\vec{X}_0 = \begin{bmatrix} \vec{r}_0 \\ \vec{v}_0 \\ \vec{\omega}_0 \\ \vec{q}_0 \end{bmatrix}$$

Which we can expand to the full form as a 13-element column vector:

$$\vec{X}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0.05 \\ 0.05 \\ \pi \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

4.3.2 Forces and Moments

We can assume that we are operating in a 0-g environment (as this is where the effect is most prevalent) and that no body forces are acting on the object. We will also assume that air resistance is negligible and there are no losses. These assumptions mean that there are no forces that are acting on our body. Additionally, there are no moments on the body either. We have chosen this case for our example so that we can focus solely on the attitude dynamics.

4.3.3 Attitude Dynamics

Our attitude dynamics are fairly simple since there are no moments acting on the body. Our equations for the angular velocity rates in (2.14) reduce to the torque free equations:

$$\begin{aligned} \dot{\omega}_x &= \frac{(I_y - I_z) \omega_y \omega_z}{I_x} \\ \dot{\omega}_y &= \frac{(I_z - I_x) \omega_z \omega_x}{I_y} \\ \dot{\omega}_z &= \frac{(I_x - I_y) \omega_y \omega_x}{I_z} \end{aligned}$$

To find our quaternion rates, we simply use the same methods as in [subsection 4.2.2](#). This gives us:

$$\dot{\mathbf{q}} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \vec{q}$$

4.3.4 Translating to Code

With all of these defined, we can write the derivative of our state vector as:

$$\dot{\mathbf{X}} = \begin{bmatrix} \vec{v} \\ \vec{a} \\ \vec{\alpha} \\ \dot{\mathbf{q}} \end{bmatrix}$$

From here, we can implement this in `ode45`. We show all of the code for this model in the appendix.

Our initialization of the state vector and all of the basic setup is contained in the main file, shown in `DzhanibekovMain.m`. The `DzhanibekovIntegrator.m` code operates on the state vector using `ode45` to get the derivative. This code is . There is additionally a file to display the rotation of the body in an animation, which is `RotationVisualizer.m`. This code can be found in [Section C.2](#).

4.4 Key Ideas

1. Read Volume II to learn more about the theory behind MATLAB's functions
2. Rotations can be encoded in a variety of ways, but the most useful, compact, and robust method of encoding rotations involves the use of complex numbers.
3. The use of MATLAB's built in suite of functions is incredibly useful in the implementation of attitude dynamics in 6DoF modeling, and rotational simulations in general.

4. Be sure to keep your chosen rotation sequence straight when utilizing MATLAB's functions, and be sure to have a thorough understanding of how these functions work!
5. No matter which style of describing rotations we choose - Euler angles or quaternions - they both result in identical answers barring any singularities generated with the former.

4.5 Further Reading

Volume II of this text contains most of the information pertaining to attitude dynamics. For reference until Volume II is released, we find the website [14] especially useful for understanding quaternions.

4.6 Practice Problems

1. Your friend Jimothy, impressed by your MATLAB skills, has challenged you to model a spinning top. The top is axially symmetric about its $\hat{X}$ -axis and has an initial spin rate of 1200 rpm and has an initial angle to the vertical of 20° . The top spins down at a rate governed by an exponential decay of form $\omega(t) = \omega_0 e^{\frac{1}{10} \ln(2)t}$ such that the spin rate is halved every 10 seconds. Gravity acts on the top in the negative $\hat{x}$ -direction. The top has a mass of 20 grams. Relevant details are given in Figure 4.2. Define any additional quantities that you deem necessary to solve the problem.
 - (a) Write the state vector for the system.
 - (b) Write the EOMs for the system. If you choose the Newtonian approach, write the forces and moments of the system. If you choose the Lagrangian Approach, write the energy equations and constraint equations.
 - (c) Numerically integrate the system. For this problem, you may be able to use either Euler angles or quaternions if you choose your singularity point carefully.
 - (d) Create a visualization of the system. Create any other plots that you deem useful.

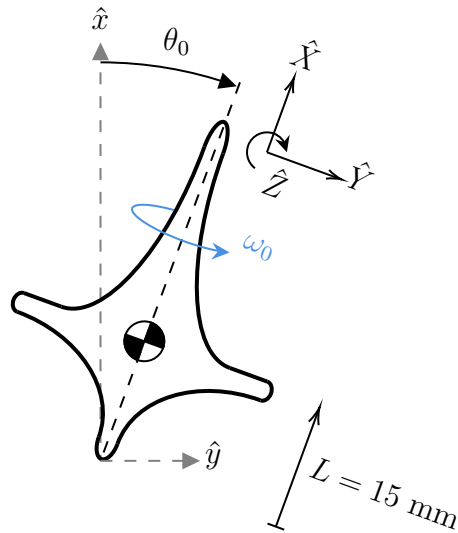


Figure 4.2: Spinning Top Set-Up

2. After your stint in modeling the Explorer 1 Satellite, NASA has asked you to come back to model a new satellite. NASA wants the satellite to be nadir-pointing at all times, and has decided to use gravity gradient stabilization to do so.

3. You find yourself flying a UH-1 Huey over Hamburger Hill in Vietnam, 1969. While preparing to land on your medevac mission, you glance at your inertial navigation system and are horrified to find it has been blown away by a RPG-7! You must now make the calculations necessary to determine how much of your thrust will be acting directly along the ground. Should this force exceed 1,000 N, the soldiers on the ground will be blown away. You decide to utilize a rotation sequence identical to what was used on the navigation system: a 3-2-1 sequence. With this, you determine the Euler angles to be 0, 0.15, and -0.05 radians, and the force of the engine is known to be 10,000 N. Will the soldiers go tumbling away as you land? A view of what you see with the reference frames imposed on it is shown below:

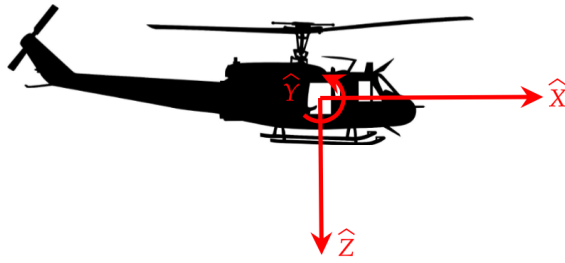


Figure 4.3: Huey Reference Frame

Notes

12. Volume II is yet to be released, and is currently scheduled to do so late Winter.
13. These intermediate rotations about the body frame are sequentially applied to our system as we move from frame to frame in our chosen rotation sequence.
14. Note there exists another similar function - `eul2rotm` - that also creates a DCM; however, this function converts a vector from the body frame to the inertial frame.
15. We are allowed to use the transpose here as the DCM is an orthogonal matrix.

CHAPTER 5

Aerodynamic Modeling

*“When I meet God, I am going to ask him two questions: Why relativity?
And why turbulence? I really believe he will have an answer for the first.”*

– Werner Heisenberg

In [Section 2.4](#) we discussed the 4 fundamental forces on a vehicle and how we define their directions. In that section we did not define the magnitude of the aerodynamic forces, $\vec{F}_L$ and $\vec{F}_D$. The magnitude of these forces are quite difficult to calculate and require the most approximation to do correctly. The governing equation for calculating the magnitude of our aerodynamics is $\frac{1}{2}\rho_\infty V_\infty^2 SC_X$, where C_X is either C_L , C_D , C_N , or C_A .

Despite the innocuous appearance of this equation, calculating the aerodynamics of our system is deceptively hard because the calculation of C_X is very complicated. In fact, without real data, any calculation that we made of these values will just be an approximation.¹⁶ That being said, there are approaches we can take which will get us close enough to true values to prove useful.

5.1 Non-Dimensionalization

Modern aerodynamics uses non-dimensional quantities very frequently. As a result, we should start by understanding how non-dimensionalized quantities emerge. This will help us understand why we have many equations with the term $\frac{1}{2}\rho V^2$, or why the mach number is important. Along the way, we will also discover some other important properties that influence the airflow.

Since our main goal is to understand the lift and the drag on an object, we will

start by enumerating the various factors that could impact the aerodynamic forces experienced by an object. Some intuition will tell us these forces are likely affected by the following factors:

1. Viscosity of the fluid - higher viscosity creates more ‘stickiness’ in the air that needs to be countered.
2. Velocity - higher velocity creates more interactions with the fluid at any given time, so it will contribute to our drag force.
3. Density of fluid - higher density will create more interactions per time.
4. Characteristic size - the size of the object traveling through the fluid will increase the particle interactions per time and drag.
5. Speed of sound - the properties of the fluid will depend on the speed of sound in the fluid.
6. Angle of attack - the angle of the object will change the apparent size in the airflow.

Already, we can see that some of the terms that we see in the lift and drag equations are present above. However, we do not know in what way they are correlated and what else they depend on.

To determine these dependencies, we can invoke *dimensional analysis* and the *Buckingham Pi Theorem*. Both of these will allow us to derive dimensionless quantities that prove to be particularly useful.

Theorem 2 (Buckingham Pi Theorem). *Given a system with N parameters and K physical dimensions, it is possible to express the result of this system with P dimensionless Π groups, where $P = N - K$.*

Corollary 2.1. *Additionally, each Π group is independent of each other. Thus, each Π group is a function of all the other Π groups:*

$$\Pi_1 = f(\Pi_2, \Pi_3, \dots, \Pi_{N-K})$$

and so on for the other Π groups.

For more information on Buckingham Pi Theorem, refer to [\[26\]](#).

Example 5.1: Aerodynamic Forces

Using the system characteristics described before, let us use Buckingham Pi Theorem to write the non-dimensional groups for our aerodynamic system. We start by writing the dimensions for each of the variables we enumerated:

Variable	Dimension
ρ	$\frac{M}{L^3}$
V	$\frac{L}{T}$
S	L
μ	$\frac{M}{LT}$
a	$\frac{L}{T}$
α	—

Applying Buckingham Pi Theorem, we can create 4 Π groups from the 7 variables. The solving of these Π groups is shown in the Appendix in [Section B.1](#) because it is quite lengthy.

Solving the Π groups, we get:

$$\begin{aligned}\Pi_1 &= \frac{\rho V S}{\mu} \\ \Pi_2 &= \frac{V}{a} \\ \Pi_3 &= \frac{F}{\rho V^2 S} \\ \Pi_4 &= \alpha\end{aligned}$$

Π_1 is a quantity known as the Reynolds number, the ratio of inertial forces to viscous forces. Π_2 is the Mach number, the ratio of velocity to the speed of sound of the fluid. Π_3 is the force coefficient that we have described earlier. Π_4 is the relation for the angle of attack, which is just itself since α is unitless.

From [Corollary 2.1](#), we can also write

$$C_x = f(Re, M, \alpha)$$

The conclusion drawn from the Buckingham Pi Theorem is that we can determine the force coefficients using only the Re , M , and α . Thus, whenever we do modeling of aerodynamics, we only need to consider Reynolds number, Mach number, and angle of attack. This is a cornerstone of how we model aerodynamics in our 6-DoF modeling.

5.2 Aerodynamics Terminology

Aerodynamics has a lot of terminology associated with it that is important to understand before we can approach modeling the aerodynamics of the system. We will mostly cover aerodynamics terminology for rockets, but the modeling done here should be transferable to aspects of aircraft systems as well.¹⁷

While some of the theory here is not directly used in our 6-DoF, having an understanding of this will be greatly beneficial so we have chosen to include some background here.

5.2.1 Lift and Drag

We start this section by noting that lift and drag are just conventions that are commonly used to describe the aerodynamic force. It should be noted that both the lift and the drag arise from the same source - the air acting on the vehicle to create a force. It is simply convenient to define this resultant force with respect to the direction of the freestream velocity, which we call lift and drag.

When modeling aerodynamic forces, we can imagine that two differential force elements act at each point along the body. At each point, there is a pressure which acts normal to the surface and a shear stress (you can think of it as a friction) that acts tangentially to the surface. This shear stress arises from the viscosity of the air. These are shown in [Figure 5.1](#).

Since a force is a pressure multiplied by an area, $F = pA$, we can integrate our pressure distribution over an area to understand the total forces that are acting on a geometry. Often, we see this expressed in the form of an integral over the length of the geometry. This length is denoted by c , the chord length, in the example of the

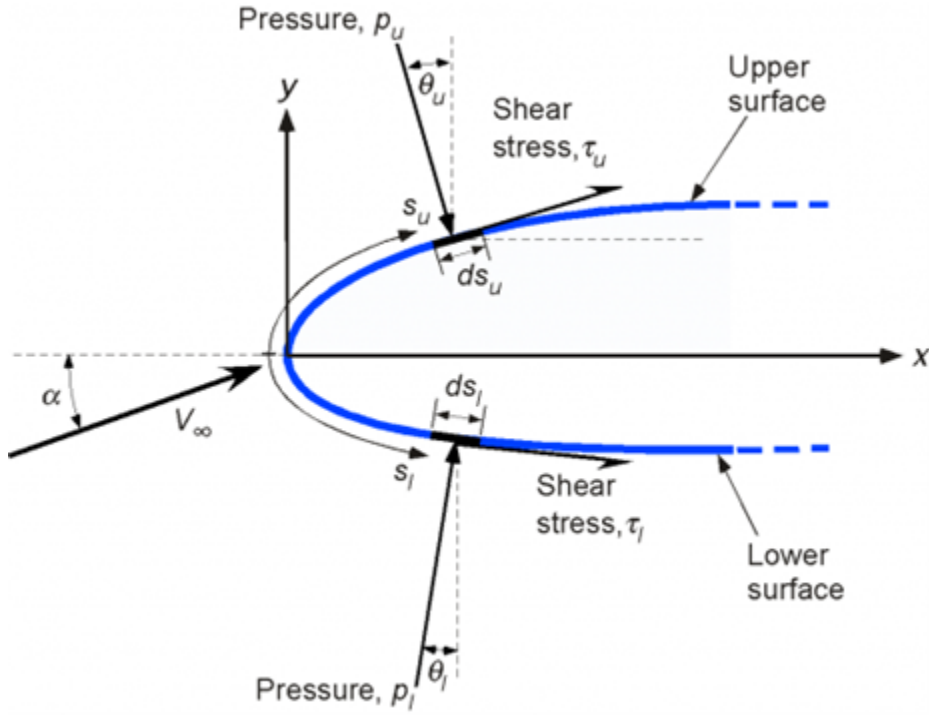


Figure 5.1: Pressure and Shear Stress on an Aerodynamic Surface

airfoil. These integrals are shown below:

$$N' = \int_0^c p_l u - p_u dx + \int_0^c \left[\tau_u \frac{dy_u}{dx} + \tau_l \frac{dy_l}{dx} \right] dx$$

$$A' = \int_0^c \left[p_u \frac{dy_u}{dx} + p_l \frac{dy_l}{dx} \right] dx + \int_0^c (\tau_u + \tau_l) dx$$

Where the N' and A' are the normal and axial force on the aerodynamic surface. Here, the apostrophe (We would say ‘N prime’) refers to the fact that the force acts over a unit length in the $\hat{z}$ direction. We are assuming a unit length here since we are considering the 2-D case in this example. More generally, we would perform this calculation in 3D, but that is beyond the scope of this example.

The normal force acts normal in the $\hat{y}$ direction and the axial force acts in the $\hat{x}$ direction as shown in [Figure 5.1](#). Normal and axial forces are not commonly used in our modeling outside of these calculations. More often, we see the quantities lift and drag, which are defined with respect to the freestream velocity. We can easily transform our normal and axial forces into a lift and drag using a rotation matrix

with our angle of attack, α . This rotation matrix takes the form:

$$\begin{bmatrix} D' \\ L' \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} A' \\ N' \end{bmatrix} \quad (5.1)$$

We can expand (5.1) to give two scalar equations:

$$\begin{aligned} L' &= N' \cos \alpha - A' \sin \alpha \\ D' &= A' \cos \alpha + N' \sin \alpha \end{aligned}$$

So, if we know the distribution of pressure over our body, we have shown that we can find the resultant aerodynamic forces acting on our body. As briefly described in subsection 2.4.3, another important parameter to find is our center of pressure.

5.2.2 Center of Pressure

Simply stated, the center of pressure is the location at which all the aerodynamic forces acting on a body produce no resultant moment. This location is particularly useful for our case because we can assume that the lift and drag forces act through the center of pressure rather than assuming that lift and drag act as force distributions. This will make our future calculations much simpler, so we would like to know the center of pressure and how to calculate it.

To find the location where the center of pressure occurs, we can take the moment about any point first. Often, we choose to take the moment about the nose of the rocket (called the leading edge when referring to a wing), since we define the origin here. This integral has a similar form to the normal and axial force integrals except that we multiply the force by the distance from the nose. For more information on why this is done, refer to [7]. Writing out this integral, we get:

$$\begin{aligned} M'_{LE} &= \int_0^c p_u - p_l dx + \int_0^c \left(\tau_u \frac{dy_u}{dx} + \tau_l \frac{dy_l}{dx} \right) x dx \\ &+ \int_0^c \left(p_u \frac{dy_u}{dx} + \tau_u \right) y_u dx + \int_0^c \left(-p_l \frac{dy_l}{dx} + \tau_l \right) y_l dx \end{aligned}$$

Knowing the normal force and the moment about the leading edge, we can find the center of pressure location as:

$$x_{cp} = \frac{-M'_{LE}}{N'}$$

We note that the convention for the moment about the nose is for a positive moment to denote a pitch up (this is why the negative sign is present). This is opposite the convention established by the RHR. For all other moments used in our 6-DoF models, we use the standard RHR convention for moments.

For more information on the derivation of the equations in this section, refer to Chapters 1 and 2 of [1].

For a rocket, we generally consider the body to be *axisymmetric* about the longitudinal axis, when the whole structure can be rotated about the axis which we define as $\hat{X}$.

Because our rocket is axisymmetric, we assume that the center of pressure always lies along the longitudinal axis. Although this assumption is not necessarily true, as we will see later, the largest effects on the aerodynamic moments will occur from changes along the longitudinal direction rather than changes along the transverse plane. We can think of this intuitively because the center of pressure is separated from the center of mass by more than the diameter of the rocket, so changes in this direction have a greater effect.

5.3 Atmospheric Modeling

Another hurdle to overcome when describing the aerodynamics of our system is to find values for our freestream quantities. We explore the calculation of V_∞ in [Chapter 2](#). The other parameters we need to calculate are ρ_∞ and a_∞ . Fortunately, the standard density and Mach number of the atmosphere on Earth is well defined. In MATLAB, we can find this density at any altitude with `atmosisa` as:

```
[T, a, P, rho] = atmosisa(height);
```

This gives us a standard value of ρ_∞ and a_∞ to use. However, the value of can vary by up to 10% depending on the weather conditions [20]. Similarly, the Mach number has a strong dependence on temperature ($M = \sqrt{\gamma RT}$), so the free-stream Mach number may also be different by 10-20% of standard values. Further derivation of this mach relation can be found in [6].

All of this is to say that using `atmosisa` gives us a good baseline model of the atmosphere, but is not the final say in atmospheric modeling. For now, we use

`atmosisa` in our modeling because we can live with this variation in our model. However, it may be useful in the future to use other sources of atmospheric data that give location- and time-specific data.

5.4 Computational Aerodynamics

In the case of our rocket, we encounter a variety of flight regimes from subsonic to high supersonic. Because of this variety, hand calculations of our aerodynamics can become very complicated. As a result, we choose to use computational tools to compute the aerodynamics of our rocket.

5.4.1 RasAero

The main tool that we use presently is RasAero. This tool allows us to input the geometry of the rocket and will output the drag coefficient as a function of the Mach number. This is less than perfect, as we lose some of the dependence on Reynolds number. Additionally, the dependence on angle of attack is quite coarse (only measurements every 2°). That being said, RasAero has been used to model many rocket systems around the same scale as our rocket with an average error in apogee of 3.47% and 80% of the flights within $\pm 10\%$ of the true apogee. For us, this is an acceptable compromise so that we can focus on flight dynamics modeling rather than aerodynamics.

More information on RasAero can be found at [\[17\]](#). The documentation on this page does a better job explaining the software than we could here in a short summary. Our implementation uses the .csv export files from RasAero in MATLAB.

5.4.2 Other Computational Models

It is worth noting that other computational models exist. RasAero does have shortcomings in modeling some of the aspects of the rocket geometry, so other methods such as Computational Fluid Dynamics (CFD) are also options to be explored. However, in the current state of our model, we have not looked into CFD methods very heavily for direct use in MATLAB. These are in the preliminary stages of

investigation and will likely be explored more heavily in future editions of this text.

5.5 Key Ideas

1. When we model a system such as our rocket, we can capture most of the phenomena by capturing data at different Re , M , and α . This is true for both CFD and wind tunnel modeling.
2. While exact analytical solutions are hard to attain in aerodynamics, we can use computational tools to estimate aerodynamics

5.6 Further Reading

There are many good resources for aerodynamics modeling. The first among these, which serves as a good introduction to the topic of fluid dynamics, is [1].

For more complex studies of aerodynamics, we recommend Modern Compressible Flow by Anderson.

5.7 Practice Problems

1. You are flying home one day in your P-51 Mustang (pictured in Figure 5.2 for your viewing pleasure). At cruise, your aircraft is 10,000 ft above sea level.
 - (a) What is the density of air at this altitude?
 - (b) Your wings provide a C_L of 0.3 at cruising speed. Your plane has a mass of 4,000 kg and a planform area of 21.8 m². Find the speed necessary to maintain level flight at this altitude.
 - (c) Your airplane is equipped with unguided HVAR rockets. As a daring pilot, you want to model the aerodynamics of this rocket so you can launch it at your friend's house with astounding accuracy (*that* will teach them to never steal your mini corn dogs again).



Figure 5.2: P-51D-5NA Aircraft

Using your best engineering judgment of the nose cone shape and any other parameters you cannot find, model the HVAR rocket in RasAero.

- (d) As it turns out, shooting rockets at Canadian civilians is illegal (I know, right?). The Royal Canadian Air Force F/A-18's are on your tail now and you will be shot down in 15 minutes unless you can make it to US airspace. You are 160 km from the airspace boundary. You push your throttles to max to book it back home.

At full throttle, your Merlin V12 produces 1676 hp (1250 kw). Your aircraft has a C_D of 0.017. At the same altitude of 10,000 ft, find the maximum flight speed. Will you make it back to American airspace in time?

Notes

16. Even with wind tunnel data, it is still likely that some approximations need to be made. Aerodynamics in general has no definite answers.
17. The principles we describe are the same, but much of the modelling is simpler for rockets since they have axial symmetry in most cases. We will utilize these assumptions early on to simplify this section.

CHAPTER 6

Numerical Integration Schemes

“The best that most of us can hope to achieve in physics is simply to misunderstand at a deeper level.”

– Wolfgang Pauli

In our exploration of numerical integration schemes, it will be useful to have a good understanding of ordinary differential equations. We highly recommend utilizing Source [5] for reviewing differential equations.

We note that this section does not include practice problems since most of the other practice problems include numerical integration as part of the process.

6.1 Euler Integration

We have previously discussed the simplest integration scheme, the explicit Euler method, which we used in [subsection 2.1.1](#). Understanding the Euler method is key to understanding and using more complex integration schemes, so we recommend writing a few of your own Euler integration scripts in MATLAB or a coding language of your choice to really solidify your knowledge before moving on to more complex integration schemes.

We formalize the explicit Euler integration below. Given a linear ODE:

$$\frac{dy}{dt} = f(x, y)$$

With initial condition:

$$y(x_0) = y_0$$

We perform Euler integration first by defining an interval over which to perform the numerical integration. For Euler integration, we equally space these points by our timestep, dt . We can parameterize these points as:

$$x_i = x_0 + i\Delta t$$

Our next step is to find the value of the tangent line at each point. This is trivial, since our differential equation tells us what $\frac{dy}{dt}$ looks like at any value. All we have to do is evaluate:

$$\frac{dy}{dt} = f(x_i, y(x_i))$$

The problem here is that $y(x_i)$ is an unknown quantity (if we knew it, we wouldn't have to do integration). So, to find the value of $y(x_i)$ we take an iterative approach. We know the starting value, $y(x_0) = y_0$, so we can find an approximation of the new value by multiplying the timestep by the current slope. For the first timestep, this will look like:

$$y_1 = y_0 + \Delta t \cdot f(x_0, y_0)$$

Now that we have found an approximation for y_1 , we can extend this to find approximations for y_2 , y_3 , etc. This formula is generally extensible as:

$$y_{n+1} = y_i + \Delta t \cdot f(x_i, y_i)$$

Euler's Method

We represent this Euler integration scheme graphically with a simple example. Suppose our differential equation is $\frac{dy}{dt} = 2t$ with initial condition $y(0) = 0$. In this example, we can see from inspection that the solution curve is $y = t^2$. Performing Euler integration with a Δt of 0.5 yields the black approximation in [Figure 6.1](#).

As can be seen from [Figure 6.1](#), large timesteps quickly lead to error in the solution. For a strictly convex curve, our Euler approximation will always give us an under-approximation of the solution. In some cases, however, our solution may diverge so greatly that it becomes unstable and values tend toward infinity.¹⁸

6.1.1 Errors in the Euler Method

We will use the example of the Euler Method to demonstrate error in numerical integrators. These same concepts will apply when we move on to higher order integrators.

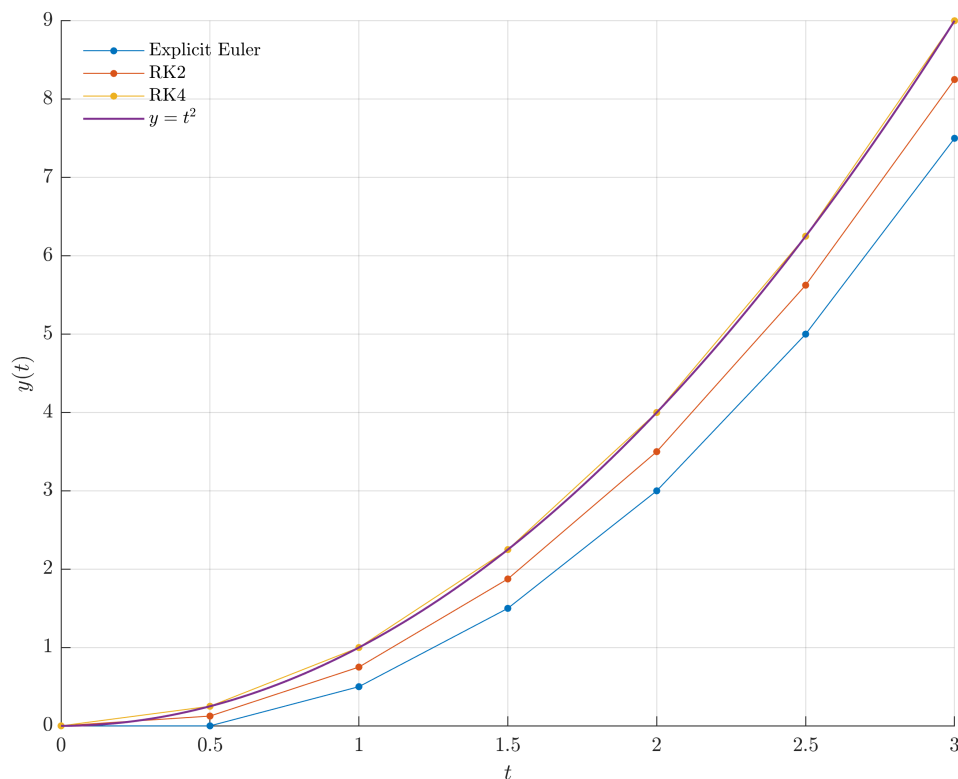


Figure 6.1: Explicit Euler Method Numerical Approximation

Errors in the integration scheme come from two different sources:

1. Euler integration schemes only account for linear terms because it only considers the slope of the tangent line at a point. Thus, any quadratic order terms or higher will not be accounted for. Thus, the error is proportional to the timestep squared, Δt^2 .¹⁹ We call this error *truncation error*.

2. Another problem is the fact that computers don't have infinite precision. The loss of precision during computing is called *roundoff error*. Computing more timesteps in total will increase this roundoff error.

It is worth noting that these errors are per timestep! So over the course of a full simulation, our errors will grow much larger than the error bounds that we've calculated above. In fact, for the Euler Method described above, the global error is proportional to the timestep, Δt .

We can of course, reduce errors in the simulation by reducing our timestep, Δt . However, this will also increase the computation time. Additionally, doing so will increase the roundoff error in the simulation. With more calculations being done, the roundoff error will have a larger impact. Thus, it is worthwhile to investigate other integration schemes.

6.2 Improved Euler / RK2

Given the limitations of the Explicit Euler method, we will start by investigating the improved Euler Method. This improved Euler method is the first of a family of integration schemes known as Runge-Kutta (RK) integration schemes. The number, 2, represents the order of the integration scheme, which we will explore more later.

The main area that led to inaccuracies in the explicit Euler method was the slope that we calculated. We can improve on this by taking two points and finding the average slope. We choose to take a point at our start and our end point, x_i and x_{i+1} , and evaluate the slope at both of those points and average them. From the previous section, we can remember that:

$$\frac{dy}{dt} = f(x_i, y(x_i))$$

So, we can write the average slope as

$$m = \frac{\dot{y}_1 + \dot{y}_2}{2}$$

Where

$$\begin{aligned}\dot{y}_1 &= \frac{dy_1}{dt} = f(x_i, y(x_i)) \\ \dot{y}_2 &= \frac{dy_2}{dt} = f(x_{i+1}, y(x_{i+1}))\end{aligned}$$

Now that we have the slope, we follow the same process as we did for the Explicit Euler method to find the next value. We take the slope and multiply it by the timestep, Δt , and add the current value, $y(x_i)$. This gives us our approximation as:

$$y = y(x_i) + \frac{\dot{y}_1 + \dot{y}_2}{2} \Delta t$$

We can write this then as:

$$y_{i+1} = y_i + \frac{\Delta t}{2} [f(x_i, y(x_i)) + f(x_{i+1}, y(x_{i+1}))]$$

To rewrite this equation in a way that is better suited for code:

$$y_{i+1} = y_i + \frac{\Delta t}{2} (k_1 + k_2) \tag{6.1}$$

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f(x + \Delta t, y_i + \Delta t k_1) \end{aligned}$$

We can note here that this method requires at least two evaluations since k_2 is dependent on k_1 . However, this is often still more computationally efficient than the Explicit Euler method because larger time steps can be used with lesser error.

6.2.1 RK2 in Code

Implementing the RK2 algorithm in code is slightly more complicated than implementing the explicit Euler method. To properly implement this, we will use *function handles* in MATLAB. Function handles act as pointers to an instance of a function. To explain what that means, we will examine the following code snippet:

```
f = @computeSquare;
a = 4;
b = f(a)
```

where `computeSquare` is defined as:

```
function y = computeSquare(x)
y = x.^2;
end
```

The function handle allows us to assign the function to a variable, which we are calling `f`. We call this a pointer because this variable ‘points’ to the function `computeSquare`.

In this example, when we call this operation, we get `b=16` in the command window. The power of this is that we could instead pass in a vector of values for `a` and get outputs from the function for all of them. If we instead input:

```
f = @computeSquare;
a = [4,6,10];
b = f(a)
```

We get an output of `b = 16 36 100`. The usefulness of this is immediately apparent for numerical integration. We can pass in a vector of many different elements and get outputs for each of them.

Another useful feature of function handles for numerical integration is that they allow us to pass in only the variables needed for numerical integration and pass other variables in the expression as constants. In general, we want our numerical scheme to only accept our state vector and the time vector. This is the same thing that `ode45` does, so it is best practice to do the same thing here.

Our RK2 algorithm is a direct translation of (6.1) into code. We present this below in [Script 6.2.1](#):

Code Listing 6.1: RK2 Integrator

```
% RK2 Integrator
% numerically integrates a first order initial value
    problem using Runge-Kutta 2nd order.
% inputs:
% fun - function of integration
% dt - time step [s]
% tIn - input time [s]
% xIn - initial value input vector
% outputs:
% out - final value output vector
function out = rk2(fun, dt, tIn, xIn)
    f1 = fun(tIn,xIn);
    f2 = fun(tIn + dt/2, xIn + dt .* f1);

    out = xIn + (dt / 2)*(f1 + f2);
end
```

6.3 RK4 Integration

RK4 integration is the last major integration scheme that we will explore. RK4 integration is often the best balance between the precision of the algorithm and the speed of computation.

To explain the RK4, we will go back to the RK2 and explain it in a slightly different way that will shed light on what we are doing with RK4. In RK2, we described taking the average slope of the function. We can also imagine taking the Taylor expansion of our function:

$$y_{i+1} = y_i + \dot{y}_i \Delta t + \ddot{y}_i \frac{\Delta t^2}{2} + O(\Delta t^3)$$

From this Taylor expansion of the solution, we can see that our approximation accounts for linear and quadratic terms. Thus, our error is proportional to terms to the 3rd power, represented by $O(\Delta t^3)$. We know y_i and $\dot{y}_i$, but the 2nd order derivative of our function is unknown. We can approximate this 2nd order derivative as:

$$\ddot{y}_i = \frac{\dot{y}_{i+1} - \dot{y}_i}{\Delta t}$$

Upon rearranging this equation, we can see that we arrive at the same expression as we did before. We can apply the same approach for the RK4 integration, except we consider terms up to the 4th order expansion of the Taylor series:

$$y_{i+1} = y_i + \dot{y}_i \Delta t + \ddot{y}_i \frac{\Delta t^2}{2} + \dddot{y}_i \frac{\Delta t^3}{6} + \dots + O(\Delta t^5)$$

Immediately, we can see that our error has shrunk to be proportional to terms of the 5th order, a dramatic reduction as compared to RK2 or Explicit Euler method.

6.3.1 RK4 Integration in Code

The RK4 Integration scheme can be expressed as a set of equations very similar to what we did with (6.1). We will not derive these equations here, because the process is very similar to that used for the RK2 method.

The form of these equations is shown in (6.2).

$$y_{i+1} = y_i + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4) \quad (6.2)$$

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= \left(x_i + \frac{\Delta t}{2}, y_i + \frac{\Delta t}{2} k_1 \right) \\ k_3 &= \left(x_i + \frac{\Delta t}{2}, y_i + \frac{\Delta t}{2} k_2 \right) \\ k_4 &= f(x_i + \Delta t, y_i + \Delta t k_3) \end{aligned}$$

An implementation of this algorithm into code is shown below.

Code Listing 6.2: RK4 Integrator

```
%% RK4 Integrator
% numerically integrates a first order initial value
% problem
% inputs:
% fun - function of integration
% dt - time step [s]
% tIn - input time [s]
% xIn - initial value input vector
% outputs:
% out - final value output vector
function out = rk4(fun, dt, tIn, xIn)
    f1 = fun(tIn, xIn);
    f2 = fun(tIn + dt/2, xIn + (dt/2) .* f1);
    f3 = fun(tIn + dt/2, xIn + (dt/2) .* f2);
    f4 = fun(tIn + dt, xIn + dt*f3);

    out = xIn + (dt / 6)*(f1 + 2*f2 + 2*f3+f4);
end
```

Often, we use a function in MATLAB called `ode45` to perform RK4 integration. This function has the benefit that it will change the step size to produce high accuracy when needed and decrease computation time otherwise.

An understanding of the operation of `ode45` is best motivated by an example. We include an example of a Lorenz Attractor in the Appendix [Section C.1](#) as an

example of `ode45`. We recommend reading through this code and running it, as well as changing parameters to see how it affects the outputs.

6.3.2 ODE45 Usage

While examples are helpful, it is useful to see the inner workings of `ode45` for a new user.

`ode45` relies on the function handles that we mentioned earlier. `ode45` takes as inputs the integration function, time span, and initial state vector.

`ode45` only takes integration functions that are a function of time and the state variable, or (t, x) in MATLAB notation. If we have an integration function that is a function of more than just these two variables, we need to pass them through using function handles. We can do this in the following manner:

```
@(t,x) integrator(t,x,a,b)
```

In this example, we have a function that performs our numerical integration, called `integrator`, the output of which will be the derivative of our state vector. We want `ode45` to see this function and only input (t, x) , so we put that in a function handle at the front. After the function handle, we are free to pass in any other variables, such as `a` and `b`. These will simply be read in as constants in the `ode45` integration scheme.

The benefit of adding these in as a function pass through instead of being computed in `ode45` will become apparent when we discuss code optimization in [Section 6.6](#). This also keeps our code hierarchy cleaner and more easily modifiable.

6.4 Other Numerical Integration Considerations

6.4.1 More on Error

We briefly discuss the error of the integration schemes in each of the previous sections. However, it is useful to see examples of these to solidify understanding.

Using the same example differential equation from subsection 6.1.1 as $\frac{dy}{dt} = 2t$, we can show the approximation for Explicit Euler, RK2, and RK4. This is shown in Figure 6.2. The code that runs this example is shown in Section C.3 in the Appendix.

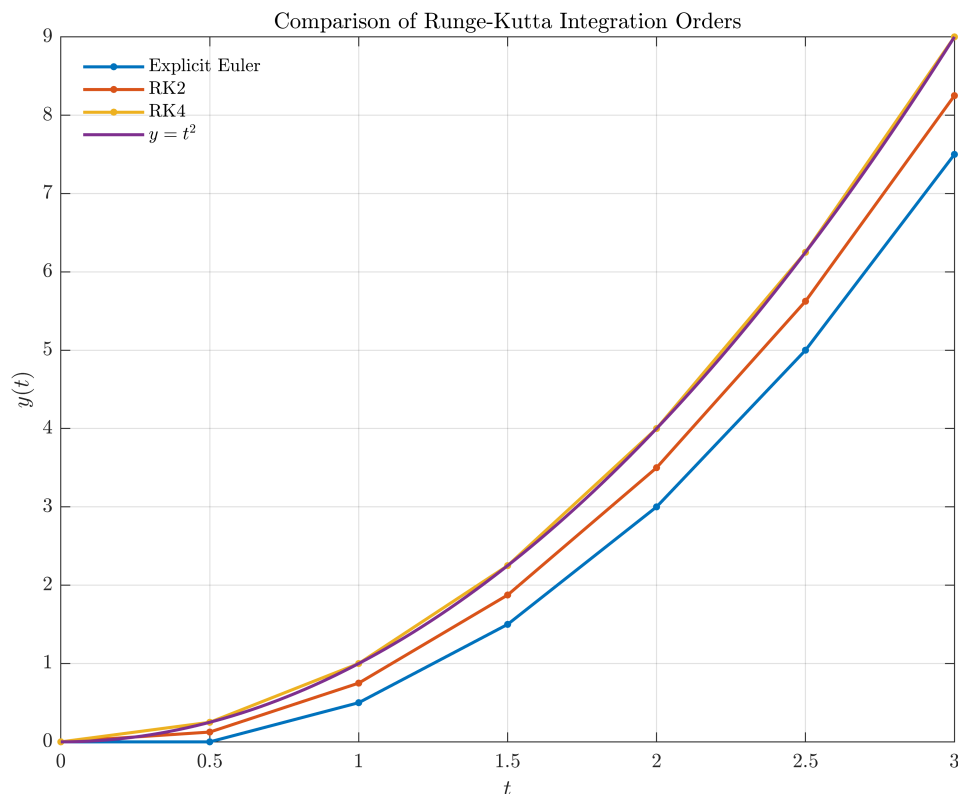


Figure 6.2: Comparison of Numerical Integration Schemes for $y' = 2t$

This code is a good example for understanding the syntax for using the integration code snippets in Listing Script 6.2.1 and Script 6.3.1.

As we can see from this example, even with a fairly large timestep of $\Delta t = 0.5s$, the RK4 perfectly matches the true curve. For this curve, there are no terms of higher order than quadratic, so the RK4 will perfectly match the solution curve.

The difference in these numerical integration schemes will become more apparent when modeling the behavior of different differential equation systems. Systems containing higher order terms will be better modeled by the higher order integration schemes, unsurprisingly. The true use of these methods is when analytical solutions

to the differential equations are hard or impossible to obtain. That being said, we still want to know characteristics of our system so we can appropriately select integration parameters.

6.4.2 Sampling and Stiffness

To begin our discussion of sampling and stiffness, we can take the differential equation $\frac{dy}{dt} = \cos(4\pi t)$ with the initial condition of $y(0) = 0$. This function is periodic in nature, so it will be good for our demonstration here. In this first example, we will keep the same timestep of $\Delta t = 0.5s$. As is seen in [Figure 6.3](#), this timestep is the same as the period of the sinusoidal solution of this differential equation. When this is the case, we can see that the solutions diverge from the original periodic function greatly for all of the integration schemes (aside from the RK2, although this is only the case when the period is exactly the same as the timestep). This occurs because we are not sampling the function at a high enough frequency to capture the nature of the periodic function.

So, what timestep is required to accurately model a periodic function? For our example, we will use RK4 since we have shown it to be the most accurate of the integration schemes. We choose to do this to highlight effects aside from error in the numerical approximation. We can vary the timestep for this same periodic function and see what the response looks like. This is shown in [Figure 6.4](#).

In this figure, we note several important things. When our timestep is slightly less than the period of the function, we infer a lower frequency. This is seen when we use a timestep of $\Delta t = 0.4s$. We can intuitively explain this as a *beat frequency*. We know that the solution curve has a frequency of 2 Hz, and a wave with a period of 0.4s would have a frequency of 2.5 Hz. Thus, a beat frequency of 0.5 Hz would be produced. This is exactly what is seen, as the resultant solution appears as a sine wave with a frequency of 0.5 Hz (2 second period).

The same is seen when Δt is close to but not quite double the frequency of the wave (called *bandwidth* of the signal). The perceived signal has a changing amplitude with a superimposed sinusoid. This again, does not match the true signal. It is only when we sample at the *Nyquist rate* that a true approximation of the function is found. In general, we must sample our function at twice the highest frequency present to ensure a good approximation!

In this example, this criterion does not greatly affect our numerical integration.

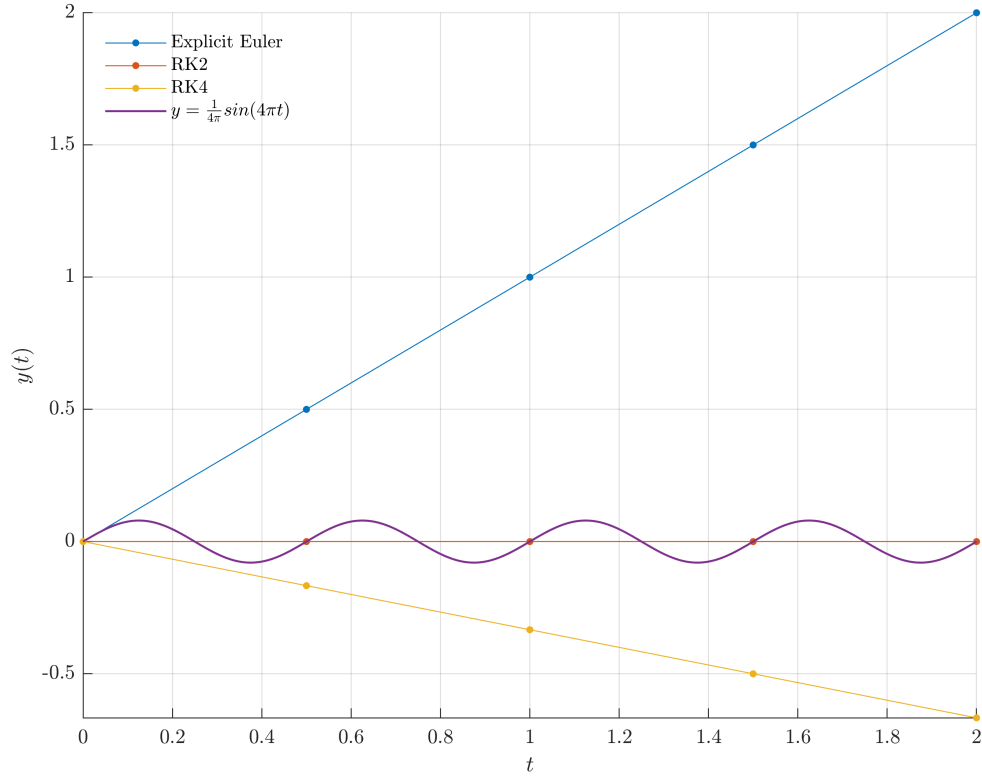


Figure 6.3: Numerical Integration of periodic function

However, when we have other dynamics in the system, the need to sample at double the highest frequency of the system may be detrimental to the performance of our simulation. We call such systems that are difficult to numerically integrate *stiff*.

Stiff systems make numerical integration challenging not only because of increased simulation time, but also because of aforementioned truncation error. In a 6-DoF model such as ours, this is important if vibrational modeling is considered. As an example, slosh modeling and parachute modeling (which are not deeply discussed here, but may be important aspects of a rocket 6-DoF) are often based on mass-spring-damper systems.

It is best practice to try and decouple the simulation of high-frequency vibrations from the flight dynamics of the system. Sometimes this may be unavoidable, such as aircraft with large amounts of wing flexure where aerodynamics and elastic defor-

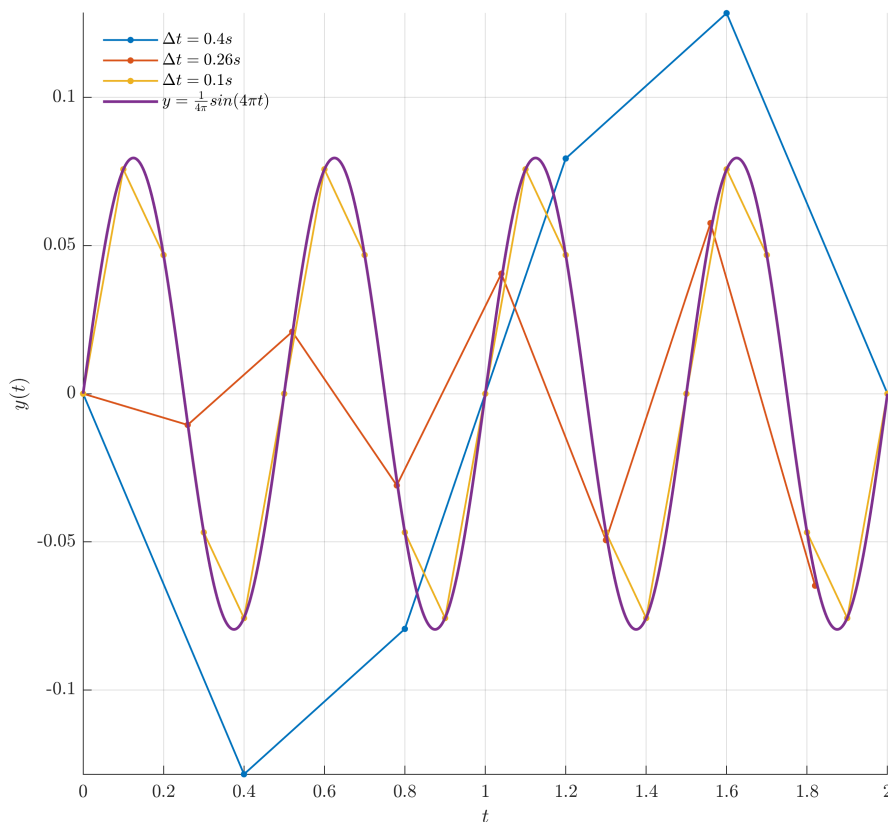


Figure 6.4: Numerical Integration of Periodic Function with varying timestep

mation are heavily coupled. However, this should be attempted whenever possible. This problem is very complicated and is the subject of much ongoing research! More information about such dynamics can be found in [19].

6.5 Other Integration Schemes

While `ode45` works very well for most use cases with numerical integration, there are cases where other integration schemes should be considered. We will mainly discuss the implementations that exist in MATLAB. More information about such systems can be found at [9] and [13].

We previously discuss the concept of stiff systems in [subsection 6.4.2](#).

For systems that have moderate stiffness, we can use `ode23` instead of the more typical `ode45` in MATLAB. Using a lower order integration leads to more course results, but it also allows very stiff parts of the system to be computed faster.

On the converse side, for systems that require very high precision, such as orbits, algorithms of the 8th or 9th order are often used. In MATLAB, such algorithms are used in the `ode78` and `ode89` functions. Using a higher order, the truncation error can be significantly reduced because fewer timesteps are needed when very high tolerances are required. However, in most scenarios, there algorithms are about 300-500% slower than `ode45` while providing few accuracy benefits. Thus, these algorithms are mostly used for only very high precision use cases.

6.5.1 Symplectic and Variational Integrators

In addition to these numerical integration schemes, there are a variety of integration schemes that use energy conservation as the basis of their integration schemes. These types of integrators are called symplectic integrator. These types of integrators directly solve Hamilton's equations for a Hamiltonian system. The form of these equations is discussed briefly in [Section 3.3](#).

Because the Hamiltonian equations are first order already, they are ready for numerical integration without the need for order reduction.

A similar operation can be done to reduce the Lagrangian to two first order equations, in a system known as a variational integrator.

MATLAB has no built in functions for variational or symplectic integrators, so we do not discuss them further here for now. Further discussion is in Volume III.

Use Case of Symplectic Integrators

Although MATLAB does not have built-in functions for these integrators, scripts to do so can be written or found online.

The best use case for these algorithms is for systems with a Hamiltonian or Lagrangian description that need to be integrated over a long time span. Because these

integrators conserve energy, they will better preserve the characteristics of these systems over long times as compared to RK integrators.

6.6 Numerical Integration Code Optimization

6.6.1 Monte Carlo Simulation

Often, we want to run many simulations with various parameters to gain a better understanding of our system. We call this method of running many simulations the Monte Carlo method. Using the Monte Carlo method, we can ascertain information about the likelihood of certain events or approximate quantities that are hard to find analytically by running a large number of simulations and performing statistical analysis on the results. In this document, we will not dive into the methods of statistical analysis and the best ways to perform Monte Carlo simulations. This topic is very deep and is the focus of many peoples' whole careers.

However, even with only rudimentary analysis, Monte Carlo can prove very useful. We will use the example of Monte Carlo to motivate the need for fast simulations. Often, we run upwards of 1000 simulations and fast computation is necessary to perform this large number of simulations. In this section, we will dive into the various methods to optimize MATLAB code and best practices for flight dynamics.

6.6.2 Array and Function Optimizations

MATLAB is generally quite computationally efficient at computing arrays and matrices. After all, MATLAB is 'MATrix LABoratory'. However, we still need to be mindful of optimizations to our code.

It is best practice to only import what you need for a function or subroutine. For example, if you have wind data for all of the months out of the year, it is desirable to cull that down to the specific month before running the simulation.

Additionally, it is also beneficial for any imported data or long matrices to be passed into a function rather than called inside a function. This is especially important inside the numerical integration scheme! Running `readmatrix` inside of `ode45`

or whichever other numerical integration scheme you are using can slow down code upwards of 500% depending on the length of the matrices being read.

Instead, it is best to pass these arrays into the function. Performing `readmatrix` can be slow because it must check if the values are numeric and organize them into tables. By passing only MATLAB arrays, computations are much faster.

6.6.3 Flame Graphs for Optimization

One important method of optimization is the MATLAB profiler. The primary capability of the profiler is the flame graph. An example of the flame graph is shown in [Figure 6.5](#)

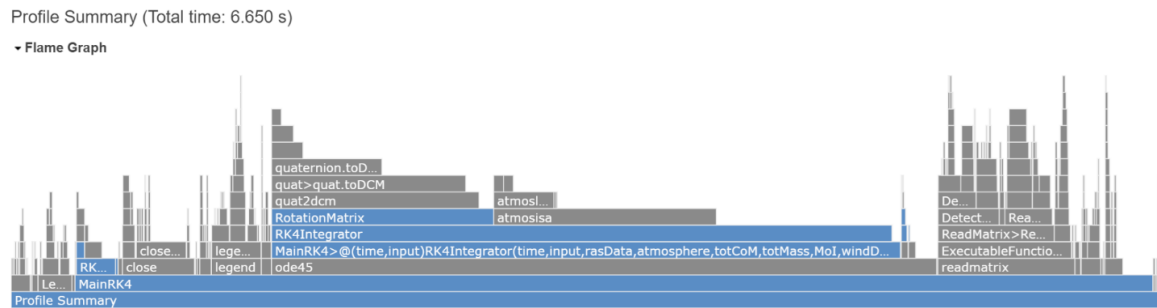


Figure 6.5: Flame Graph Showing Runtime of Individual Code Elements in MATLAB

The flame graph gives a visual representation of how long the code spends on each function. Functions in blue are those defined by the user. Those in grey are built in MATLAB functions.

Poorly optimized code will spend a long time running an excessive number of operations. In some cases, this is unavoidable. For example, `ode45`, which runs the RK4 integration scheme we discussed in [Section 6.3](#), must run for every time step and is likely to comprise a large part of simulation time.

However, other operations, such as `atmosisa`, should not comprise a large portion of simulation time. As seen in [Figure 6.5](#), calling the `atmosisa` function on every loop takes about 40% of the total time spent in ‘`RK4Integrator`’ function. By instead creating a table of the `atmosisa` data before running the simulation, we can reduce the runtime quite drastically. Implementing this simple change, we can see the reduction in simulation time as shown in [Figure 6.6](#)

Profile Summary (Total time: 3.112 s)

▼ Flame Graph

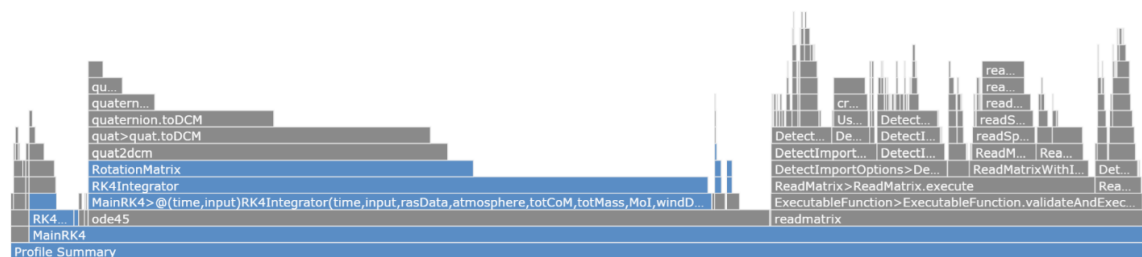


Figure 6.6: Flame Graph After Optimizations

6.7 Key Ideas

1. Numerical integration schemes of higher order can better approximate systems. Generally, a higher order will require more up front computation but will be more accurate and will take fewer steps to achieve a given accuracy
2. When numerically integrating functions with high frequencies or rapid changes, we need to be mindful of stiffness of the system and ensure to sample at at least the bandwidth of the signal.
3. Keep in mind what you need to calculate inside the integrator and what should be passed through. Calculations inside the integrator are very time expensive!

6.8 Notes and Further Reading

For more information and derivations on this section, refer to [23]. This source includes some more information than included here and some useful examples.

The MATLAB documentation for numerical integration schemes is quite good. For non-`ode45` integration, we recommend the following two documentation sites: [9] and [13]. Other documentation exists within MATLAB for other functions.

Notes

18. This is especially true for periodic functions. When working with periodic functions, make sure to sample well below the Nyquist limit.
19. This intuitive explanation is not entirely correct, see [\[23\]](#) for a full derivation using the Taylor Series. We also note that the technically correct way to denote this is $O(\Delta t^2)$, meaning the error is of order Δt^2

CHAPTER 7

The 6-DoF

In this section we will combine everything from the previous sections and explain how our 6-DoF rocket model works. We will follow a similar set up to [Section 4.3](#) here, describing the elements of the model in detail and then attaching relevant code either in-line or in the appendix.

Our definitions of our frames follow from those discussed in [Section 2.2](#). For the earth-centered inertial frame, we follow the conventions discussed in [Figure 2.2](#). Recall that the $\hat{x}$ axis points upward toward the zenith, the $\hat{y}$ axis points east along the surface, and the $\hat{z}$ axis finishes the right-handed coordinate system, pointing north.

For the body frame, we follow the convention in [Figure 2.4](#). This has $\hat{X}$ pointing through the nose of the rocket and $\hat{Y}$ and $\hat{Z}$ along the transverse plane. The directionality of $\hat{Y}$ and $\hat{Z}$ is not particularly important since the rocket is axially symmetric, so we arbitrarily select the $\hat{Y}$ axis to point through one of the fins and choose $\hat{Z}$ to complete the right-handed frame. For our Euler angles, we use a 3-2-1 rotation sequence. We define the zero angles to be when the body frame and the earth-centered inertial frame are coincident.

We note here that this is the section where we have the most room for improvement. We hope that you can translate your knowledge into better features for the 6-DoF as we note some of the simplifications we make in this section.

7.1 State Vector

Our state vector is very similar to that shown in [Section 4.3](#) section, but it bears repeating here because there are some important differences. Another thing to note is that some of the values of the state vector are altered for Monte Carlo simulations.

These possible changes, when applicable, are discussed.

We define our original position at the origin, giving us a position vector. It is also possible to define our initial position in another coordinate system, such as (lat, long, elevation). Generally it is good practice to set the origin to zero and add any offsets later to keep floating point errors in numerical integration small:

$$\vec{r}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

It is typical to initialize the simulation with zero velocity. In some instances, we may add an initial velocity if we are modeling a sounding rocket launched with an initial speed or similar for validation of our model:

$$\vec{v}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

We also need the initial orientation. Here, we use Euler angles for initialization. We often choose to launch at a slight angle to simulate adverse affects that may push the launch vehicle slightly off-course during the initial launch. We choose to add a small angle in the θ and ψ directions in this case:

$$\begin{bmatrix} \psi \\ \theta \\ \phi \end{bmatrix} = \begin{bmatrix} 0.1 \\ 0.1 \\ 0 \end{bmatrix} \quad (7.1)$$

We note that we could also parametrize this as a 45° rotation on the ϕ axis and then a 0.1 radian angle on either the θ or the ψ axis. For different scenarios one may be more useful than another. For example, if we want to simulate a 10° launch angle (from zenith), we can then use the ϕ angle to determine the azimuth at which we launch.

We use the command `eul2quat` with a specification of ZYX for the frame and chosen 3-2-1 rotation sequence. The specification is important now because we need to explicitly define our rotation sequence when two or more rotations are present. In the case of (7.1), this gives an the initial quaternion vector of :

$$\vec{q}_0 = \begin{bmatrix} 0.9975 \\ -0.0025 \\ 0.0499 \\ 0.0499 \end{bmatrix}$$

Lastly, we need to describe the initial angular velocity. We assume for most simulations that there is no angular velocity. For some cases, such as the simulation of *rail whip*, we may choose to put an angular velocity on the rocket.

$$\vec{\omega}_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \text{ rad/s}$$

Putting all of the together, we arrive at our state vector:

$$\vec{X}_0 = \begin{bmatrix} \vec{r}_0 \\ \vec{v}_0 \\ \vec{\omega}_0 \\ \vec{q}_0 \end{bmatrix}$$

7.2 Forces and Moments in the 6-DoF

The forces and moments on our rocket are what separate this case from the more simple case of the Dzhanibekov Effect in [Section 4.3](#). We have discussed most of these forces in [Figure 2.8](#), and in [Chapter 5](#), but the exact implementation into the 6-DoF model bears repeating, specifically relating to frame conversion.

7.2.1 Forces

We define 4 main forces that act on the vehicle, those being the 4 forces described in [Section 2.4](#). We will start with the most simple, which is gravity.

Gravity

Because we define the $\hat{x}$ direction as the zenith direction in the inertial earth frame, we define gravity as:

$$F_g = \begin{bmatrix} -mg \\ 0 \\ 0 \end{bmatrix}$$

For gravity, we note that because it acts through the center of mass of the rocket, we do not have a resultant moment to worry about.^{[20](#)}

Thrust

Thrust is also fairly simple to model. If we consider the thrust to be coincident to the longitudinal axis, the thrust is given by (2.8). For the magnitude, we consider a slightly more complex model than before. Because the rocket is changing altitude, the thrust of the engine will change as the pressure changes. The thrust of the engine, accounting for this pressure change, is given by:

$$F_T = \dot{m}V_e + (P_e - P_{atm}) \cdot A_e$$

Where $\dot{m}$ is the mass flow rate of the engine, V_e is the exit velocity of the engine, P_e is the exit pressure of the engine, P_{atm} is the atmospheric pressure, and A_e is the area of the nozzle exit. For a given propulsion system, these parameters are usually specified or can be easily derived, although we will not do so here. See [4] or the propulsion team for more details.

Here, we introduce the use of the rotation matrix to convert between different frames. MATLAB allows for the creation of a rotation matrix with the input of a quaternion using the `quat2dcm` function. Using this, we can perform frame conversion to and from the body and earth frame. We show this below in [Script 20](#):

Code Listing 7.1: Rotation Matrix

```
function [vecOut] = RotationMatrix(vecIn, quatIn,
    bodyOrEarth)
% PSP FLIGHT DYNAMICS:
%
% Title: DCM
% Author: Hudson Reynolds - Created: 7/2/2024
% Last Modified: 7/8/2024
%
% Description: This program takes inputs from the body or
% earth frame and
% converts it to the alternate frame given the euler
% angles using a
% rotation matrix.
%
%
% Inputs:
% vecIn = 3x1 column vector [x;y;z]
% angleIn = 3x1 vector with euler angle inputs (x-angle, y
% -angle, z-angle)
```

```

% quatIn = 4x1 vector with euler parameters
% bodyOrEarth = 0 -> convert earth to body, 1 -> convert
    body to earth
%
% Outputs:
% vecOut = 3x1 vector in the converted frame

    % make the input real for edge cases in Monte Carlo:
    rotIn = real(quatIn. ');

    % create the rotation matrix
    R = quat2dcm(rotIn);

    %perform transformation:
    if bodyOrEarth == 0
        vecOut = R * vecIn;

    elseif bodyOrEarth == 1
        vecOut = R.' * vecIn;
    else
        fprintf("Input for body or earth frame is invalid\n"
            n");
    end
end

```

Using this code, we can convert the thrust from the body frame into the earth frame through the function call `RotationMatrix(thrustForceBody, quat, 1)`. The input of 1 in this function means that the vector is transformed from the body frame into the earth frame. In our case, this means that we multiply by the transpose of the rotation matrix in the MATLAB notation convention.

Lift and Drag

Our other two forces follow in a similar manner. The quantities of lift and drag are heavily discussed in [Chapter 5](#), so we will keep them succinct here. The directions of lift and drag in the earth frame are described by (2.5) and (2.6). To convert these forces into the body frame, we use the function call `RotationMatrix(force, quat, 0)`. The specification of 0 in the function converts a force in the earth frame into

one in the body frame. We need the body frame force for computing the moments in the body frame, as discussed in [subsection 2.5.3](#).

Parachute Forces

The other force that sometimes acts on our rocket is the drag force imparted by the parachute(s). A parachute acts as another body, constrained to the vehicle body which it interacts with. To create this multibody simulation, all parachute drag forces are calculated in a separate function `calculateParachuteForce.m`. The form of the drag equation for the parachute is identical to the aerodynamic drag: $\frac{1}{2}\rho_{\infty}V_{\infty}^2SC_D$. The only difference between the two is the reference area and drag coefficients. For the latter, we use an experimentally determined value. For the former; however, we have to estimate the area of the fully inflated parachute. Luckily, it has been shown that the equation for the circle with a deform value scale factor of approximately 0.70 is adequate to model the reference area of the parachute.

In addition, it is necessary to consider the time delay of inflation, as the parachute will exponentially inflate as it is deployed. This means the equation for the area in our drag calculation is the following: $S = 0.7\pi(\frac{1}{2}D)^2(1 - e^{-.5t_{Drogue}})$. After the drag calculation, it is applied to the opposite of the freestream velocity. This force is only active when the vertical velocity is negative, so this is controlled by an `if/else` structure as: `if vel(1)< 0`. In addition, our vehicle has both a drogue and main chute. As such, another `if/else` structure controls the deployment of the main chute at a specified altitude. The parachute dynamics are far more complicated than this, but currently we do not model this added complexity.

Currently, there are no other forces that act in our 6DoF model. Internal forces and vibrations are not currently modeled, as we assume a completely rigid vehicle.

7.2.2 Moments

The modeling of moments in the 6-DoF is fairly straightforward. In the previous [subsection 7.2.1](#), we converted each of our forces into the body frame. Here, we will use these forces in the body frame to describe the moments acting on our vehicle.

The moment arm of the aerodynamic forces is equivalent to the distance from the

center of mass to the center of pressure.

$$M^c = \vec{r}_{cm} - \vec{r}_{cp}$$

Assuming that both lie along the longitudinal axis, we can express this moment as:

$$M^c = \hat{X}(x_{cm} - x_{cp})$$

Since gravity acts through the center of mass, the moment arm has length of 0 and no resultant moment exists. For thrust, since it is applied through the longitudinal axis, the $\vec{r} \times \vec{F}_T$ term is zero because $\vec{r}$ and $\vec{F}_T$ are parallel.

Thus, the only moments that are present are from the aerodynamic forces. The moment generated by the aerodynamic forces is:

$$M^c = \hat{X}(x_{cm} - x_{cp}) \times \vec{R}$$

Where $\vec{R}$ is the resultant aerodynamic force (the combination of both lift and drag).

7.2.3 Attitude Dynamics

These moments are used to find angular acceleration, $\vec{\alpha}$ using the Euler rigid body equations described in (2.14).

Lastly, the quaternion rates are described in the same as in [Section 4.3](#).

$$\dot{\mathbf{q}} = \frac{1}{2} \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \vec{q}$$

So, taking all of these together, we have the derivative of our state vector needed for numerical integration.

$$\dot{\mathbf{X}} = \begin{bmatrix} \vec{v} \\ \vec{a} \\ \vec{\alpha} \\ \dot{\mathbf{q}} \end{bmatrix}$$

7.2.4 6-DoF Code

We take this integrated state vector along with the initial conditions and integrate them using `ode45`. The two most important scripts in this process are the `MainRK4.m` and `RK4Integrator.m` MATLAB scripts. Because these MATLAB scripts are quite long, we have chosen not to include them in the appendix. Instead, a link to the GitHub is given where the scripts can be found. The full code can be found at [16] or in Appendix C.

7.3 Validation of 6-DoF

The validation of our 6-DoF model is achieved through modeling other rocket systems that have flown to check the validity of our model and modeling other physical phenomena. Validation can range from very simple to far more complex depending on the system that is being validated. For us, validation falls under two main categories:

1. Sub-component validation - validation of parts of the 6-DoF by modeling certain types of systems or doing hand calculations to verify correct behavior
2. Full model validation - validation of the whole 6-DoF by modeling a different rocket system and comparing 6-DoF outputs to real outputs.

Sub-component validation may look like the Dzhanibekov modeling in Section 4.3, where we test the attitude dynamics. Full model validation is usually more complex. One such example is our validation of our model against the V-2 rocket launch on March 7, 1949. This comparison is shown in Figure 7.1

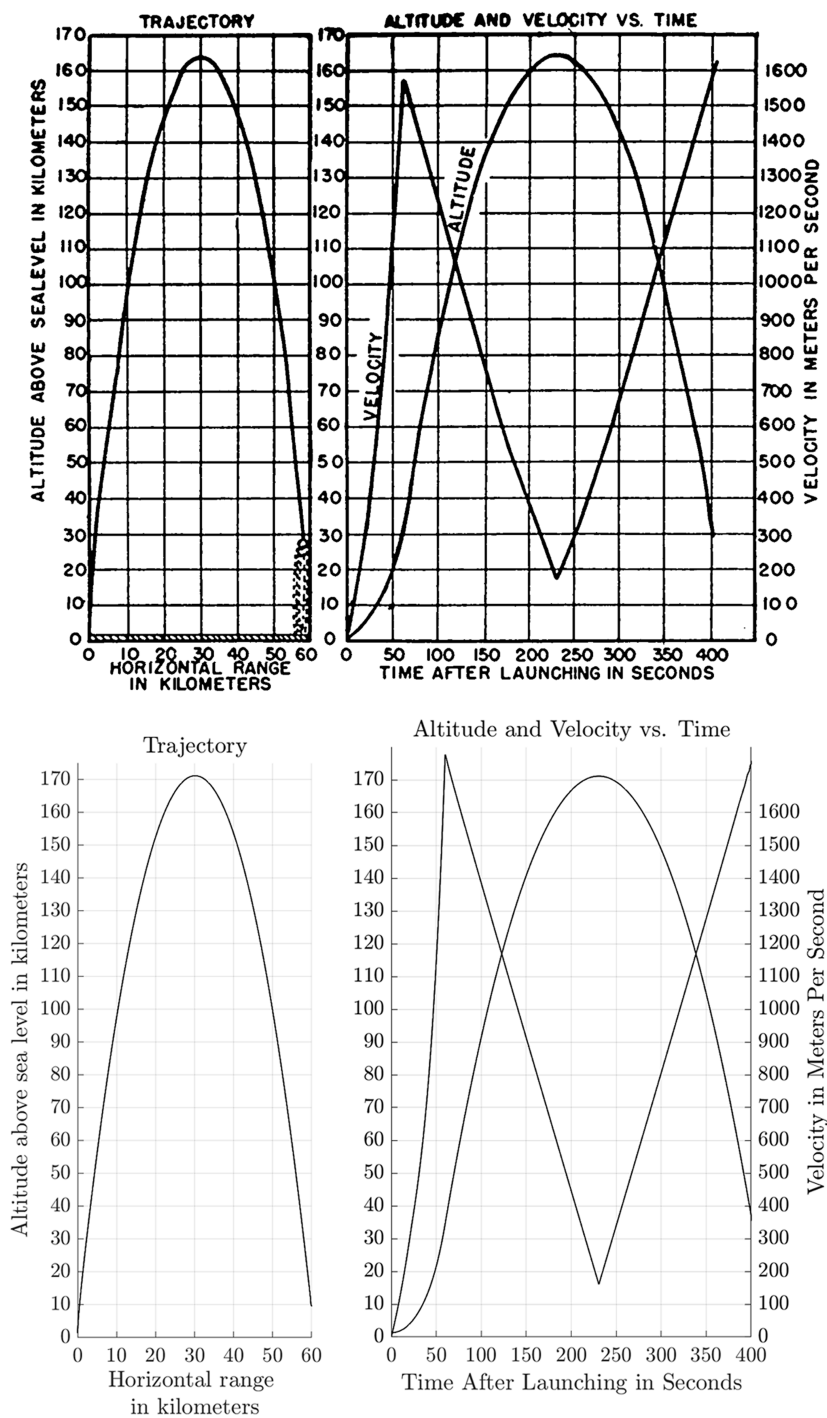


Figure 7.1: Comparison of Real V-2 Rocket Launch Data against 6-DoF Modeling

Notes

20. There is a slight difference between the center of mass and the center of gravity, which may produce a very small moment. In this instance, it is so small that we do not consider it.

CHAPTER 8

MATLAB and Coding Practices

In this final chapter, we discuss the techniques and practices for implementing a 6-DoF in MATLAB. We will also discuss more general practices for large MATLAB code bases and methods to keep these organized. As of Fall 2025, these practices are currently being implemented on our 6-DoF model. A large portion of the content in this chapter comes from what we have seen as practices adopted in industry for systems much more complex and larger than our own.

That being said, as our system grows in complexity, following these practices becomes increasingly important for organization. If the reader is working on their own large code base or contribute to our code base, it is a good idea to follow these practices as well to stay organized.

8.1 Object Oriented Programming in MATLAB

Public and private methods / properties

argument

Up until now, we have mainly been using a procedural approach to programming. The procedural approach uses multiple interacting functions to run the code. Each function takes in inputs and outputs. Together, these functions work together to run the code and built the final output. In most introductory coding classes, this is the type of programming that is taught.

This is a very good method for writing short segments of code, but for larger databases that need to have greater code reusability, there are better approaches. One such approach that is particularly powerful is Object-Oriented Programming (OOP).²¹

In OOP, the basis of our code is *objects*. These objects hold information of all types, whether that be numeric, string, or time data. The most important aspect of OOP is that it allows us to make user interaction with a large code base simpler, and also allows us to reuse large parts of the code.

If it has not already become apparent, the 6-DoF model described so far is fairly simple in the grand scheme of things. We are modeling a single rocket and most of the subsystems are simplified. What happens if we want to model a huge rocket system with different systems, sensors, measurements, errors, and so on? In this case, we need a better scheme for organization that makes the process simpler for the end user. OOP is one method to achieve such organization that we use for our model.

OOP, like any skill in coding, is best understood with practice. As a starting point, we highly recommend [21], which gives a guided introduction to OOP in MATLAB. It may take several repetitions and read-throughs to become familiar with the terminology of OOP.

8.1.1 Objects, Properties, and Classes

Objects

The basis of the OOP approach are, as the name suggests, objects. Objects hold much more information than standard variables. Typically, a variable contains only one piece of information, such as a string, an integer, or a floating point number. These variables have a value / values, and a type. Aside from this, they contain no other information. An object, on the other hand, can contain as much information as we want it to!

An object is a reusable part of code that has certain attributes and functions associated within it. An object may be a `rocket`, and it has features such as `propellant type`, `mass`, and `thrust`, to name a few. The rocket might also have some functions associated with it, such as `SimulateFlight`, which would simulate the rocket flight based on all the given inputs.

Classes

Objects are inherently related to what are called *classes*. A class defines the attributes and the functions that are associated with an object. The class is essentially a blueprint for what an object may look like.

The attributes associated with an object are called *properties* and the associated functions are called *methods*. Properties and methods are exactly analogous to the variables and functions seen elsewhere, but they are applicable only to objects of that class.

In MATLAB, these classes can be created by clicking the dropdown when creating a new file and selecting ‘Class’. The default class looks like:

Code Listing 8.1: Class Example

```
classdef untitled
    %UNTITLED Summary of this class goes here
    % Detailed explanation goes here

    properties
        Property1
    end

    methods
        function obj = untitled(inputArg1,inputArg2)
            %UNTITLED Construct an instance of this class
            % Detailed explanation goes here
            obj.Property1 = inputArg1 + inputArg2;
        end

        function outputArg = method1(obj,inputArg)
            %METHOD1 Summary of this method goes here
            % Detailed explanation goes here
            outputArg = obj.Property1 + inputArg;
        end
    end
end
```

A class often has a *constructor* method, which constructs an object of that class.

A constructor method is often useful to instantiate an object of a given class. In MATLAB, a constructor method takes the same name as the class itself. The class constructor and some of the subsequent ideas are best demonstrated with some example code:

Code Listing 8.2: Class Constructor Example

```

properties
    Name (1,1) string
    SamplingRate (1,1) double
    Variance double
end

methods
    function out = Sensor(name,samplingRate,variance)
        arguments
            name (1,1) string
            samplingRate (1,1) double
            variance = 0;
        end
        out.Name = name;
        out.SamplingRate = samplingRate;
        out.Variance = variance;
    end
end

```

In this code, the function **Sensor** is a class constructor method of the file **Sensor.m**. The class constructor takes user input for parameters such as a **name**, and **samplingRate** for the sensor, which are assigned within the function. These are then assigned to the properties of the object of the same name. If we were to call

Another useful property of classes which is demonstrated here is input validation. Input validation allows us to ensure that the input into a class or one of its methods is of a certain type. In this example, the **name** can only be a **string** of size 1 (scalar). Input validation allows us to build robust functions that only allow proper data to be passed through.

8.1.2 Methods

Methods are the backbone of OOP, and allow us to manipulate our objects and change their attributes in useful ways. In MATLAB, methods are somewhat distinct from functions and have some special features that we can use.

Most methods require that the first input to the method is an object of the class. This is required because methods are designed to only work on objects of that class. In MATLAB, there are two ways which a method of an object can be called. The first way is to input the object as the first input of the method:

```
Plotter(app)
```

The second way which this can be done is with the ‘dot indexing notation’ which looks like:

```
app.Plotter
```

Both of these are used interchangeably when calling a method, but when you write a method, you *must* use the first notation:

```
out = Plotter(app, input1, input2, etc.)
```

Aside from the requirement of an object as the first input, methods are written in the same way that standard functions are written in MATLAB. Further information about methods can be found in the MATLAB onramp.

Static Methods

Sometimes, we do not want to have to call a class constructor method or create an object of a certain class to employ a method. In this case, we can use a *static method*. A static method allows us to call a method without the object as an input.

A static method is associated with a class, but does not require a specific instance of that class to be run. Static methods are written by appending **Methods (static)** in the class:

```
classdef MyClass
    methods(Static)
        function p = pi(tol)
```

```
        [n d] = rat(pi,tol);  
        p = n/d;  
    end  
end  
end
```

8.1.3 Overloading and Precedence

One special feature of OOP is that it allows us to change the behavior of the default MATLAB functions when applied on certain objects. For example, overloading allows the user to change the functionality of the `plot` or `disp` functions when they are acting on a user defined object.

8.1.4 Subclasses and Superclasses

Classes have the feature that they can share portions of their code with other classes. When a class takes its base code from another class, this is referred to as *inheritance*. The class which is inheriting is the *subclass*, and the class which is being inherited from is the *superclass*.

The construct of the subclass and superclass is useful when code should be shared between different classes. When an overarching structure will be required multiple times, it is ideal to have a superclass which contains the basic structure, and subclasses which have the specific elements of that class.

8.1.5 Standards for Classes

Class structure in MATLAB is very powerful. If it is used inappropriately, though, it can make coding much more difficult. When you are building a class within MATLAB, you should consider the following:

- Does this functionality need a new class? Should it be a subclass of something else or a new method of an existing class?

Code Listing 8.3: Class Header

```
% Short 2-3 line explanation of the function
%
% Required
%
% <a href="documentation link">Documentation</a>
```

- What does the additional layer of abstraction help with? Is it simplifying the code structure or convoluting it?

There are some other considerations, but these questions should start to guide you on your implementation. Understanding these considerations and building a good class structure is oftentimes more difficult than writing the code! It will take practice to gain an understanding for the specific problem and how to build the class structure around that.

Classes in MATLAB carry lots of information. As a result, documenting them is quite critical for maintaining and improving their functionality. In our 6-DoF modeling, we require that all class definitions use the following header:

8.2 Continuous Integration and Testing

Continuous development is a principle that software development should occur in small pieces that are developed rapidly, rather than the more conventional approach of doing large overhauls of a code base. We find that the continuous integration approach is more useful for us, as it allows us greater flexibility in how we code and allows for features to be developed in parallel to each other.

In practice, this style of implementation can be difficult because it can be hard to check that each piece of code that is added to the database is properly written and constructed. To tackle this, continuous integration (CI) development employs automatic checks for the code to ensure quality standards and that the code being pushed has no errors.

This can be accomplished through the use of a program such as GitHub Actions, which checks code each time it is pushed to a new branch. We have set up a continuous

integration platform for our 6-DoF already, but more information about setting up such a system can be found [\[10\]](#).

It should be noted that GitHub Actions has a few limitations when using the built in Linux runner on GitHub. Firstly, the repository on which the information is stored must be public. Second, the runner only works for base MATLAB and not for any of the toolboxes such as Aerospace toolbox which we use quite frequently. For this reason, we use a self-hosted runner which allows us to avoid both of these issues.

8.2.1 Unit Testing

Unit testing works in combination with continuous integration practices and the object-oriented approach to provide the capability to test the functionality of MATLAB code.

8.3 Data Visualization

Here, we will briefly discuss data visualization and best practices for MATLAB. No matter how good your simulation is, you must be able to appropriately share data in a way that is not only easily conveyed to a non-expert audience, but also visually interesting and descriptive.

At the most basic level, your MATLAB plots should have proper formatting. The default MATLAB plot formatting isn't up to snuff with the standards of scientific publications or presentation of your work professionally. Luckily, MATLAB has many tools to improve the presentation of figures.

MATLAB has the ability for plots to utilize LaTeX formatting for figures. We highly recommend to enable this option for all plots as it allows for much better formatting on figures. This can be done with the following commands:

```
set(groot, 'defaultAxesTickLabelInterpreter', 'latex');  
set(groot, 'defaultLegendInterpreter', 'latex');  
set(groot, 'defaultTextInterpreter', 'latex')
```

8.3.1 Conventions for Plots

There are several conventions for plots which are generally followed in engineering practices. These will change depending on where your figures are to be published, but for our purposes, we use the following conventions:

- All plots should have clear axis labels. Units are typically included in brackets as $[m]$, in example of a length. Units should be SI base units wherever possible.
- Be knowledgeable of the use of color. Overuse of color may distract the viewer from the information being presented. Additionally, if your content is to be presented to a large audience, be mindful of those with color vision deficiencies. If your content may be printed, plots should be easily readable in black and white. Using different markers and line types is a good strategy.
- Always include a legend when plotting multiple data on a single graph.
- When the magnitudes of data are vastly different, a logarithmic axis or two y-axes are best practice.

8.3.2 Changing Default MATLAB Plotting

If you want to have these changes to figures universally applied in MATLAB, you can create a file in MATLAB called 'startup.m' that is run when MATLAB is opened. This file must exist in the user path for MATLAB. To find the user path for MATLAB, the command `userpath` can be typed into the command line. This will output something like:

```
C:\Users\YourName\OneDrive\Documents\MATLAB
```

The code in [Script 8.3.2](#) is the default plotting used in this document, which should be a good baseline for professional plots.

Code Listing 8.4: Startup File for Plotting

```
% set linewidth
set(groot, 'defaultLineLineWidth', 1.5)

% set the interpreter to latex
set(groot, 'defaultAxesTickLabelInterpreter', 'latex');
set(groot, 'defaultLegendInterpreter', 'latex');
```

```
set(groot, 'defaultTextInterpreter', 'latex');

%change font sizes
set(groot, 'defaultLegendFontSize', 12)
set(groot, 'defaultTextFontSize', 12)
set(groot, 'defaultAxesFontSize', 10)

% grid on and box off
set(groot, 'defaultLegendBox', 'off');
set(groot, 'defaultAxesXGrid', 'on');
set(groot, 'defaultAxesYGrid', 'on');
```

And with that, we have completed the first Volume of the Flight Dynamics Bible. Additional information and code can be found in the Appendix. If you have any questions, concerns, or suggestions, feel free to reach out to Hudson Reynolds or Preston Wright on Slack or via email at Hudsonj.reynolds@gmail.com and preston.wright0330@gmail.com.

Notes

21. There exists some debate whether OOP is a superior method of coding. Of course, the true answer is nuanced and depends on the application. Object-Oriented programming, when done poorly, can make code quite unreadable. However, proper implementation of OOP can be extremely useful. See [\[3\]](#) for more information.

APPENDIX A

Answers to Selected Problems

A.1 Chapter 2 Solutions

2-1

- (a) We can derive the EOM of the system using either the Lagrangian or Newtonian methods. The Lagrangian method is detailed in [Appendix 3](#), so it will not be covered here. The EOM of the system is derived by considering the force balance of the system:

$$\sum \vec{F} = m\vec{a}$$

In this problem, motion is constrained to only the $\hat{x}$ -direction. We also know the spring force is of the form:

$$F_S = -k(L - L_0)\hat{x}$$

We can substitute F_S as the force, and $m\vec{a}$ remains on the RHS of the equation:

$$ma_x = -k(L - L_0)$$

If we choose L_0 to be the origin of the x -coordinate, we can rewrite the equation quite simply as:

$$m\ddot{x} + kx = 0$$

Finally, we typically divide each term by the mass:

$$\ddot{x} + \frac{k}{m}x = 0$$

Plugging in the given values of m and k into the equation will give us the final equation:

$$\ddot{x} + 4x = 0$$

- (b) The analytical solution of this equation is quite easy to find. There are a few methods we can take, but the easiest is to create a characteristic equation (assuming the solution to take form $x = e^{rt}$). This allows us to turn the differential equation into an algebraic equation.¹

$$r^2 + 4 = 0$$

We can solve this equation in terms of r :

$$r = \pm 2i$$

Remembering that we chose $x = e^{rt}$ as the assumed solution, we have:

$$x(t) = C_1 e^{2it} + C_2 e^{-2it}$$

Using the Euler identity, we may rewrite this in terms of trigonometric functions (although you are free to input this into MATLAB as is).

$$x(t) = A \cos 2t + B \sin 2t$$

Finally, we need initial conditions. The initial position, $x(0) = 0$, and for velocity, we may use the impulse-momentum theorem:

$$J = m\Delta v$$

As a result, the initial velocity must be 10 m/s in the $-\hat{x}$ -direction, or $\dot{x}(0) = -10$.

We can use these two initial conditions to find the particular solution of the differential equation:

$$x(t) = -5 \sin(2t)$$

This may be plotted as any function in MATLAB would be, which is shown in part C

- (c) Parts C and D are shown in the same script:

¹A more robust method, known as the Laplace transform, will be explored in Volume III.

Code Listing A.1: Mass Spring Damper Solution

```

% PSP FLIGHT DYNAMICS:
% Title: Problem 2-1 Script
% Author: Hudson Reynolds
% Last Modified:9-6-2025
%
% Description: This script runs a mass spring
               damper simulation. The
% analytical solution is compared against the
               numerical integration.
clear;clc;close all

% Write the analytical solution:
t = linspace(0,10,300);
xAnal = -5*sin(2*t);

% Run the ODE45 integration for the no damper
    scenario

% pos, vel
x0 = [0;-10];
[t,xNum] = ode45(@(t,x)integrator(t,x), t, x0);

% run the ODE45 for the damper situation:

% pos, vel
x0 = [0;-10];
[t,xNumDamp] = ode45(@(t,x)integrator2(t,x), t, x0)
    ;

% plot:
figure
sgtitle('Mass Spring Damper Simulation');
subplot(2,1,1)
plot(t,xAnal)
hold on;
plot(t, xNum(:,1), '--');
title('Undamped MSD')
xlabel('Time (s)');

```

```

ylabel('Position (m)');
legend('Analytical Solution', 'Numerical Solution',
       'Location','best');
hold off;
yLims = ylim;

subplot(2,1,2)
plot(t,xNumDamp(:,1))
ylim(yLims)
title('Damped MSD')
xlabel('Time (s)');
ylabel('Position (m)');

function out = integrator(t,x)
vel = x(2);
accel = -4*x(1);

out = [vel;accel];
end

function out = integrator2(t,x)
vel = x(2);
accel = -4*x(1) - x(2);

out = [vel;accel];
end

```

A.2 Chapter 4 Solutions

4-2

To determine if the soldiers will be knocked over by the force of the engine, we need to convert that thrust force from being expressed in the body frame to the inertial frame. To do this, we will take our known rotation sequence and corresponding Euler

angles, use them along with MATLAB's built in function to construct the DCM, and then transpose that DCM in order to convert from body to inertial coordinates. Finally, we will take the magnitude of the components acting in the xy-plane and compare that to the known maximum allowable force.

Doing all of this, we find that the force acting in the inertial xy-plane is 1574 N, which exceeds our maximum value of 1000 N. This means the soldiers will indeed go tumbling away as you go in to land. The code used to perform these calculations is shown below:

Code Listing A.2: Huey Landing Solution Code

```
% PSP FLIGHT DYNAMICS:
%
% Title: Problem 4-2 Script
% Author: Preston Wright
% Last Modified: 9-6-2025
%
% Description: This script computes a frame conversion of
%              the thrust force to determine the component acting in
%              the inertial xy-plane.
clear;clc;close all

% Initialize known variables
thrustBody = [0;0;10000]; % Thrust force in body
                        coordinates [N]
maxForce = 1000;          % Max force that can act in xy
                        -plane [N]
psi = 0;                  % Yaw angle/z-axis angle [rad]
theta = 0.15;              % Pitch angle/y-axis angle [
                        rad]
phi = -0.05;              % Roll angle/x-axis angle [rad
                        ]

% Convert the thrust force to be expressed in inertial
                        coordinates
DCM = angle2dcm(psi,theta,phi,"ZYX");
DCM = DCM';
thrustInertial = DCM*thrustBody;

% Calculate the magnitude of the force acting in the xy-
```

```
plane
thrustPlane = sqrt(thrustInertial(1)^2+thrustInertial(2)
^2);

% Compare that magnitude to see if it exceeds the maximum
allowable force
if thrustPlane > maxForce
    fprintf("The thrust exceeds the maximum allowable
force and the soldiers will be blown away!\n")
else
    fprintf("The thrust is below the maximum allowable
force and the soldiers will be fine.\n")
end
```

APPENDIX B

Proofs and Derivations

It is often useful, especially for mathematical reasoning, to see more exact derivations. We have included several of the most useful derivations in this appendix.

B.1 Buckingham Pi

Variable	Dimension
ρ	$\frac{M}{L^3}$
V	$\frac{L}{T}$
S	L
μ	$\frac{M}{LT}$
a	$\frac{L}{T}$
α	—

For Buckingham Pi Theorem, we want to select repeating variables that span the base dimensions and do not want to select two variables that have the same dimensions. Somewhat arbitrarily, we select ρ , V , and S as the 3 repeating variables.

With each Π group, we solve by equating the exponents of the expressions to zero. For example:

$$\Pi_1 = \rho^a V^b S^c \mu = M^0 L^0 T^0$$

Plugging in the dimension of each of these:

$$\Pi_1 = \left[\frac{M}{L^3} \right]^a \left[\frac{L}{T} \right]^b [L]^c \left[\frac{M}{LT} \right] = M^0 L^0 T^0$$

This yields a system of equations with three equations:

$$\begin{aligned} M : a + 1 &= 0 \\ L : -3a + b + c - 1 &= 0 \\ T : -b - 1 &= 0 \end{aligned}$$

Solving this system gives $a = b = c = -1$. Plugging in these values for the coefficients, we arrive at:

$$\Pi_1 = \frac{\mu}{\rho V S}$$

Which is $\frac{1}{Re}$. The same analysis is done for the rest of the Π groups:

$$\begin{aligned} \Pi_1 &= \frac{\rho V S}{\mu} \\ \Pi_2 &= \frac{V}{a} \\ \Pi_3 &= \frac{F}{\rho V^2 S} \\ \Pi_4 &= \alpha \end{aligned}$$

B.2 Equivalence of Newtonian and Lagrangian Mechanics

Proof. We start with Lagrange's equations in rectilinear coordinates $x_i = x, y, z$:

$$\frac{\partial L}{\partial x_i} - \frac{d}{dt} \frac{\partial L}{\partial \dot{x}_i} = 0$$

We know that the Lagrangian is given as $L = T - U$, so we can express the Euler-Lagrange equations as:

$$\frac{\partial(T - U)}{\partial x_i} - \frac{d}{dt} \frac{\partial(T - U)}{\partial \dot{x}_i} = 0$$

We know that the translational kinetic energy is a function of the velocity only, so $\frac{\partial T}{\partial x_i} = 0$. A conservative potential is a function of the position only, so $\frac{\partial U}{\partial \dot{x}_i} = 0$.

This simplifies the Euler-Lagrange Equation to:

$$-\frac{\partial U}{\partial x_i} = \frac{d}{dt} \frac{\partial T}{\partial \dot{x}_i}$$

For a conservative system, $-\frac{\partial U}{\partial x_i} = F_i$, because the force is the gradient of the potential function.

We know the kinetic energy is given as $\frac{1}{2}m\dot{x}^2$. The derivative with respect to the velocity $\dot{x}$ is $m\dot{x}_i$. Taking the time derivative, we have $\frac{d}{dt}(m\dot{x}_i) = m\ddot{x}_i$ in the case of constant mass (or more generally, $\frac{d}{dt}(m\dot{x}_i) = \dot{p}_i$).

So, we have $F = m\ddot{x}_i$.

□

Corollary 2.2. *From the proof, we can also see that the validity is dependent on the choice of rectangular coordinates.*

It can further be shown that the Newtonian derivation must be performed in an inertial frame.

APPENDIX C

Code

This chapter contains lengthier segments of example code from either the 6-DoF itself or a proof/example relating to it. Since the code is cumbersome to show in full, we have opted to link to the GitHub repository instead. The code for the 6-DoF can be found at [\[16\]](#) or accessed via GitHub with the QR code. This GitHub also contains the most up-to-date version of this book in .pdf and .tex format:



The rest of the code shown below link to certain sections of this GitHub repository and are reproduced only for convenience.

C.1 Lorenz Attractor RK4 Example

[Lorenz Attractor](#)

C.2 Dzhanibekov Effect Model

[Dzhanibekov Effect](#)

C.3 Numerical Integration Error Examples

[Numerical Integration Examples](#)

Special Terms

action

The time integral of energy is known as the action, which is often denoted S .
44, 60, 67

angle of attack

The angle of offset of the freestream velocity with the $\hat{X}/\hat{b}_1$, or longitudinal axis, of the vehicle. This is often denoted with the Greek letter α . 18, 23, 24, 84–86, 88, 90

axisymmetric

An object in which there is rotational symmetry about one or more of the axes.
89

bandwidth

The bandwidth of a signal is the difference between its upper and lower frequencies. In our context of numerical integration, this bandwidth refers to twice the highest frequency of a function. 105, 111

beat frequency

A beat frequency is the result of constructive and destructive interference between two waves of a similar frequency. The similarity of the frequencies creates variations in the amplitude of the resultant wave. 105

BKE

Basic Kinematic Equation. 19, 22, 23, 31, 35, 47, 54, 56, 57

body frame

A reference frame that is fixed to the body of an object and follows the translations and rotations of the vehicle. 16–19, 26–31, 35, 66, 67, 69–72, 74, 115, 119, 120

Buckingham Pi Theorem

The Buckingham Pi Theorem is a method to non-dimensionalize parameters of a system. In a system with N parameters and K physical dimensions, it is possible to express $P = N - K$ dimensionless groups. 84–86, 145

CFD

Computational Fluid Dynamics. 90, 91

CI

continuous integration. 133

conservative

A system or potential which is independent of the path. A closed integral in a conservative field is 0. 45, 57, 59, 146, 147

difference equation

A discretized form of a differential equation, with the continuous time properties replaced by discrete time intervals.. 10

dimensional analysis

The process of determining information from a physical equation by considering the dimensions, or units, of quantities in the equation. 84

DoF

Degree of Freedom. xi, xiii, xv, xvi, 1, 3–5, 7, 8, 14, 15, 17–19, 22, 23, 26, 37, 39, 41, 42, 48, 50, 55, 86, 89, 106, 115, 120, 122, 128, 133, 134, 149

drag coefficient

A non dimensional parameter that describes the amount of drag acting on an object. This non-dimensionalized value is given by $\frac{F_D}{q_\infty S}$, where q_∞ is the dynamic pressure, $\frac{1}{2}\rho_\infty V_\infty^2$. 8, 90

Dzhanibekov Effect

A phenomenon in which rotation about an intermediate axis results in rotation about another axis. This phenomenon occurs with any body where the three PMOIs are not equal. It is easily seen with a tennis racket or a cell phone. 73, 117, *see* PMOI

EOM

Equation of Motion. 9, 12, 35–37, 41, 42, 45, 47, 54–57, 59, 61, 62, 77, 139

Euler angles

A group of three scalar values used to describe the orientation of a rigid body in space. 65–67, 69, 70, 74, 77, 115, 116

Euler rates

A way to describe rotational rates with orientation for 3-dimensional rotations, they are denoted by the dot of the corresponding Euler angle: $(\dot{\psi}, \dot{\theta}, \dot{\phi})$. 67, 69, 70

Euler's equation

The fundamental equation in the calculus of variations describing the maximum or minimum conditions of a functional. 43, 58, *see* functional

Euler-Lagrange Equation

The fundamental equations describing the solutions that give the extrema of the action in a Lagrangian system. 44, 50, 52, 147, *see* action

freestream

A freestream quantity is one that is far away from the body, such that the influences of the body are negligible. For example, freestream velocity is the velocity that would be seen outside the influence of the body at the same conditions. 8, 23, 39, 86, 87, 89, 153

function handle

A pointer to an instance of a function in MATLAB. A function handle essentially stores a function call in a variable in MATLAB. 99, 100, 103

functional

A quantity that maps a function onto the real number line. Often, this is called a 'function of a function'. 43, 44

generalized momenta

A generalized coordinate that is defined as $\frac{\partial L}{\partial \dot{q}_i}$. 56, 57

generalized velocity

A free direction of movement for a vehicle. These can be translational or rotational directions. 52

geodesic

The shortest path between two points in an arbitrarily curved space. 36, 43

holonomic constraint

A constraint on a Lagrangian system which is a function of only the position of the particles. 48, 49, 51, *see* Lagrangian

holonomic system

A system that is entirely composed of holonomic constraints. 51, *see* holonomic constraint

inertia tensor

A matrix that describes the moment of inertia along each of the axes of a body. This matrix, often denoted I , transforms the angular velocity, ω into an angular momentum, H . 29, 30, *see* moment of inertia

inertial frame

A reference frame that is static and experiences no accelerations. We use the Earth's surface as the reference frame for our rocket. 15–17, 23, 26–28, 35, 39, 47, 53, 54, 66, 67, 69–72, 74, 115

Lagrangian

A description of the energy of a system given by $L \equiv T - U$. Denoted with the letter L . 41, 43–47, 52, 53, 55–60, 63, 77, 108, 139, 146

moment of inertia

A measure of its resistance to rotational acceleration. It is in essence, the rotational analogue of mass. Often denoted with the letter I . 27–30

non-holonomic constraint

A constraint that depends on more than the position of the particles in the system. 51, 52, *see* holonomic constraint

numerical integration

The process of making a numerical approximation of a differential equation through discretization of the independent variable (usually time for most physical phenomenon). 9, 36, 39, 57, 69, 72, 95, 96, 100, 103–111, 116, 121

Nyquist rate

The rate at which a signal must be sampled to be free of distortions and aliasing. This rate is typically twice the highest frequency present in the signal. 105

OOP

Object-Oriented Programming. 127, 128, 131, 132, 137

orthonormal

A set of basis vectors that are mutually orthogonal to each other in a vector space and have unit length. We mostly refer to orthonormal vectors in $\mathbb{R}^3$ space. 14, 15

parallel axis theorem

A theorem relating the moment of inertia and product of inertia about different rotation points. 30, *see* moment of inertia

PMOI

principle moment of inertia. 30, 31

principal axes

The axes defined such that the products of inertia for a body are zero. These axes are the values that solve the eigenvalue problem $I\vec{\omega} = I^*\vec{\omega}$. 29–31

principle of least action

A form of Hamilton's Principle that states that the action of a system should be minimized in a physical mechanical system. 44, *see* action

quaternion

An extension of the complex numbers with elements j and k added. The common form is expressed $q_1 + q_2i + q_3j + q_4k$ (our notation) or $q_1i + q_2j + q_3k + q_4$. The algebra of quaternions is generally denoted $\mathbb{H}$. 65, 69–74, 76, 77, 116, 118, 121

rail whip

A phenomenon where the flexibility of the launch rail causes a large moment to be exerted on the rocket during launch. 117

reference area

An area that is used to define a characteristic area of an object in aerodynamics. This is used to normalize quantities such as lift and drag. 8

RHR

right hand rule. 15, 21, 26, 89

rigid body

A rigid body is a system of particles whose positions with respect to each other is fixed. Rigid body dynamics are the basis of our analysis of rotational dynamics. 27, 48, 50, 57–59, 121

roundoff error

Error in numerical integration that comes from inaccuracies in the floating point precision of computers. This is worsened by a larger number of timesteps, as this error will compound. 97, 98

state vector

A vector containing information about the position and velocity of a body at a given time. This vector can also include more information, such as the angular velocity and quaternion. Generally, this vector includes all quantities needed to be integrated to find the new state. 5, 16, 18, 26, 27, 73, 74, 76, 77, 100, 103, 115, 117, 121, 122, *see* quaternion

stiff

A differential equation that is hard to numerically integrate as a result of instability when larger timesteps are used. Stiffness is a property of the differential equation itself, and not the solution. Stiff systems are not completely well defined, but generally follow from this definition. 106, 108

symplectic integrator

A class of numerical integration schemes that are energy preserving because they preserve certain geometric properties of a differential equation. 56, 108

truncation error

Error in numerical integration that comes from the inaccuracies of the integration scheme in modeling the true behavior of the function. This error is related to the number of higher order terms ignored. 97, 106, 108

wind frame

A reference frame in reference to the direction of freestream velocity, denoted V_∞ . 18, *see* freestream

References

- [1] John Anderson. *Fundamentals Of Aerodynamics 6 Edition*. 6th ed. Mc-Graw Hill, 2017. 1154 pp. ISBN: 978-1-259-12991-9. URL: <http://archive.org/details/fundamentals-of-aerodynamics-6-edition> (visited on 02/23/2025).
- [2] Daniel W. Baker and William Haynes. *Statics: Products of Inertia*. 2020. 419 pp. URL: <https://engineeringstatics.org/products-of-inertia.html> (visited on 02/25/2025).
- [3] Brian Will. *Object-Oriented Programming is Bad*. Jan. 18, 2016. URL: <https://www.youtube.com/watch?v=QM1iUe6IofM> (visited on 08/28/2025).
- [4] Marcelo Dasilva. *Chemical Equilibrium with Applications*. Glenn Research Center | NASA. Apr. 17, 2023. URL: <https://www1.grc.nasa.gov/research-and-engineering/ceaweb/> (visited on 03/04/2025).
- [5] Paul Dawkins. *Differential Equations*. Paul's Online Math Notes. June 26, 2023. URL: <https://tutorial.math.lamar.edu/Classes/DE/DE.aspx> (visited on 03/06/2025).
- [6] Hugo Filmer. *Fluid Systems Bible*. URL: <https://www.overleaf.com/project/67e4548eec2d615a2e90fce1> (visited on 04/12/2025).
- [7] Gregory Gundersen. *Understanding Moments*. Understanding Moments. Apr. 11, 2020. URL: <https://gregorygundersen.com/blog/2020/04/11/moments/> (visited on 02/23/2025).
- [8] N. Jeremy Kasdin and Derek A. Paley. *Engineering dynamics: a comprehensive introduction*. Princeton, NJ: Princeton University Press, 2011. 1 p. ISBN: 978-0-691-13537-3. URL: https://students.aiu.edu/submissions/profiles/resources/onlineBook/g2B5P5_Engineering_Dynamics_A_Comprehensive_Introduction.pdf.
- [9] Mathworks. *Choose an ODE Solver*. 2024. URL: <https://www.mathworks.com/help/matlab/math/choose-an-ode-solver.html> (visited on 03/08/2025).
- [10] Mathworks. *Continuous Integration with MATLAB on CI Platforms - MATLAB & Simulink*. 2025. URL: https://www.mathworks.com/help/matlab/matlab_prog/continuous-integration-with-matlab-on-ci-platforms.html (visited on 07/13/2025).

- [11] Mathworks. *eul2quat*. *eul2quat*. URL: <https://www.mathworks.com/help/robotics/ref/eul2quat.html> (visited on 03/31/2025).
- [12] Mathworks. *quat2dcm*. URL: <https://www.mathworks.com/help/aerotbx/ug/quat2dcm.html> (visited on 03/31/2025).
- [13] Mathworks. *Summary of ODE Options*. 2024. URL: <https://www.mathworks.com/help/matlab/math/summary-of-ode-options.html> (visited on 03/08/2025).
- [14] Ashwin Narayan. *How to Integrate Quaternions*. Ashwin Narayan. Sept. 10, 2017. URL: <https://www.ashwinnarayan.com/post/how-to-integrate-quaternions/> (visited on 03/31/2025).
- [15] Eqab M Rabei, Tareq S Alhalholy, and A A Taani. “On Hamiltonian Formulation of Non-Conservative Systems”. In: (2004), p. 10. URL: <https://journals.tubitak.gov.tr/cgi/viewcontent.cgi?article=1763&context=physics> (visited on 03/18/2025).
- [16] Hudson Reynolds. *HudsonReynolds/Flight-Dynamics-Bible*. URL: <https://github.com/HudsonReynolds/Flight-Dynamics-Bible> (visited on 03/30/2025).
- [17] Charles Rogers and David Cooper. *RasAero*. Rogers Aeroscience RASAero Aerodynamic Analysis and Flight Simulation Software. 2019. URL: <https://www.rasaero.com/> (visited on 03/22/2025).
- [18] Shane Ross. *Lagrange’s Equations Intro, Generalized Coordinates, Constraints, Degrees of Freedom, Lecture 17*. URL: <https://www.youtube.com/watch?v=g9QD43P3v8&list=PLUeHTafWecAUl2DuWWdRU1MckJv7M5LEH&index=17> (visited on 03/22/2025).
- [19] W. Shyy et al. “Recent progress in flapping wing aerodynamics and aeroelasticity”. In: *Progress in Aerospace Sciences* 46.7 (Oct. 1, 2010), pp. 284–327. ISSN: 0376-0421. DOI: [10.1016/j.paerosci.2010.01.001](https://doi.org/10.1016/j.paerosci.2010.01.001). URL: <https://www.sciencedirect.com/science/article/pii/S0376042110000023> (visited on 03/07/2025).
- [20] Petra Svickova. *Air Density Changes Due to Weather*. BARANI. Mar. 1, 2020. URL: <https://www.baranidesign.com/faq-articles/2020/2/20/air-density-changes-due-to-weather> (visited on 02/23/2025).
- [21] Matt Tearle. *Object-Oriented Programming Onramp / Self-Paced Online Courses - MATLAB & Simulink*. MATLAB. URL: <https://matlabacademy.mathworks.com/details/object-oriented-programming-onramp/oroop> (visited on 08/28/2025).

- [22] Stephen Thornton and Jerry Marion. *Classical Dynamics Of Particles And Systems Thornton*. 5th ed. URL: <http://archive.org/details/ClassicalDynamicsOfParticlesAndSystemsThornton> (visited on 03/18/2025).
- [23] William Trench. *3.1: Euler's Method*. Mathematics LibreTexts. July 20, 2020. URL: https://math.libretexts.org/Courses/Community_College_of_Denver/MAT_2562_Differential_Equations_with_Linear_Algebra/03%3ANumerical_Methods/3.01%3AEuler's_Method (visited on 02/23/2025).
- [24] *Variational Principles in Classical Mechanics (Cline)*. Physics LibreTexts. Nov. 11, 2017. URL: [https://phys.libretexts.org/Bookshelves/Classical_Mechanics/Variational_Principles_in_Classical_Mechanics_\(Cline\)](https://phys.libretexts.org/Bookshelves/Classical_Mechanics/Variational_Principles_in_Classical_Mechanics_(Cline)) (visited on 03/18/2025).
- [25] Veritasium. *The Closest We've Come to a Theory of Everything*. Oct. 29, 2024. URL: https://www.youtube.com/watch?v=Q10_srZ-pbs (visited on 03/18/2025).
- [26] Carl Wassgren. *Buckingham Pi*. URL: https://engineering.purdue.edu/~wassgren/teaching/ME30800/NotesAndReading/DimensionalAnalysis_BuckinghamPi_MethodOfRepeatingVariables_Reading.pdf (visited on 03/16/2025).